Python for computational Problem Solving · Unit_02 · Unit 2: Collections & Functions

DRAFT — UE26CS151A · Unit_02 — Unit 2: Collections & Functions · Complete Unit Master

Source-complete searchable master · transcript, slide text, page visuals and archive references

Hugging Face PESU_Materials source tree, official units.json catalogue, transcript sidecars, slide text/PDF/PPT sources and the selected First_Year_8enterprise archive; see the provenance datasets in the release ZIP. Generated 18 October 2026.

· How to use this pack

Search this single unit master by lesson, timestamp, slide text or source page. Read the transcript in timestamp order, then compare extracted slide text with its page-faithful source figure. Formula candidates and archive question mappings are explicitly flagged for review. Topics include the applicable parts: overview, key terms, formulas, figures, flowcharts, important points, worked examples, past-paper questions and a recap. IDs are shared with the separate documents: E = formula, G = graph, F = flowchart, W = worked example. Each ID is numbered by topic, so E4.2 is the second formula of topic 4.

Source text	Verbatim or OCR-derived text, kept separate from editorial notes.
Candidate	Automated topic/source association requiring manual confirmation.
Low ASR confidence	Transcript segment requires audio verification.
DRAFT	Human review remains incomplete.

Contents

1. 1 · UE26CS151A · Unit_02: Unit 2: Collections & Functions
2. 2 · Lists and Tuples
3. 3 · Dictionary
4. 4 · Files
5. 5 · Function_Call_Definition_Variable_arguments
6. 6 · Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets
7. 7 · Source slide deck: JSON
8. 8 · Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args
9. 9 · Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args
10. Appendix A · Symbol glossary
11. Appendix B · Sources, coverage and gaps
12. Appendix C · Revision and retrieval practice

M1

Unit_02 · Unit 2: Collections & Functions

1 UE26CS151A · Unit_02: Unit 2: Collections & Functions

This unit master organizes source material for Python for computational Problem Solving, Unit_02, officially titled Unit 2: Collections & Functions. It includes 4 video transcripts and 8 source slide decks, with page-level text and rendered images where available. This draft preserves source wording and page order; it does not claim a complete human subject review or full audiovisual review. EV0000001

SHORT-NOTE RECAP

Official catalogue entry: UE26CS151A, Unit_02 (Unit 2: Collections & Functions). Bucket folder present: yes. Videos: 4; slide decks: 8.

EV0000001

2 Lists and Tuples

This lesson is indexed from the source video titled Lists and Tuples. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000002 · EV0000003

Graphs and diagrams

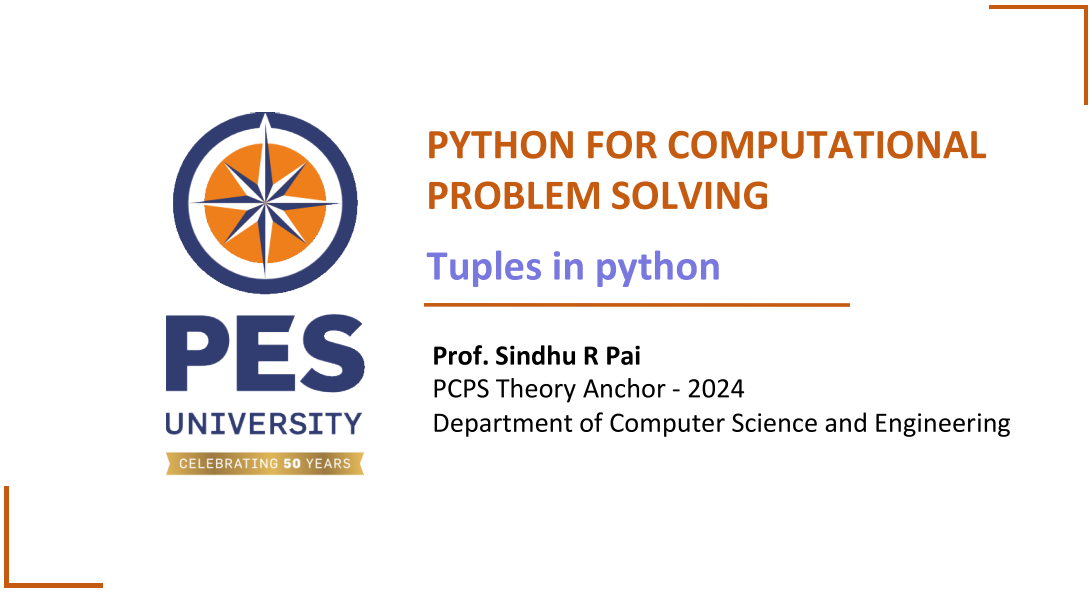
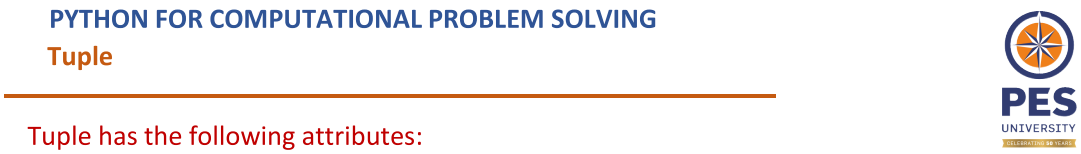


Figure G000001. UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples · source slide page 1.

EV0000079

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.



Tuple has the following attributes:

- The element of the tuple can be of any type.

(There is no restriction on the type of the element.). A tuple is same as list, except that the **set of elements is enclosed in parentheses instead of square brackets and these elements are immutable.**

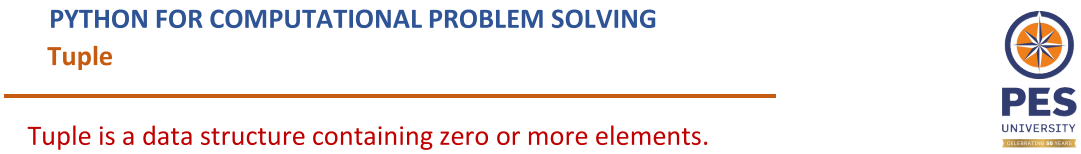
Ex:

```
t = ('python',1,3.4) # heterogeneous
t1 = (11,22,33,44) #homogeneous
```

Figure G000002. UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples · source slide page 2.

EV0000082

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.



Tuple is a data structure containing zero or more elements.

Creation of tuple

- An empty tuple can be created using a constructor,
  tuple () or ().
- Tuple with two elements.
  t3 = (1, 2)
- Tuple with a single element.
  t3=(1,)

Figure G000003. UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples · source slide page 3.

EV0000085

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

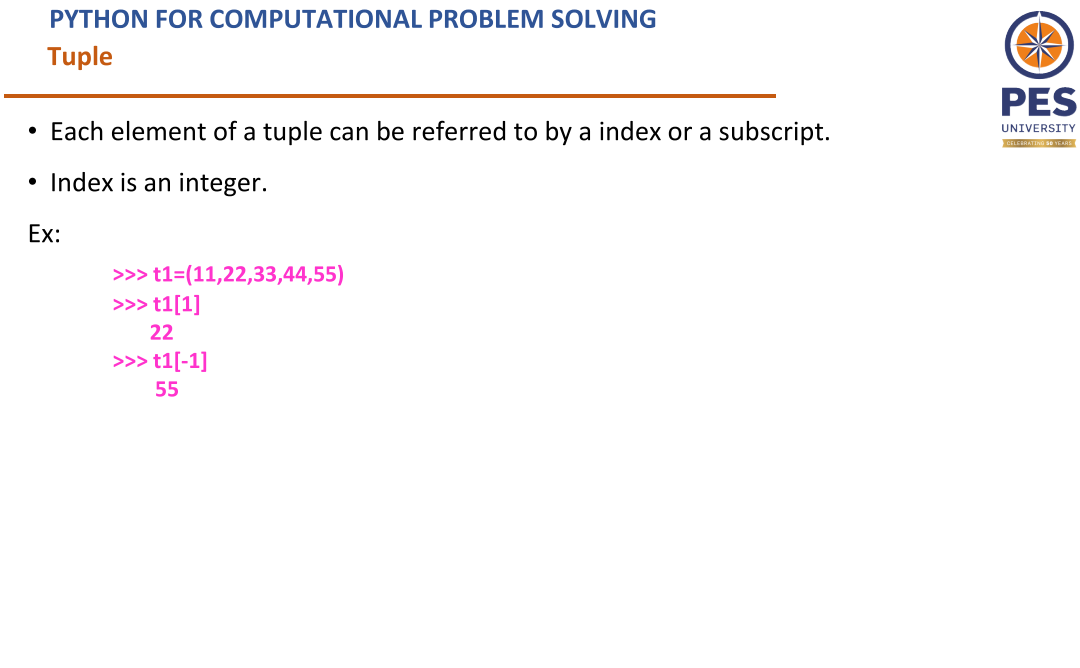


Figure G000004. UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples · source slide page 4.

EV0000088

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

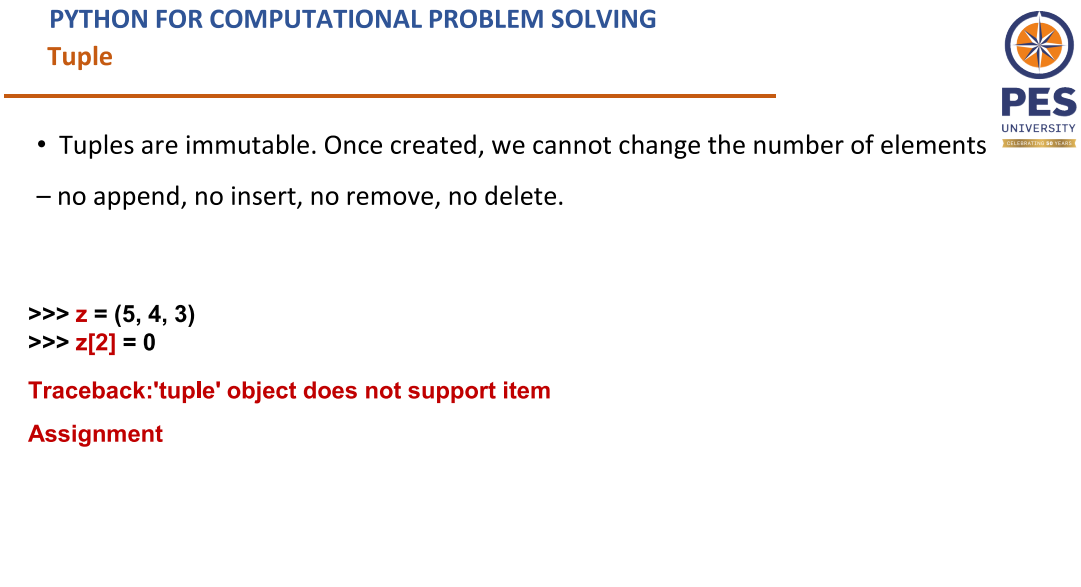


Figure G000005. UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples · source slide page 5.

EV0000091

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

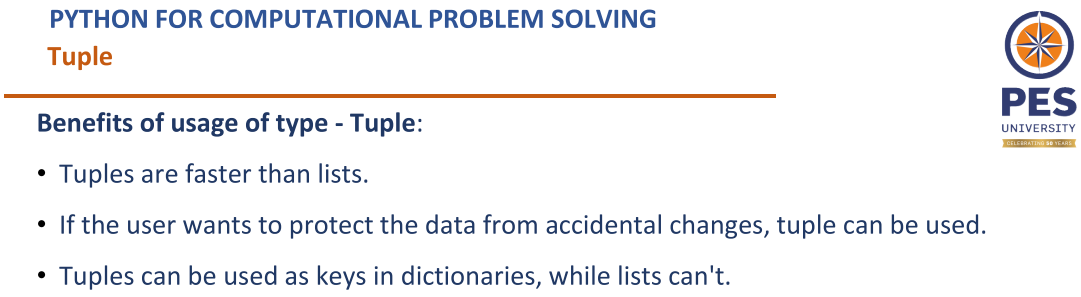


Figure G000006. UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples · source slide page 6.

EV0000094

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

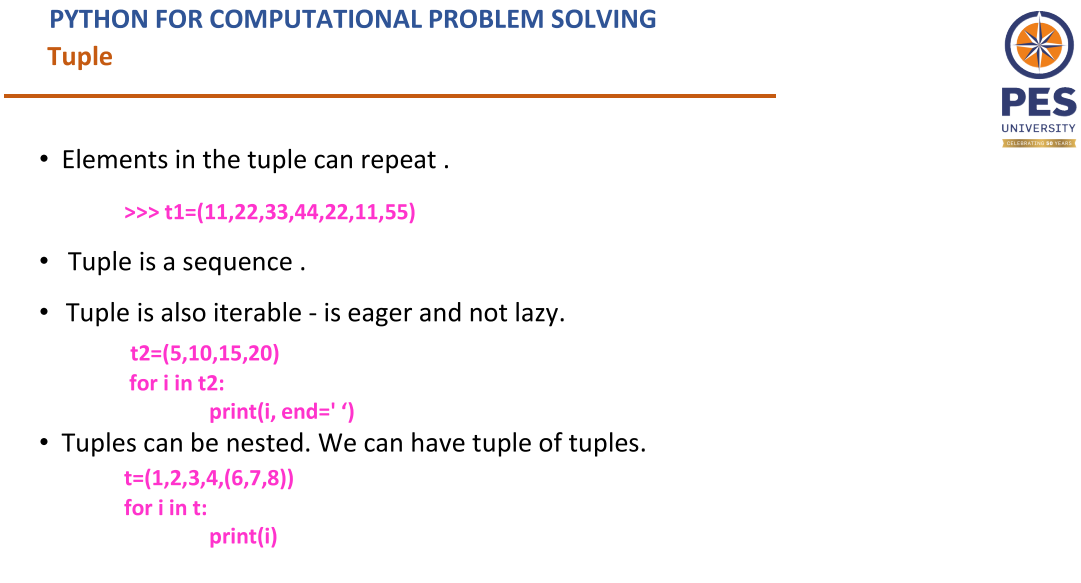


Figure G000007. UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples · source slide page 7.

EV0000097

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

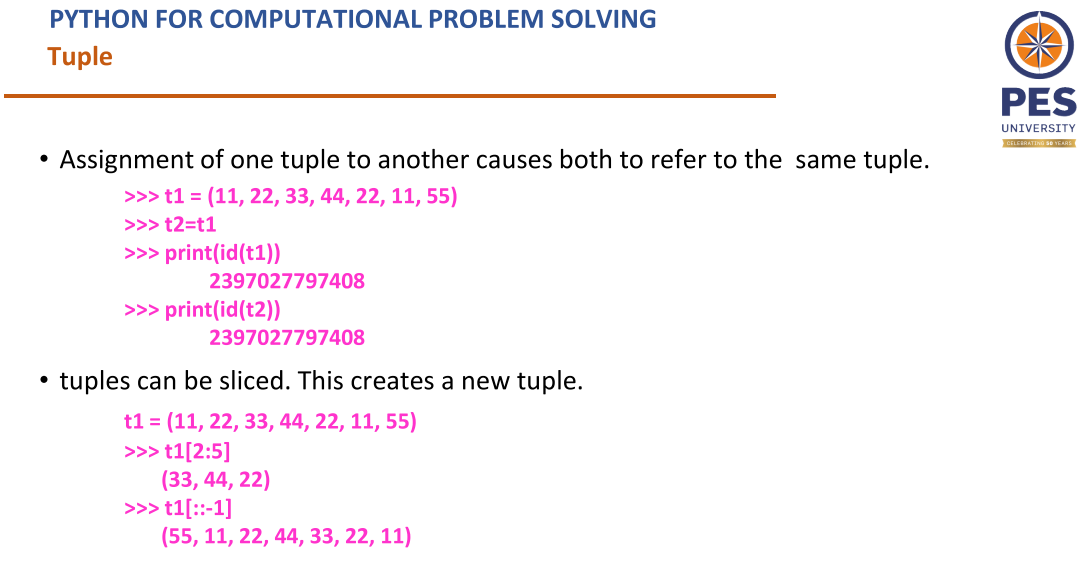
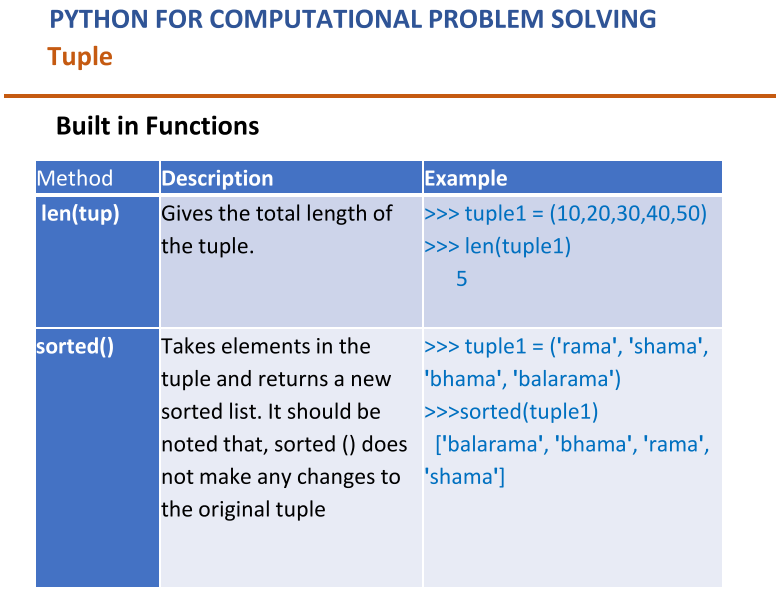


Figure G000008. UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples · source slide page 8.

EV0000100

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.



PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Tuple

PES UNIVERSITY

Built in Functions

Method	Description	Example

len(tup)	Gives the total length of the tuple.	>>> tuple1 = (10,20,30,40,50)
		>>> len(tuple1)
		5

sorted()	Takes elements in the tuple and returns a new sorted list. It should be noted that, sorted () does not make any changes to the original tuple	>>> tuple1 = ('rama', 'shama', 'bhama', 'balarama') >>>sorted(tuple1) ['balarama', 'bhama', 'rama', 'shama']

Figure G000009. UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples · source slide page 9.

EV0000103

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

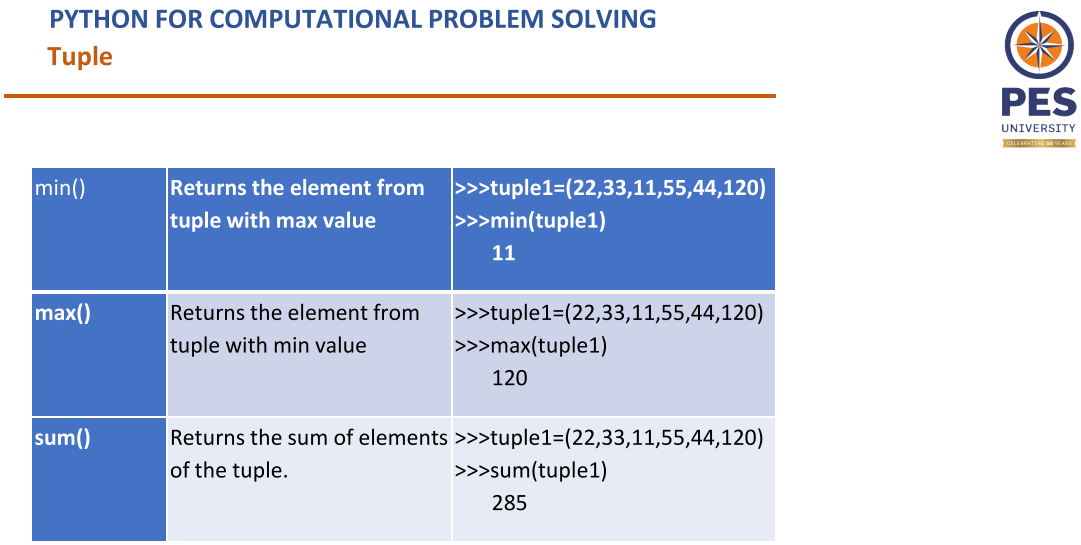


Figure G000010. UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples · source slide page 10.

EV0000106

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

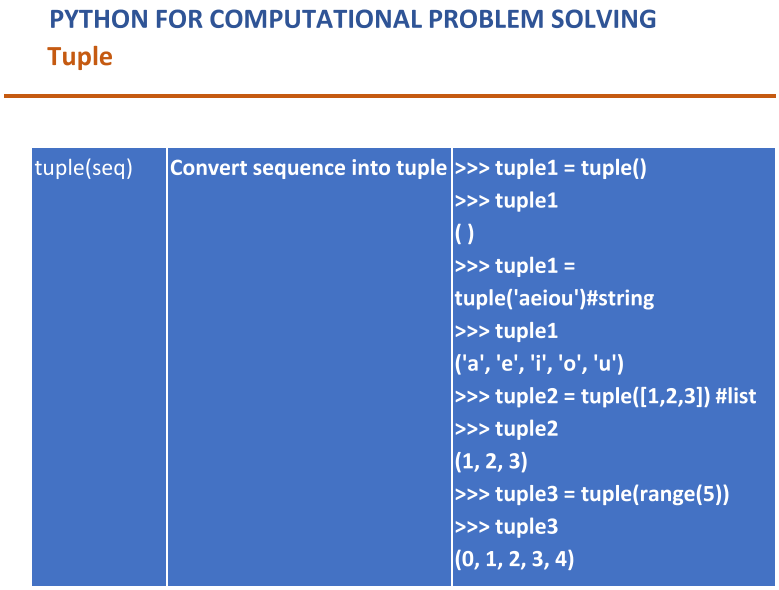
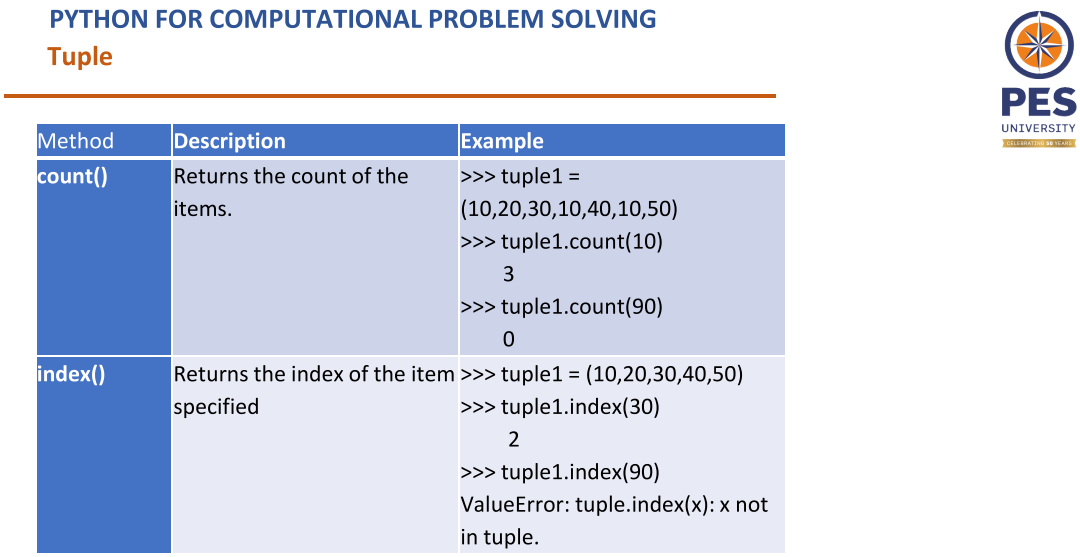


Figure G000011. UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples · source slide page 11.

EV0000109

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.



Method	Description	Example

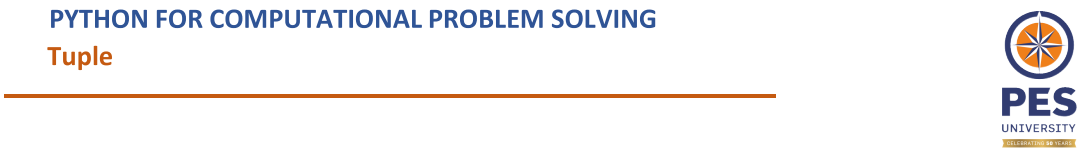
count()	Returns the count of the items.	>>> tuple1 = (10,20,30,10,40,10,50)
		>>> tuple1.count(10)
		3
		>>> tuple1.count(90)
		0

index()	Returns the index of the item specified	>>> tuple1 = (10,20,30,40,50)
		>>> tuple1.index(30)
		2
		>>> tuple1.index(90)
		ValueError: tuple.index(x): x not in tuple.

Figure G000012. UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples · source slide page 12.

EV0000112

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.



- **Concatenation**:

  Allows us to join two tuples by using operator '+'.

Ex:

```
>>> tuple1 = (11,33,55,77,99)
>>> tuple2 = (22,44,66,88,100)
>>> tuple1 + tuple2
#concatenates two tuples
(11, 33, 55, 77, 99, 22, 44, 66, 88 100)
```

Figure G000013. UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples · source slide page 13.

EV0000115

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

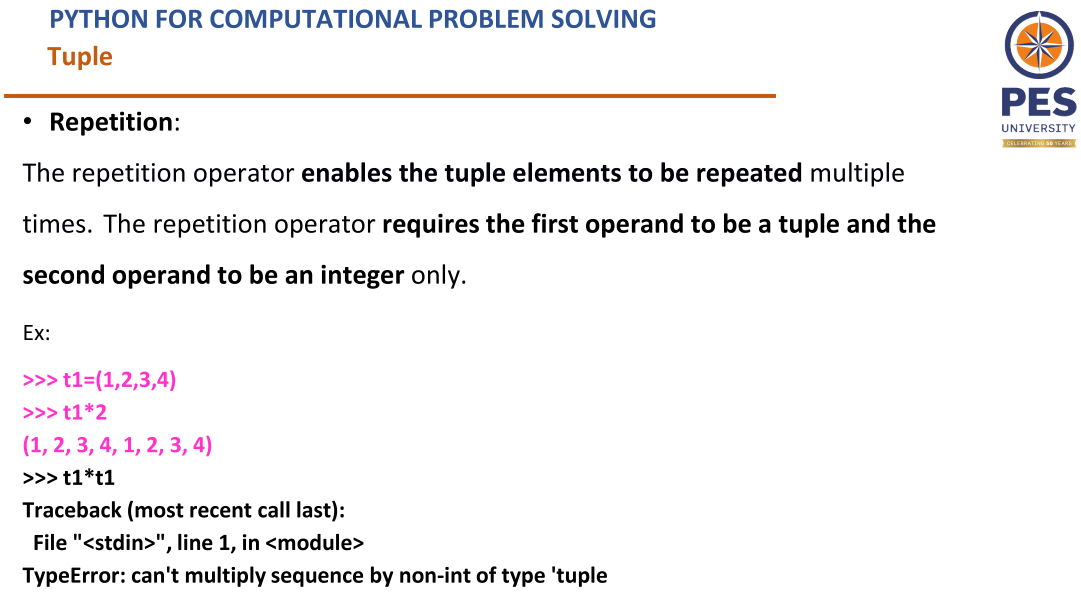


Figure G000014. UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples · source slide page 14.

EV0000118

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

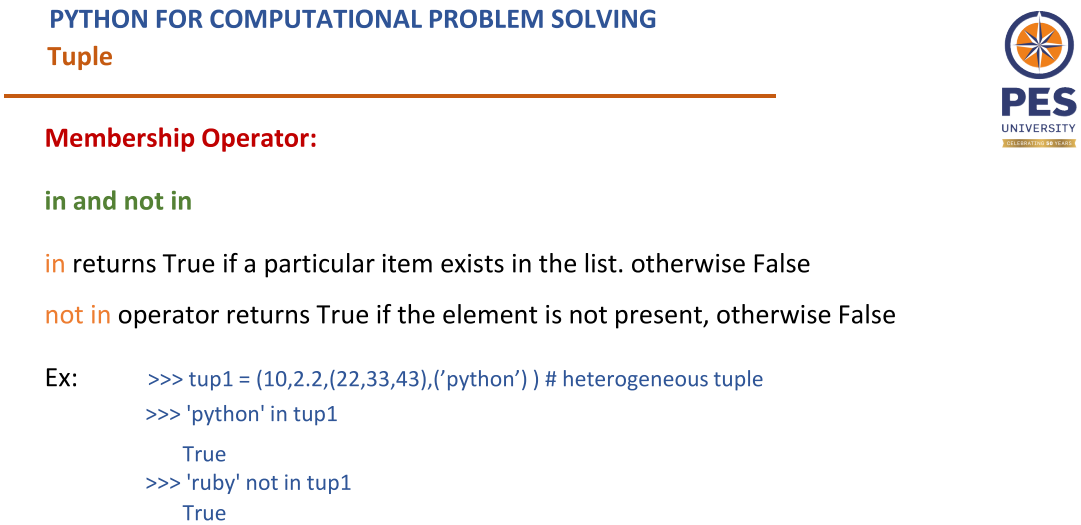


Figure G000015. UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples · source slide page 15.

EV0000121

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

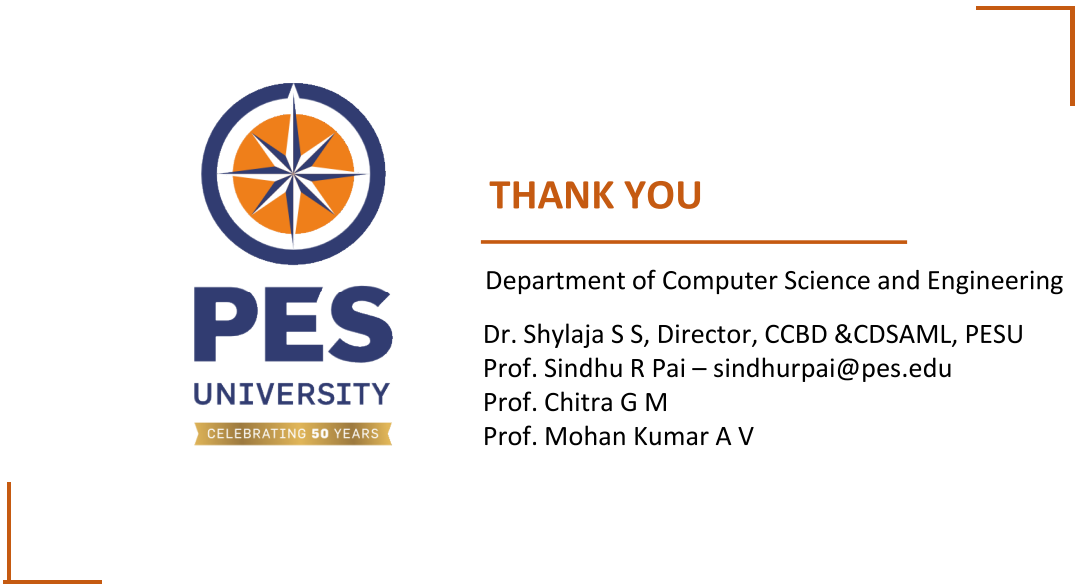


Figure G000016. UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples · source slide page 16.

EV0000124

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Tuples in python Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000078 · EV0000079 · EV0000080
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Tuple Tuple has the following attributes: • The element of the tuple can be of any type. (There(There isis nono restrictionrestriction onon thethe typetype ofof thethe elementelement.)). AA tupletuple isis samesame asas listlist, exceptexcept thatthat the set of elements is enclosed in parentheses instead of square brackets and these elements are immutable. Ex: t = ('python',1,3.4) # heterogeneous t1 = (11,22,33,44) #homogeneous EV0000081 · EV0000082 · EV0000083
3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Tuple Tuple is a data structure containing zero or more elements. Creation of tuple • An empty tuple can be created using a constructor, tuplep ()() or ().() • Tuple with two elements. t3 = (1, 2) • Tuple with a single element. t3=(1,) EV0000084 · EV0000085 · EV0000086

4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Tuple • Each element of a tuple can be referred to by a index or a subscript. • Index is an integer. Ex: >>> t1=(11,22,33,44,55) >>> t1[1] 22 >>> t1[-1] 55 EV0000087 · EV0000088 · EV0000089
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Tuple • Tuples are immutable. Once created, we cannot change the number of elements – no append, no insert, no remove, no delete. >>> z = (5, 4, 3) >>> z[2] = 0 Traceback:'tuple' object does not support item Assignment EV0000090 · EV0000091 · EV0000092
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Tuple Benefits of usage of type - Tuple: • Tuples are faster than lists. • If the user wants to protect the data from accidental changes, tuple can be used. • Tuples can be used as keys in dictionaries, while lists can't. EV0000093 · EV0000094 · EV0000095
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Tuple • Elements in the tuple can repeat . >>> t1=(11,22,33,44,22,11,55) • Tuple is a sequence . • Tuple is also iterable - is eager and not lazy. t2=(5,10,15,20) for i in t2: print(i, end=' ‘) • Tuples can be nested. We can have tuple of tuples. t=(1,2,3,4,(6,7,8)) for i in t: print(i) EV0000096 · EV0000097 · EV0000098
8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Tuple • Assignment of one tuple to another causes both to refer to the same tuple. >>> t1 = (11, 22, 33, 44, 22, 11, 55) >>> t2=t1 >>> print(id(t1)) 2397027797408 >>> print(id(t2)) 2397027797408 • tuples can be sliced. This creates a new tuple. t1 = (11, 22, 33, 44, 22, 11, 55) >>> t1[2:5] (33, 44, 22) >>> t1[::-1] (55, 11, 22, 44, 33, 22, 11) EV0000099 · EV0000100 · EV0000101
9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Tuple Built in Functions Method Description Example len(tup) Gives the total length of >>> tuple1 = (10,20,30,40,50) the tuple. >>> len(tuple1) 5 sorted() Takes elements in the >>> tuple1 = ('rama', 'shama', tuple and returns a new 'bhama', 'balarama') sorted list. It should be >>>sorted(tuple1) noted that, sorted () does ['balarama', 'bhama', 'rama', not make any changes to 'shama'] the original tuple EV0000102 · EV0000103 · EV0000104
10. Source slide page 10 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Tuple min() Returns the element from >>>tuple1=(22,33,11,55,44,120) tuple with max value >>>min(tuple1) 11 max() Returns the element from >>>tuple1=(22,33,11,55,44,120) tuple with min value >>>max(tuple1) 120 sum() Returns the sum of elements >>>tuple1= (22,33,11,55,44,120) of the tuple. >>>sum(tuple1) 285 EV0000105 · EV0000106 · EV0000107
11. Source slide page 11 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Tuple tuple(seq) Convert sequence into tuple >>> tuple1 = tuple() >>> tuple1 ( ) >>> tuple1 = ttuple('aeiou')#stringl (' i ')# t i >>> tuple1 ('a', 'e', 'i', 'o', 'u') >>> tuple2 = tuple([1,2,3]) #list >>> tuple2 (1, 2, 3) >>> tuple3 = tuple(range(5)) >>> tuple3 (0, 1, 2, 3, 4) EV0000108 · EV0000109 · EV0000110
12. Source slide page 12 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Tuple Method Description Example count() Returns the count of the >>> tuple1 = items. (10,20,30,10,40,10,50) >>> tuple1.count(10) 3 >>> tuple1.count(90) 0 index() Returns the index of the item >>> tuple1 = (10,20,30,40,50) specified >>> tuple1.index(30) 2 >>> tuple1.index(90) ValueError: tuple.index(x): x not in tuple. EV0000111 · EV0000112 · EV0000113

13. Source slide page 13 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Tuple • Concatenation: Allows us to join two tuples by using operator '+'. Ex: >>> tuple1 = (11,33,55,77,99) >>> tuple2 = (22,44,66,88,100) >>> tuple1 + tuple2 #concatenates two tuples (11, 33, 55, 77, 99, 22, 44, 66, 88 100) EV0000114 · EV0000115 · EV0000116
14. Source slide page 14 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Tuple • Repetition: The repetition operator enables the tuple elements to be repeated multiple times. The repetition operator requires the first operand to be a tuple and the second operand to be an integer only. Ex: >>> t1=(1,2,3,4) >>> t1*2 (1, 2, 3, 4, 1, 2, 3, 4) >>> t1*t1 Traceback (most recent call last): File "<stdin>", line 1, in <module> TypeError: can't multiply sequence by non-int of type 'tuple EV0000117 · EV0000118 · EV0000119
15. Source slide page 15 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Tuple Membership Operator: in and not in in returns True if a particular item exists in the list. otherwise False not in operator returns True if the element is not present, otherwise False Ex: >>> tup1 = (10,2.2,(22,33,43),('python')) # heterogeneous tuple >>> 'python' in tup1 True >>> 'ruby' not in tup1 True EV0000120 · EV0000121 · EV0000122
16. Source slide page 16 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shylaja S S, Director, CCBD &CDSAML, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Prof. Chitra G M Prof. Mohan Kumar A V EV0000123 · EV0000124 · EV0000125

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:01.668] Hello all, welcome back a Python for computational problem solving. In today's session, we are going to be dealing with the concepts of LIS. So let's get started. [00:00:18.180] Linear data structure where elements have a linear ordering and list has got some of the characteristics. Before we discuss uh to find out the characteristics of a list, let's try to find out why do I really need a list. [00:00:36.964] Okay, suppose assume that you've been given a task in order to find the sum of three numbers. [00:00:46.404] Uh a straightforward task, you would read three variables A B C and then each one of them have been assigned some value and then you will perform A plus B plus C and at the end you're going to be getting the sum. Followed by [00:01:03.172] If I ask you to perform the sum of thousand numbers, the sum of thousand numbers. [00:01:11.204] In this case, what you can do is very simple, where you can iterate over the loop or run over the loop a thousand times, and then try to find out the sum by updating the sum O variable. Okay? But [00:01:27.460] If we ask you to arrange these thousand elements in an ascending order. [00:01:34.404] Fine, arranging the thousand elements in an ascending order. In this case, it becomes impossible for us to make use of thousand different variables and then perform the arranging operation. [00:01:48.324] Fine. So in such case, the main reason is you need all the thousand items to be there at a same point of time because you are trying to do the comparison. So in such case. [00:02:00.048] it is always uh useful to have all the different elements under a similar name. So this is where the list comes into picture. Fine. So moving uh forward to the characteristics of the list. [00:02:16.740] Elements in a list are heterogeneous in nature, can be heterogeneous in nature. It could be either homogeneous or heterogeneous. The moment we say it as homogeneous, it is of the similar type, and when it is an heterogeneous, it could be of different types. [00:02:34.084] Okay and as mentioned there is no uh separate name that is given for each element in a list. Lists can be uh accessed using indexing operation. Lists can be [00:02:49.200] Elements of a list can be accessed using indexing operation.

Request you to uh see to the uh video on indexing operation. Indexing is also called as a subscript. And uh indexing always starts with zero. [00:03:06.480] indexing always starts with zero and this that's the reason uh because of this particular reason it is also called as zero based indexing. [00:03:17.156] And followed by a list can grow or shrink in the sense you once the list is created you can add or also you can remove the elements. List is an iterable. [00:03:31.076] Okay, list is an I trouble. In this case it is eager. Okay, range is lazy. I uh the list is eager in nature. I can have uh a nested list, a list within one list. [00:03:45.860] Fine. And uh I can uh assign one list to the another list. In such case both the list would be pointing to the same list. [00:03:55.524] I can also create a sublist by making use of an operation called as a slicing operation. Please also look into the video of slicing operations. [00:04:08.068] And uh Python provides you so many number of built-in functions where you can manipulate the list, you can work with the list, you can uh uh modify the lists. [00:04:18.720] Okay, we are gonna be looking into each one of these attributes uh with respect to a particular example. Before uh me going and uh executing the things, let me quickly iterate over uh the uh PPTs that are there. Okay. [00:04:35.428] So how do we create a list? I'm going to be demonstrating it to you, so do not worry. And as I said you, there are so many operations that are available for us in order to perform the operations which we are going to be also seeing. [00:04:52.292] Right. So there is an append operation, insert, extend, pop and remove operations. Other operations also include concatenation, sorting. Okay, there's something called as a membership operator. So followed by slicing operations.

EV0000002 · EV0000003 · EV0000004 · EV0000005

Transcript block 2 (source interval 000300-000599 seconds): [00:05:12.420] So there are other list operations also they are mentioned here, but I'm going to be demonstrating all of them for you. Okay. So before we do any other things, let's get started with the demonstration. So let me go here. [00:05:33.636] So I will start demonstrating I've used the same uh previous uh file itself, that's why the name of the file is still a for loop itself. Okay, so do not worry about the name of the file. [00:05:49.604] So let us uh try to uh find out [00:05:56.484] So let me create a list. Let's say L S T is equal and twenty thirty forty fifty. Linear ordering means [00:06:15.908] Just look at this and say print list. So I'll open a command prompt. [00:06:33.380] Okay, so trying to minimize it's easy for us, as usual, same things that I always do. I'm trying to do the same thing here. [00:06:46.148] I can I'm just getting my uh execution part ready. Okay, and if you see when I say uh when I say this ten, twenty, thirty, forty and uh so on. [00:07:00.772] When I say linear ordering, my input, the way in which the input is given and the way in which the output is given is same. Okay so to avoid uh the confusions let me just save it as uh on the desktop itself, I'll say it as [00:07:19.492] list dot ppy so no confusions for us let me execute and see [00:07:29.348] list dot py okay no confusions for us now hopefully uh the first point is clear to you it's a linear ordering in the sense the way in which the values are input the output or is also going to be the same the order of the uh values within a list is going to be the same [00:07:50.052] So there are um other functionalities that we could uh do it on the uh list. So we will see each one of them. I will do all of them at the end I will run so I will say print of what is the length of the list. [00:08:07.076] Okay. This is the first operation that uh I want to do. List provides there are some common functionalities that we can do with the list. Okay. So if you don't mind, I will go back uh to the Python mode to [00:08:24.388] Explain it to you in an easier way. And then again I will come back as and when it is required for us. I will say list is equal to ten comma, twenty comma, thirty, forty and fifty. [00:08:40.932] So I will see what is the length of the list. Only difference between uh interactive and your uh what is

that you have to give state [00:08:55.076] So hope uh it is clear to you what is the length of the list. The length of the list is five in this case. So I can also do find out what is the minimum in the lists. [00:09:09.796] Okay min min is a function that is available. It tries to find out what is the minimum element that is present. So minimum element is 10 and uh the maximum element in the list is 50. I can also perform something called as a sum operation on the list. [00:09:29.732] it which adds every element in the list and gives you. So ninety, fifty plus forty, ninety, hundred, hundred and thirty, hundred and fifty is the answer. So hope um it is clear to you [00:09:46.244] Okay, so I can also do something like this. List plus list. In this case, plus is called as a concatenation operation. Concatenation operation merges two lists and returns you a single list. Okay.

EV0000006 · EV0000007 · EV0000008 · EV0000009

Transcript block 3 (source interval 000600-000899 seconds): [00:10:02.992] concatenation operation merges two lists and it returns you a single list. And there is something also called as a repetition operation. That's called as a repetition operation. Multiply a list and number of times [00:10:21.924] Okay, and if you can see when I say list, this is my original list, and if you want any changes that you have done, you can definitely assign it to something. Okay, so now in this case when I say L1. [00:10:40.004] Values have been repeated. Okay, so thus it also list also allows us to use duplication. List elements can be duplicate in nature. [00:10:53.188] Okay and uh these are the few operations uh that we have uh seen. Okay, so before this I just took a list. So let me do a few things [00:11:11.396] And see I should have uh taught you how to create a list. Nevertheless, uh we will uh uh do it now. Okay. So what I'll do is I will create an empty list. [00:11:23.904] Creating an empty list is with the help of an open bracket and a close bracket. Okay so when I see what is the type of list, it gives me class list. [00:11:38.276] Okay, so I can also mention I can also do something like this. L S T is equal to L I S T of this. And I'll see what is the type of list. [00:11:52.996] It is still a list itself. In this case, list of this is called as a constructor. It's called as a constructor. Okay. And uh in this case, the list is called as an empty list. [00:12:08.132] Fine. How do you create a list? Make use of an open and closed brackets in order to represent. Okay. I can also create a list with the elements where I have already demonstrated. [00:12:21.188] Okay. And uh uh let me s check with the list. At this point list is empty. So what I will do is I will create my list again. So I'll say ten, twenty, thirty, fourteen. [00:12:38.564] This is my uh list. [00:12:43.396] What and all I can do with this particular list. As list you can see there are n number of elements, but they are all belong to or they all have a similar name called as LST. But LST has a so many number of elements that are present. [00:13:01.732] Now what I can do is I can access every element that is present in a list. [00:13:08.548] and access every element present in a list. How will I access the elements present in the list? I can make use of an indexing operation. As mentioned earlier, please look into my indexing video. Okay. So I will say list of zero, I get 10 [00:13:26.692] list of um four zero this is zero one two three four so getting and I'm expecting an answer to be equal to fifty in this case [00:13:40.644] Okay. Um as indexing starts from zero, it is also called as what? Zero based indexing. Called as zero based indexing. This is fine. So we'll uh go ahead with this. Lists also supports slicing operation. [00:13:59.012] List also supports a slicing operation. Okay? Again, see to the video on slicing. I can just mention from one and I will not mention anything to stop. This is a start value and this is a stop value. [00:14:16.356] Okay, so I will say one to end which means start from a index one and go till the index end which is 50 in this case. This is fine. So one more uh we'll do. [00:14:30.212] From three, start from one and stop at four. So zero, one, two, three, four. The moment I give four, which means four minus one because one less than the existing one.

[00:14:45.188] That from one go till three and one and three the values that I'm telling are the indexed values. They are the indexed values. [00:14:54.148] So this is how uh the slicing operation works. Okay so now there are so many operations it provides Python provides so that I can manipulate the list. Okay so this is my list

EV0000010 · EV0000011 · EV0000012 · EV0000013

Transcript block 4 (source interval 000900-001199 seconds): [00:15:11.332] And I will try to find out what is the length of my list. Length of my list is five. Okay so now we will try to work with the b uh the functionalities that are available so that we will try to uh modify this particular list. [00:15:31.140] So, list can grow or shrink, grow in the sense increase in its number of items, shrink is to reduce the number of items in a list. [00:15:42.308] Okay, so there are so many functionalities uh that are available, we will be looking into them quickly. Okay, so before that also see to this I will say L is equal to I'm trying to create a new list one two or two one five three hundred and so on. [00:16:02.724] And if you can see the example says that a list has some integer values. All the values of a list are of same type. [00:16:20.228] Anything. Okay. And then followed by a string. [00:16:28.516] So this is my new list. Okay. In this case list L one is called as heterogeneous in nature. The reason is it has an integer type, it has a floating type, and it also has a string type. [00:16:44.324] Typo mistink in terms of the string, never never mind, do not worry about it. Okay. So [00:16:52.420] Hope it is uh clear to you. Okay, so list elements can be an heterogeneous in nature or it could also be a homogeneous in nature. Okay, let's quickly find out uh what and all I can do with the list. [00:17:06.992] How will I do it? I'll say DIR of a list to find out what functions does the list have. So if you can see list has append, clear, copy, count, extend, and so on. [00:17:23.364] Uh for the copy operation look into the video for copy okay so let's quickly uh do these operations. Okay. So let's get started. [00:17:39.940] So this is my list. [00:17:42.436] Right. Uh one more thing. If I want to um find out or access fifty, what do I have to do? I will say list of uh minus one. So please remember Python also provides a negative indexing. So I'm getting a value called as 50. So [00:18:00.036] Um there are many things that I've worked on the um indexing uh topic. Please look into the video. [00:18:11.716] Do with a few of the operations quickly. So this is my list and say list dot append. Now I am looking into the for functions, working with the functions. Okay. Append hundred. When I say append 100, it always appends the value to the end. [00:18:31.812] Okay, add an element always at the end. So I will say lists. You can see that the value hundred is present. Okay. I can also insert an item at a particular position. So I will say insert [00:18:51.940] insert a value 200 okay so 200 but I have to specify where I have to insert. So maybe I will say insert at the zeroth location. [00:19:05.636] First parameter is the location that you need to specify and the second parameter is the value of which you want to insert. Okay so list let's look into this. List is this and as I mentioned zero is now being replaced by two hundred. [00:19:24.932] So hopefully you have uh understood the append method followed by the insert method. So we will uh try to work with the other operations. So I'll see a or uh list dot [00:19:46.276] [ASR low confidence; check audio] List.pop [00:19:50.756] See, it's popped out the last element. Again, pop out one more, so it will pop out fifteen and so on. Okay and I can also do something like this list dot pop of one where one represents an index value.

EV0000014 · EV0000015 · EV0000016 · EV0000017

Transcript block 5 (source interval 001200-001499 seconds): [00:20:07.940] Okay, one represents an index value. If nothing it represents uh it removes an item from the left side right to left, and if I say pop without any argument, it always removes the last item. And if it is pop [00:20:23.632] with

a particular argument. In this case the argument is uh representing the index value. So hope uh these things are clear to you. Okay so I will also say list dot remove hundred. [00:20:43.908] I will remove the value hundred. Okay. So in this case, the value hundred X is not in the list. One second, just wait. Let's uh look into what is our uh lists. [00:21:00.900] Okay, the list is this. Previously I had removed the value 100 and I am trying to remove it. So before uh going to this what I will do is remove 10 is what I'll say. 10 is also not there. So let's remove 200 in this case. [00:21:17.408] And when you remove the elements gets removed. Okay. So you have 20, 30, 40. As previously seen, if you're trying to remove a value which is not present, then it throws you an error. Okay. Remove. I'll say remove thousands. [00:21:36.708] Okay, I cannot remove because the value thousand is not part of the list. Okay. List X does not contain this particular Y. So hope um it is clear to you. I can also do this. This is my new list. [00:21:58.020] List is 20 present in list? Answer is yes. Okay. Is 200 present in list? No. Is 200 not in? 200 not in the list. Yes, it is still true because it is not there. [00:22:18.020] Okay, so this is how the membership operator works. Index value we have uh already seen [00:22:30.180] Did we check with the index value? Just give me time. No, pop append insert. Okay. So let's work with the index value in this case. So this is my list. I will also try to add few values to the list. List.append [00:22:50.884] of four hundred. Okay, when I say this list has got a four hundred. I can also do list dot index of um twenty. [00:23:05.572] What is the index value of 20? 20 is present at the zeroth location. Suppose if I say [00:23:15.268] Two hundred and five, some number, some random number. Is uh is two not five present in the list? No, it is not present, so it says it is an error. Okay it shows if a value is not present in the list, it clearly shows it as an error. [00:23:31.524] So I can also do few more things. I'll say list dot count. [00:23:41.764] How many times a particular element is present? For example, 200. Okay. So it says 200 is not present. If something is not present from where it can give you. Okay. So this is about uh [00:23:59.172] the list view operations of the list okay I can also do one more uh thing list dot insert anyway insert we have already worked with okay so and count also we have worked with [00:24:18.916] And I can also say list.sort. [00:24:27.876] list dot sort when I say this the my list is sorted even though if uh it is already started it just ignores it okay if not sorted it is going to create uh or uh do a new list again okay so [00:24:50.148] So hope uh things are all uh very clear to you. There are some programming examples that we have to work with respect to the uh lists which I'm going to be doing in your coming classes.

EV0000018 · EV0000019 · EV0000020 · EV0000021

Transcript block 6 (source interval 001500-001799 seconds): [00:25:08.836] Hope you have understood. Happy uh coding. Keep learning. Thank you.

EV0000022 · EV0000023 · EV0000024 · EV0000025

Full transcript: 3276 normalized words, 100 timestamped segments, 1514.538667 seconds duration. Low-confidence segments flagged: 1. Review locators: 00:19:46.276

EV0000002 · EV0000003 · EV0000004 · EV0000005

3 Dictionary

This lesson is indexed from the source video titled Dictionary. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not

been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000026 · EV0000027

Graphs and diagrams

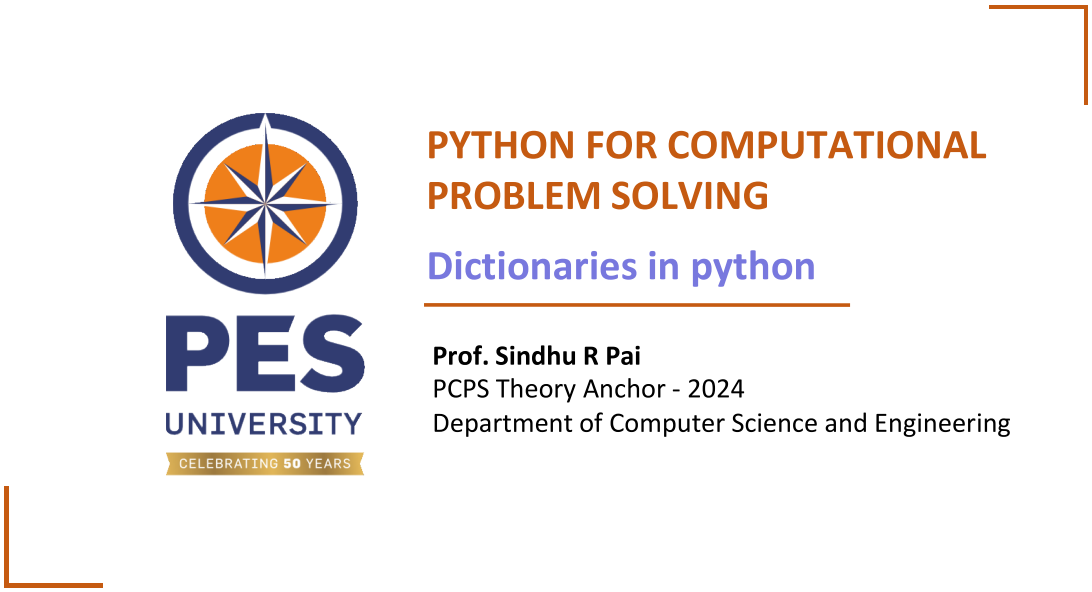


Figure G000017. UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary · source slide page 1.

EV0000127

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

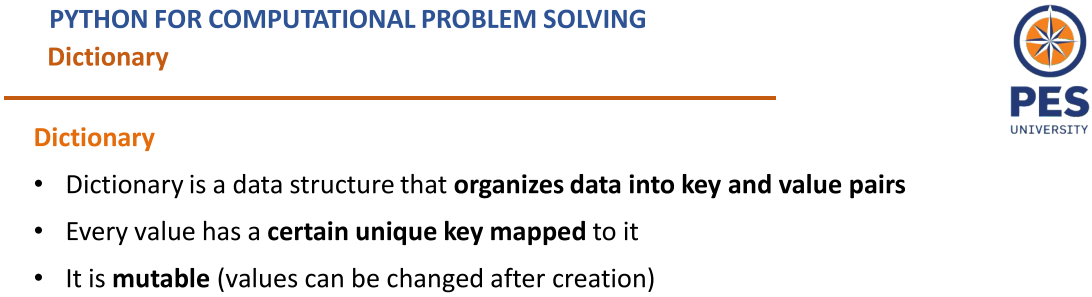


Figure G000018. UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary · source slide page 2.

EV0000130

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

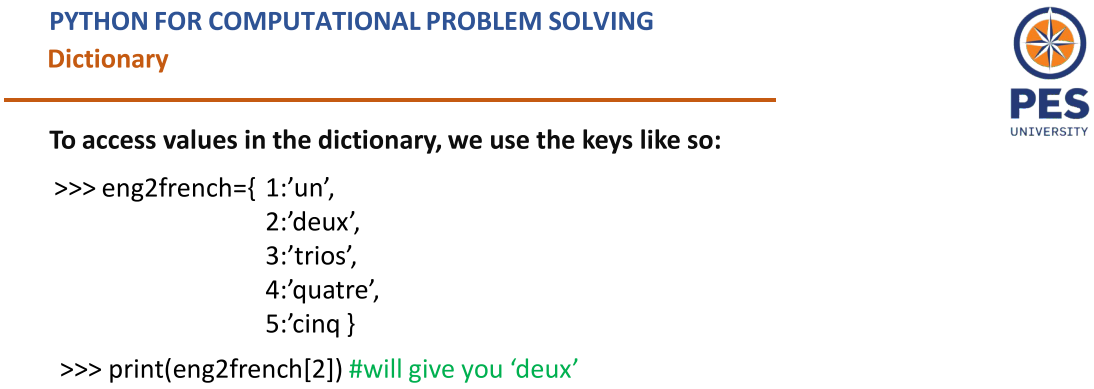


Figure G000019. UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary · source slide page 3.

EV0000133

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

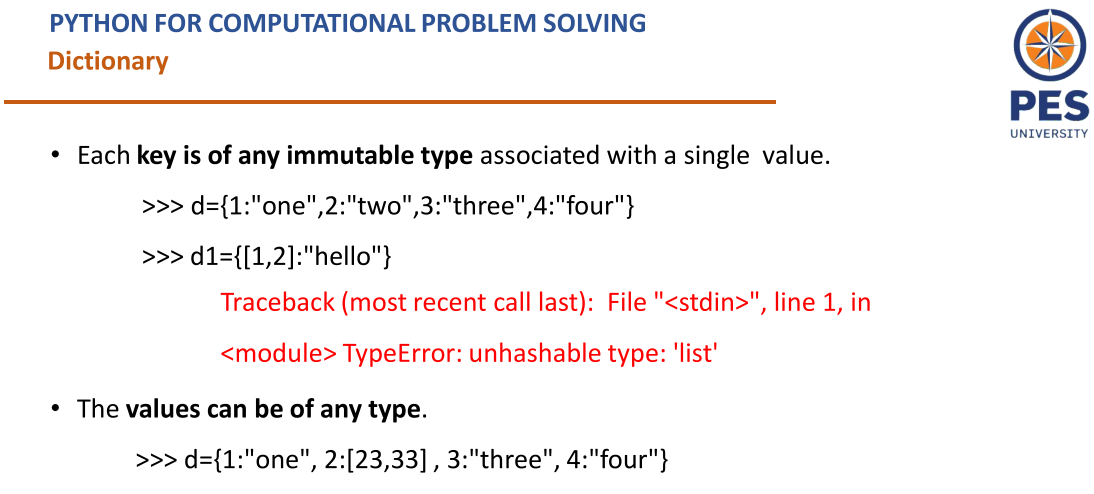


Figure G000020. UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary · source slide page 4.

EV0000136

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

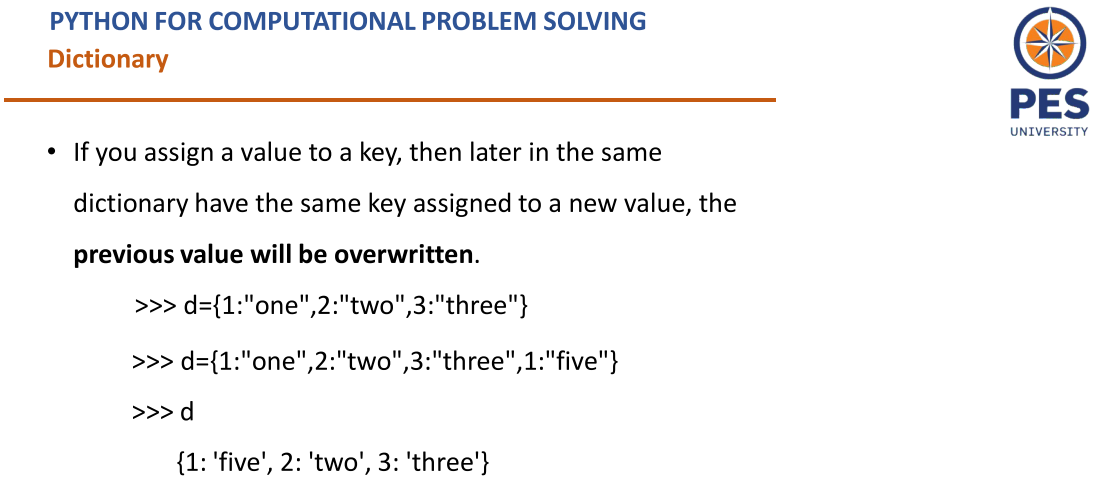
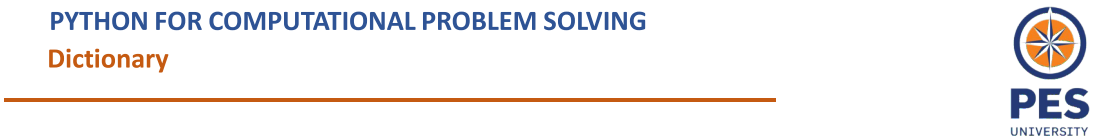


Figure G000021. UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary · source slide page 5.

EV0000139

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.



- The items (key-value pair) in dictionary are unordered, which means that the order isn't decided by the programmer but rather the interpreter.
- The ordering is based on a concept called "**hashing**" .

```
>>> d={'w':11,'a':33,'e':44}
>>> f= {'e':44,'w':11,'a':33}
>>> print(d==f)
    True
```

Figure G000022. UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary · source slide page 6.

EV0000142

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

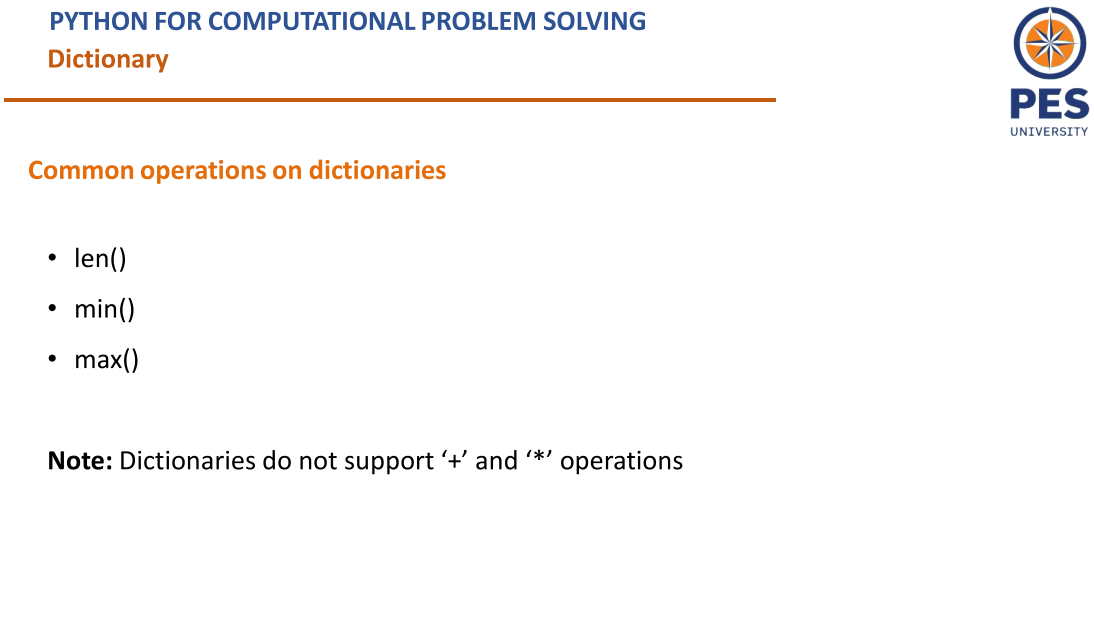


Figure G000023. UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary · source slide page 7.

EV0000145

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING
Dictionary

PES UNIVERSITY

Dictionary Functions

- **get():** returns the value for a given key, if present.

```
>>> print(phonebook.get('Jill'))
    938547565
```

- **items(...)**

  D.items() -> a set-like object providing a view on D's items.

```
>>> phonebook.items()
    dict_items([('Johan', 938477565), ('Jill', 938547565)])
```

Figure G000024. UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary · source slide page 8.

EV0000148

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

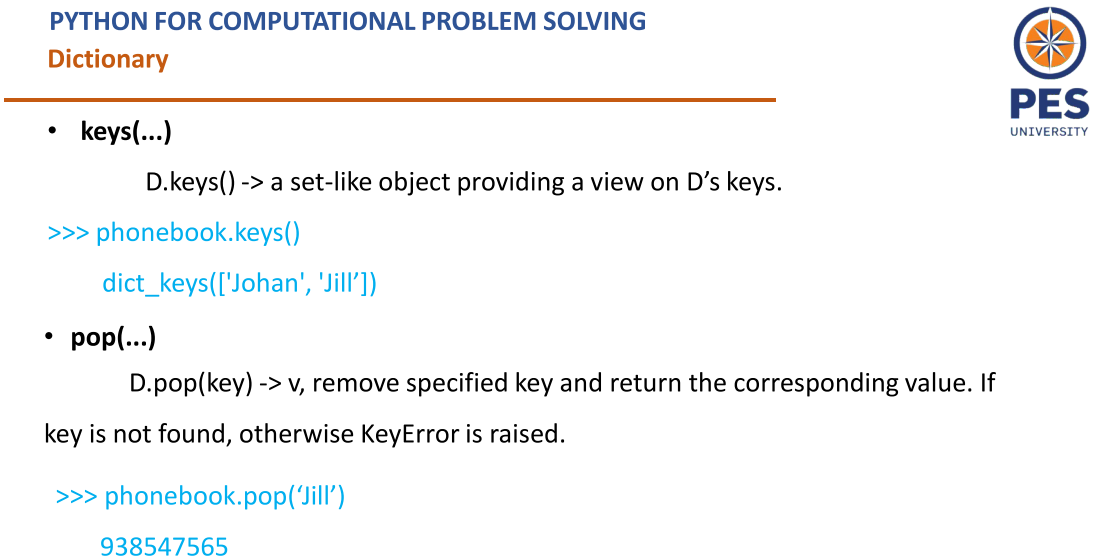


Figure G000025. UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary · source slide page 9.

EV0000151

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

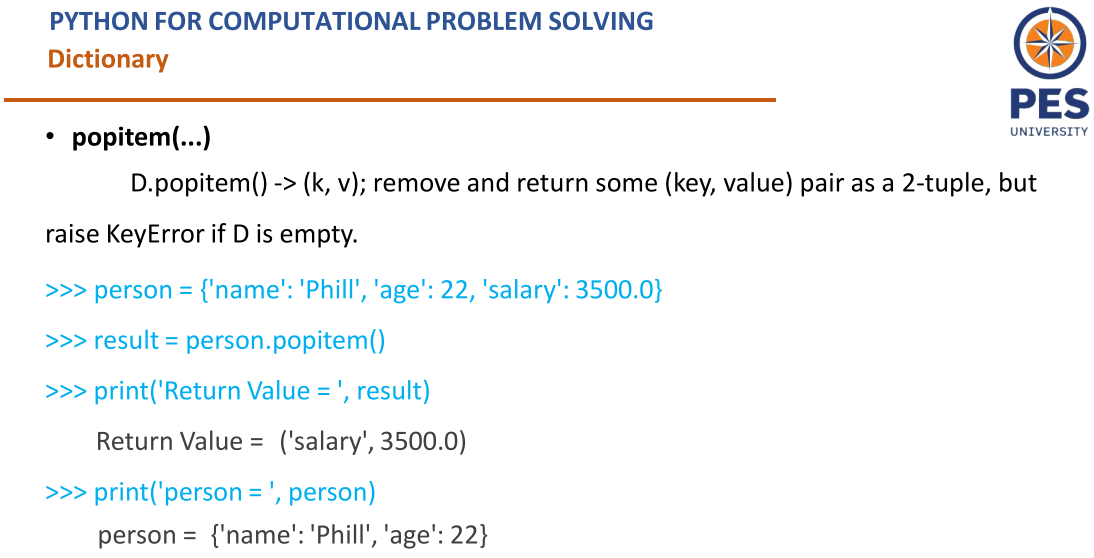


Figure G000026. UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary · source slide page 10.

EV0000154

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

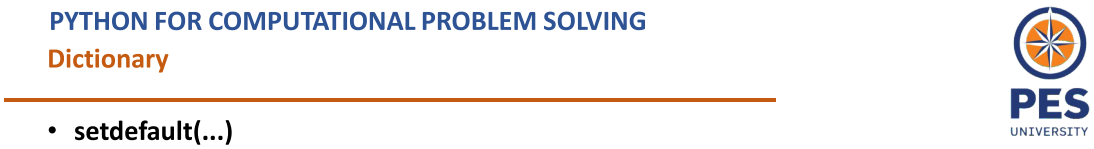


Figure G000027. UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary · source slide page 11.

EV0000157

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Dictionary

PES UNIVERSITY

- **update(...)**

  D.update() -> updates content of D with key-value pairs from a dictionary/iterable that it is given

```
>>> marks = {'Physics':67, 'Maths':87}
>>> internal_marks = {'Practical':48}
>>> marks.update( [('Chemistry', 90), ('Python', 100)] )
>>> marks.update(internal_marks)
>>> print(marks)
```

{'Physics': 67, 'Maths': 87, 'Chemistry': 90, 'Python': 100, 'Practical': 48}

12

Figure G000028. UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary · source slide page 12.

EV0000160

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Dictionary

PES UNIVERSITY

- **values(...)**

D.values() -> returns a view object that displays a list of all the values in the dictionary.

```
>>> marks = {'Physics':67, 'Maths':87}
>>> print(marks.values())
```

dict_values([67, 87])

13

Figure G000029. UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary · source slide page 13.

EV0000163

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

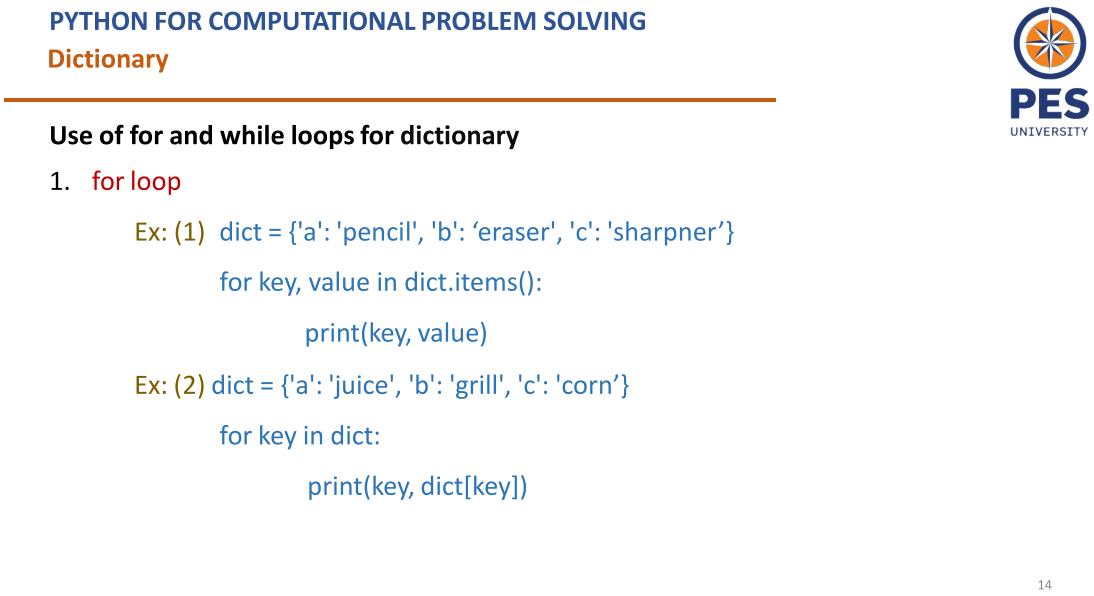


Figure G000030. UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary · source slide page 14.

EV0000166

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

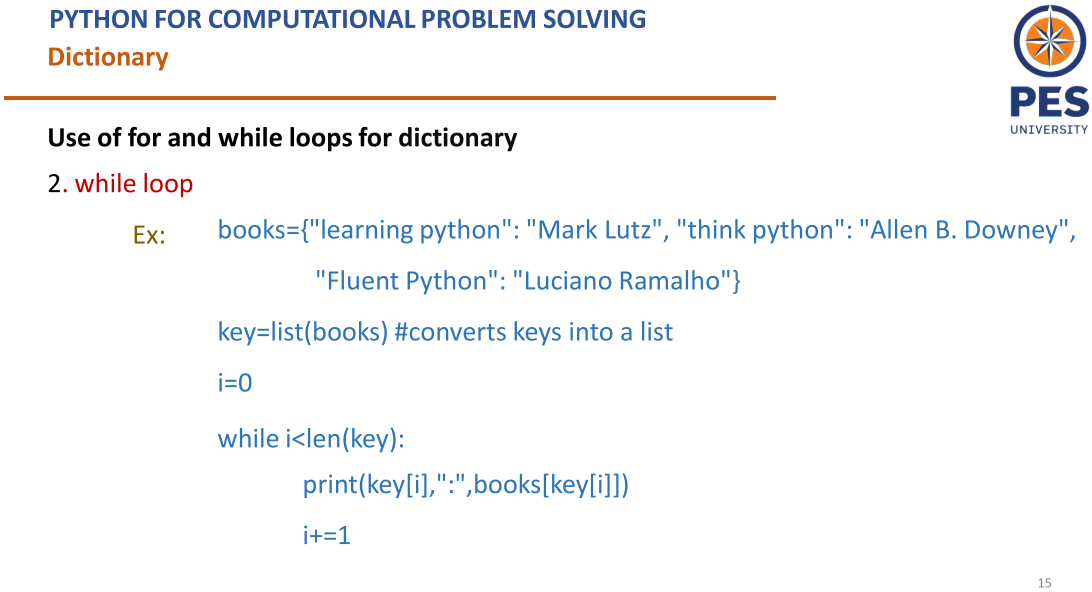


Figure G000031. UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary · source slide page 15.

EV0000169

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

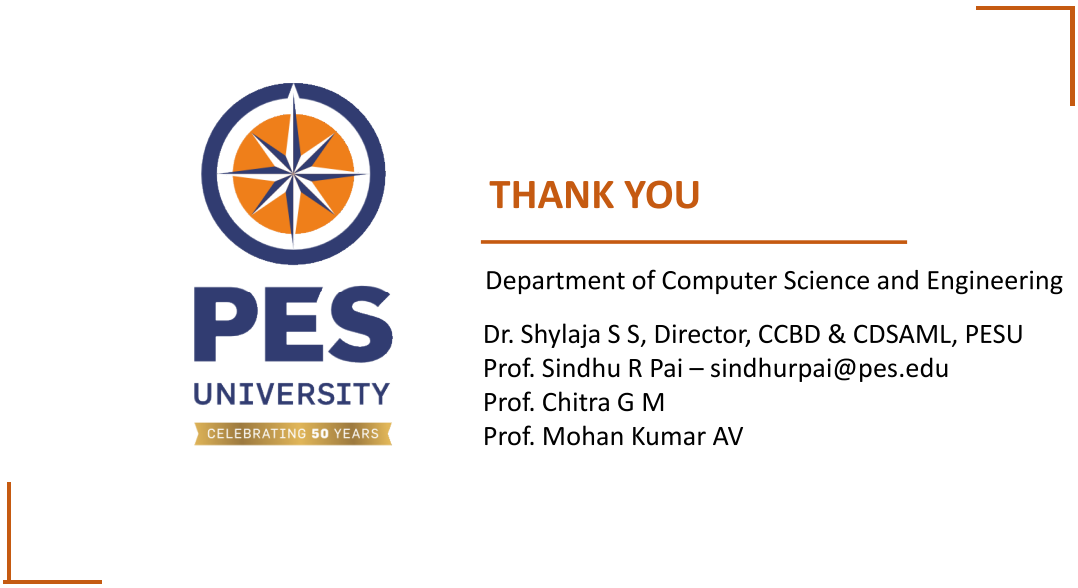


Figure G000032. UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary · source slide page 16.

EV0000172

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Dictionaries in python Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000126 · EV0000127 · EV0000128
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Dictionary Dictionary • Dictionary is a data structure that organizes data into key and value pairs • Every value has a certain unique key mapped to it • It is mutable (values can be changed after creation) Dictionary Creation >>> phonebook={} # Creation of empty dictionary >>> phonebook={"Johan":938477565} # Dictionary with one key-value pair >>> phonebook= {"Johan":938477565,"Jill":938547565} # Dictionary with two key-value pair EV0000129 · EV0000130 · EV0000131
3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Dictionary To access values in the dictionary, we use the keys like so: >>> eng2french={ 1:'un', 2:'deux', 3:'trios', 4:'quatre',4: quatre , 5:'cinq } >>> print(eng2french[2]) #will give you 'deux' EV0000132 · EV0000133 · EV0000134

4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Dictionary • Each key is of any immutable type associated with a single value. >>> d= {1:"one",2:"two",3:"three",4:"four"} >>> d1={[1,2]:"hello"} TracebackTraceback (most(most recentrecent callcall last):last): FileFile "<stdin>""<stdin>", lineline 11, inin <module> TypeError: unhashable type: 'list' • The values can be of any type. >>> d={1:"one", 2:[23,33] , 3:"three", 4:"four"} EV0000135 · EV0000136 · EV0000137
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Dictionary • If you assign a value to a key, then later in the same dictionary have the same key assigned to a new value, the previous value will be overwritten. >>> dd= {1:"one",2:"two",3:"three"}{1 " " 2 " " 3 " h "} >>> d={1:"one",2:"two",3:"three",1:"five"} >>> d {1: 'five', 2: 'two', 3: 'three'} EV0000138 · EV0000139 · EV0000140
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Dictionary • The items (key-value pair) in dictionary are unordered, which means that the order isn't decided by the programmer but rather the interpreter. •• TheThe orderingordering isis basedbased onon aa conceptconcept calledcalled "hashing"“hashing" . >>> d={'w':11,'a':33,'e':44} >>> f= {'e':44,'w':11,'a':33} >>> print(d==f) True EV0000141 · EV0000142 · EV0000143
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Dictionary Common operations on dictionaries • len() • min() • max() Note: Dictionaries do not support '+' and '*' operations EV0000144 · EV0000145 · EV0000146
8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Dictionary Dictionary Functions • get(): returns the value for a given key, if present. >>> print(phonebook.get('Jill')) 938547565938547565 • items(...) D.items() -> a set-like object providing a view on D's items. >>> phonebook.items() dict_items([('Johan', 938477565), ('Jill', 938547565)]) EV0000147 · EV0000148 · EV0000149
9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Dictionary • keys(...) D.keys() -> a set-like object providing a view on D's keys. >>> phonebook.keys() dict_keys(['Johan',_ y ('Jill'])) • pop(...) D.pop(key) -> v, remove specified key and return the corresponding value. If key is not found, otherwise KeyError is raised. >>> phonebook.pop('Jill') 938547565 EV0000150 · EV0000151 · EV0000152
10. Source slide page 10 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Dictionary • popitem(...) D.popitem() -> (k, v); remove and return some (key, value) pair as a 2-tuple, but raise KeyError if D is empty. >>>>>> personperson = {'name':{ name : 'Phill',Phill , 'age':age : 22,22, 'salary':salary : 3500.0}3500.0} >>> result = person.popitem() >>> print('Return Value = ', result) Return Value = ('salary', 3500.0) >>> print('person = ', person) person = {'name': 'Phill', 'age': 22} EV0000153 · EV0000154 · EV0000155
11. Source slide page 11 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Dictionary • setdefault(...) D.setdefault(key,value) -> if the key is in the dictionary, returns its value. If the key is not present, insert the key with a specified value and returns that same value. >>> personp = {'name':{ 'Phil',, 'age':g 22}} >>> phone = person.setdefault('phone', 90909090) >>> print('person = ',person) >>> print('phone = ',phone) Output: person = {'name': 'Phil', 'age': 22, 'phone': 90909090} phone = 90909090 EV0000156 · EV0000157 · EV0000158

12. Source slide page 12 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Dictionary • update(...) D.update() -> updates content of D with key-value pairs from a dictionary/iterable that it is given >>> marks = {'Physics':67, 'Maths':87} >>> internal_marks = {'Practical':48} >>> marks.update([(‘Chemistry’, 90), (‘Python’, 100)]) >>> marks.update(internal_marks) >>> print(marks) {'Physics': 67, 'Maths': 87, ‘Chemistry’: 90, ‘Python’: 100, 'Practical': 48} 12 EV0000159 · EV0000160 · EV0000161
13. Source slide page 13 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Dictionary • values(...) D.values() -> returns a view object that displays a list of all the values in the dictionary. >>> marksk = {'{'Physics':67,h ' '''Maths':87}h ' } >>> print(marks.values()) dict_values([67, 87]) 13 EV0000162 · EV0000163 · EV0000164
14. Source slide page 14 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Dictionary Use of for and while loops for dictionary 1. for loop Ex: (1) dict = {'a': 'pencil', 'b': ‘eraser', 'c': 'sharpner’} for key, value in dict.items(): print(key, value) Ex: (2) dict = {'a': 'juice', 'b': 'grill', 'c': 'corn’} for key in dict: print(key, dict[key]) 14 EV0000165 · EV0000166 · EV0000167
15. Source slide page 15 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Dictionary Use of for and while loops for dictionary 2. while loop books={"learning python": "Mark Lutz", "think python": "Allen B. Downey", Ex: "Fluent Python":y "Luciano Ramalho"} key=list(books) #converts keys into a list i=0 while i<len(key): print(key[i],":",books[key[i]]) i+=1 15 EV0000168 · EV0000169 · EV0000170
16. Source slide page 16 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shylaja S S, Director, CCBD & CDSAML, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Prof. Chitra G M Prof. Mohan Kumar AV EV0000171 · EV0000172 · EV0000173

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:01.252] Hello, all, welcome back Python for computational problem solving. In today's session, we are going to be discussing the concepts of sets. So let's get started. [00:00:22.340] So set is an unordered collection of unique elements with zero or more elements present in it. Let me repeat, a set is an unordered collection of unique elements with zero or more elements present in it. [00:00:41.668] Right. So in Python, if you want to create a set or a set is represented using a curly brace. Okay. So let's create our first set. I'll say S is equal to this. [00:01:00.292] open curly breeze and closed curly breeze. Right? And I want to find out what is the type of yes. [00:01:10.276] And if I say what is a type of S, but basically say it as a dictionary. So which means my dictionary as well as set, we've in order to create them, we are going to be making use of curly braces itself. [00:01:26.288] I need to work out what the set. So in such case, what I can do is I can still make use of the curly brace itself and then represent some values in it. Find out what is the type of S [00:01:45.700] Sets now. Okay. But if your intention is to create an empty set, then make use of your constructor called as set in order to represent an empty set. So now what is the type of S? [00:02:05.220] Go to be set. So now in order to create a set, you can use a curly brace with some elements present in it in order to represent it as a set or make use of a constructor. So in this case, set [00:02:24.068] He is called as a constructor. You can use constructor in order to create an empty set. [00:02:32.452] Has got these some attributes. So we will try to understand what are the attributes of a set. Okay. So let's do this. I'll be using a conventions yes, s1, s2, s3 in order to represent it as a set. Okay. So

[00:02:52.964] The first attribute is that set elements are unique in nature. Set elements are unique in nature. What does it mean? I'll say two, one, four again, and a three again. [00:03:11.876] Right. So I have one repeated twice, two repeated twice, three and four repeated twice itself. So let's find out. So I'll say S, what is the value of S in this case? And if you can see I've got one, two and three and four. [00:03:31.440] Okay, so which means elements in a set are unique. So you it doesn't support elements being repeated. Okay, one of the important attributes of a set is you cannot have duplicate elements. [00:03:51.620] So also there is also one more uh important thing that we have to remember. The elements are hashable in nature. [00:04:02.724] Elements are hashable in nature. Now what is hash or what is hashing? So set makes use of a concept of hash in order to represent its element. [00:04:18.020] So what is hashing? At this point, for our level, we just need to understand that hashing is a mechanism used to convert a given element into an integer. [00:04:30.560] That's all we need to know at this particular point. Okay so what is hashing? Hashing is a mechanism used to convert a given element into an integer. Fine. So any object which is hashable. [00:04:47.076] Fine, if it has an hash value which never changes during its lifetime. So let me repeat, an object is hashable if it has a hash value which never changes during its lifetime.

EV0000026 · EV0000027 · EV0000028 · EV0000029

Transcript block 2 (source interval 000300-000599 seconds): [00:05:03.236] Fine, which also means that it needs to have a function called as underscore underscore hash underscore underscore as one of its method. Okay if you find any [00:05:19.824] Data structure having this particular method, then we call it as what it is hashable in nature. So hopefully it is clear to you. So let's point out I'll say DIR of set. And if you can see [00:05:35.460] and hashable method if it is available then we basically call it as what? It is uses a mechanism called as hash method. Okay. So this is an evident from here. It is clear to you it's a mechanism that is used in order to [00:05:56.100] Convert a given element into an integer. Fine. So now the ones that are hashable in nature are [00:06:05.540] your primitive data structures which are int, float, boolean, strings, complex, they are all hashable in nature. The ones that are not hashable are [00:06:22.052] list, dictionaries, and all of them. Okay? So hopefully it is uh clear to you. Okay. Let's go ahead with the next one. [00:06:41.572] Sorry I'm creating a set [00:06:53.540] So now my set has elements being one, two, three, four, and five. Now what am I trying to do with this particular set? I'll say set [00:07:09.828] Or before that let's find out what are the common operations that we could do it on this set. Let's quickly do it. I'll say L of S. [00:07:19.908] This is five. What is the minimum element in this S? Minimum element is one. And I can can I do max of yes. Max is element is five. I also see some of yes. [00:07:40.068] Okay, some is fifteen in this particular case. Okay. And I'll do one more thing. I'll create something like this. Yes one is equal to hundreds twenty two. [00:07:59.044] and one. Okay. So let's find out. When I say S1, if you can see, if you can see, it's evident that set elements are unordered in nature. The order of the set cannot be determined. Please remember [00:08:19.044] An important attribute is that the set elements are unordered in nature. We can never assume the order of elements. The reason is the way in which I have entered and the way in which it is present internally is different. [00:08:36.516] because of which because of which I cannot do something like this. [00:08:43.588] I cannot do because it does not support indexing. The main reason is the set elements are unordered in nature. Okay, hopefully it is clear to you. Fine. So let me do one more thing. I'll see S1. [00:09:03.940] dot add and I'll say two hundred. This is fine. I said s one dot add which is two hundred. Okay, let's find out what is s one in nature. [00:09:20.036] You could see S1 where 200 is being added. The moment you say add, it adds a well. Where does it add a value? We never know

because it is based on the concept of hashing, which means sets are mutable in nature. [00:09:37.732] sets are mutable. I can change the contents after it has been created. Fine. Remember, please always remember [00:09:49.380] Set is mutable, but it elements are. Its elements are immutable. Fine? Set is mutable in nature, but its elements are immutable. Okay. So now what does it mean? It means that let me create this.

EV0000030 · EV0000031 · EV0000032 · EV0000033

Transcript block 3 (source interval 000600-000899 seconds): [00:10:08.772] One, two, three, and I'll create a list within this. So let me do this and it says unhashable type list, which means my set has a whole [00:10:27.492] is mutable but its elements are immutable, which always requires you to have the values that are hashable objects. Okay? So always remember this. [00:10:45.636] So hope it is clear to you what are the things that we have learned quickly. Set is mutable, it is unordered, duplications are not allowed, it is unique in nature, okay. [00:11:00.900] Fine. It does not support indexing. As it does not support indexing, I cannot also perform slicing operation. Okay. So let's quickly do on one more thing. [00:11:14.436] This is my S L C for I N S print I. [00:11:29.028] And see I've got the values, which means set is iterable. Set is iterable. Okay. So what is the purpose of the set then? What is the use of a set? [00:11:47.364] Sets are basically used in order to remove the duplicate elements, which in turn allows us to have or find the unique elements that are present. [00:12:00.452] Sets are very useful in order to remove the duplicate elements. The moment I remove the duplicate elements, sorry, duplicate elements, I'm going to be having the unique elements that are present. Fine. I can also compare [00:12:16.740] Two I trables for common elements or common differences. You fine? Is this clear? We will also be working on this particular concept. Fine. So it is clear. And there's one more thing that I want to um [00:12:33.316] Tell you. Okay. So this is S, I can say um three in S. Membership operator. I'm talking about a membership operator. Returns true if the element is present in set. [00:12:51.268] returns false if the required element is not present in a set. Okay and I can also do not in it's true because it is not present in the set. [00:13:09.572] Okay, so these are uh a few of the attributes of a particular set. Well quickly uh perform few operations on the sets. Now I just want to show you few things. I'll say D I R of sets. [00:13:27.364] And you can see these are the functionalities that are available. I'm going to be showing you a few of them. The rest of them are left for you to understand. Okay. So let me do this. In order to do, I will try to make use of two different sets, and the values of these sets are [00:13:51.172] Five and six. And uh I will have my S two being [00:14:01.124] Six, seven, eight [00:14:06.532] Want some common elements also, so I will retain five because I can have it in any order, so order doesn't matter. So this I will also say as one [00:14:24.260] Plus true. Okay. So what am I trying to do? I'm trying to perform concatenation operation. So let's find out. [00:14:33.060] So it says concatenation operation is not supported. Okay. Set and set concatenation operation is not supported. Can I do this? Let's find out. S1 plus two. [00:14:51.716] Says still concatenation operation is not supported. Okay. So can I do S1

EV0000034 · EV0000035 · EV0000036 · EV0000037

Transcript block 4 (source interval 000900-001199 seconds): [00:15:03.044] Repetition operation. So it also says repetition operation is not allowed. [00:15:10.244] Is it fine? Okay, so let's quickly go on and understand S1 is this, my S two is this. Okay, so I will quickly do few operations for you. I'll say S1 [00:15:29.668] [ASR low confidence; check audio] dot union of estop. [00:15:37.700] I'll say s1 dot union of s two, which means I am combining the values of s one and s two. Returns the union of sets as a new set for us. [00:15:50.724] Okay, and if you can see again duplicate elements have

been removed, the repetitive elements are not present. I can also say this as one, you can also make use of this particular symbol in order to represent it as what? The union operation. [00:16:10.884] Okay so hope it is clear and I can also do S1 and S2, which gives me intersection. I can also do something like this. Intersection. [00:16:30.852] So I can also do in this manner. So similarly I can also say S1 minus S2. Okay. So [00:16:42.212] S1 minus S2, the ones that are present in S1 and the ones that are present in S2 have been removed from S1. [00:16:51.392] So according to this one, two, three and four because five six is here, five six is also here. The other way is also true for this. I can also say S1 minus S2. Fine. Remove the ones that are. [00:17:07.172] There in both S1 and S two and give the elements of S two only. Okay? So the expected answer would be seven, eight, nine, and ten. Okay. And if you can see even though we said seven, eight, nine and ten, the order has not been maintained. [00:17:25.828] So every time remember the set elements are unordered in nature. Okay. So let me do a few quick things for you. [00:17:42.244] [ASR low confidence; check audio] This is my S1. [00:17:46.692] We will see S1.Remove. [00:17:53.028] S1 dot remove. Which one shall we remove? We'll remove three from here. Okay so it says three has been removed. Is this fine? What does a remove operation will do? It removes the specified item in a set. [00:18:10.628] If an item is not present, okay, now we have already removed three. I'm trying to remove three again. Let's find out what does it do. It throws us an error. So if the element is present, remove will remove it. Otherwise it throws us an error. [00:18:29.508] Okay. Fine. So hope this is uh clear to you. [00:18:38.564] Also do discard [00:18:43.812] Discard what? Discard four is what we will do. We'll try to find out. What does this discard will do? [00:18:50.244] I'll say yes one. It has also removed. Remo also did the same thing for us. Okay. I will try to remove the same item again. And in this case, it does not throw me an error. Remove. Removes a particular element. Discord also does the same thing. [00:19:09.284] If an item is not present, remote throws us an error, but whereas discard does not throw us an error. And S1.pop is an operation. [00:19:26.244] It pops out some element. In this case it has popped out one. Let me pop out one more. So it has popped out two. [00:19:36.772] It has popped out five, in this case it has popped out six, and so on. Okay. Removes an element or an item in a set. [00:19:48.452] In this case, we are lucky enough that the pop operation is removing elements one by one. Is it guaranteed? No, it is not guaranteed. The reason is sets are unordered.

EV0000038 · EV0000039 · EV0000040 · EV0000041

Transcript block 5 (source interval 001200-001499 seconds): [00:20:02.340] Okay, so hope uh these things are clear for you. There are a few more uh things that you can always work out with the set. Fine. I've just shown you a few of them. [00:20:13.860] I request you all um to try out be the other ones because they're all uh self uh uh explanatory. Is this fine? Symmetric difference, symmetric difference update. Wherever you have uh something called as an update. [00:20:29.248] It always does the changes to an existing set. Please remember, whenever you have an update, operation is performed at the same time where it is being done, it is going to be done to that particular set itself. [00:20:45.924] Is this fine? So that is what the uh importance of update. I request you all to try out uh uh with the other functionalities that are present. Okay so hope uh the concepts are uh of sets are clear to you. [00:21:04.740] uh keep uh uh performing the operations. Okay thank you.

EV0000042 · EV0000043 · EV0000044 · EV0000045

Full transcript: 2470 normalized words, 84 timestamped segments, 1270.08 seconds duration. Low-confidence segments flagged: 2. Review locators: 00:15:29.668, 00:17:42.244

EV0000026 · EV0000027 · EV0000028 · EV0000029

4 Files

This lesson is indexed from the source video titled Files. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000046 · EV0000047

Graphs and diagrams

Figure G000057. UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files · source slide page 1.

EV0000247

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING
Introduction

Different ways of giving input to the program:

•Using command line arguments.

•Through the keyboard using input() function.

In both the cases the amount of input given would be **minimal** and also be **prone to errors**.

- To store and deal with large data, Files care helpful.

Figure G000058. UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files · source slide page 2.

EV0000250

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

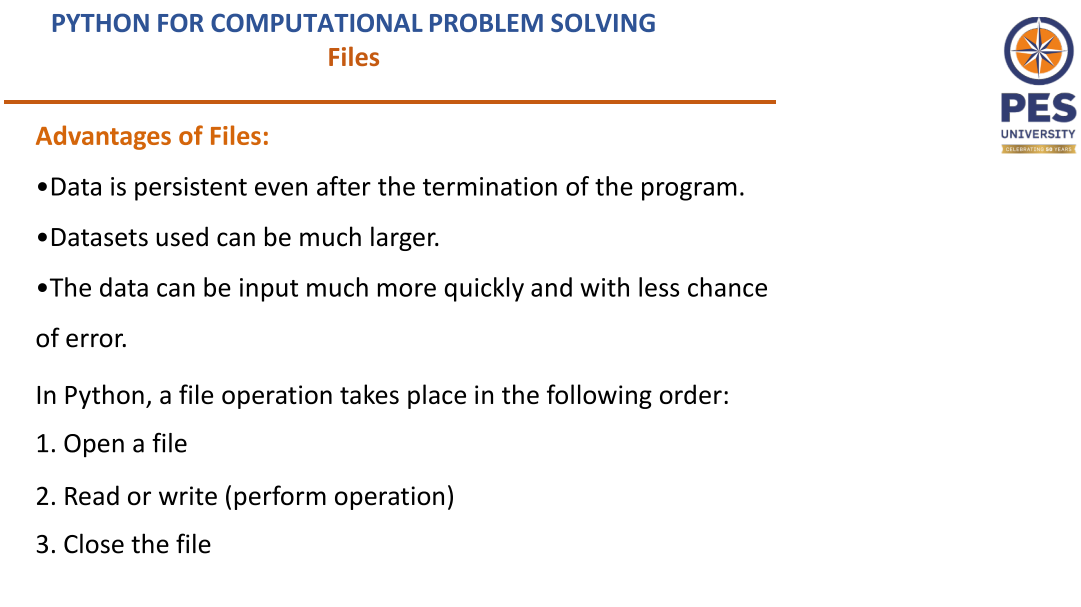


Figure G000059. UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files · source slide page 3.

EV0000253

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING
Files

Opening A File: open() Function

open(file, mode='r', buffering=-1, encoding=None, errors=None, newline=None, closefd=True, opener=None)

file is either a text or byte string giving the **name of the file** to be opened if it is in the current working directory, or the **path to a file** if it is in a different directory.

mode is an optional string that **specifies the mode** in which the file is opened. It defaults
to 'r' which means open for reading in text mode. Other modes are ‘a’, ‘a+’, etc.

We don’t need to worry about the other parameters right now.

Figure G000060. UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files · source slide page 4.

EV0000256

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING
Files

Opening A File

•A file opening **may not always succeed**. If it fails, it throws an exception.

•A few possible exceptions:
- opening a non-existent file for reading.
- opening a file for writing where directory does not allow us to create files.

•If the required file exists, then a file object is returned.

•OS provides the required resources when the file gets created.

Figure G000061. UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files · source slide page 5.

EV0000259

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

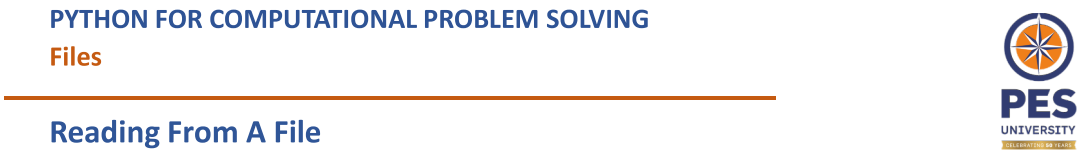


Figure G000062. UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files · source slide page 6.

EV0000262

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

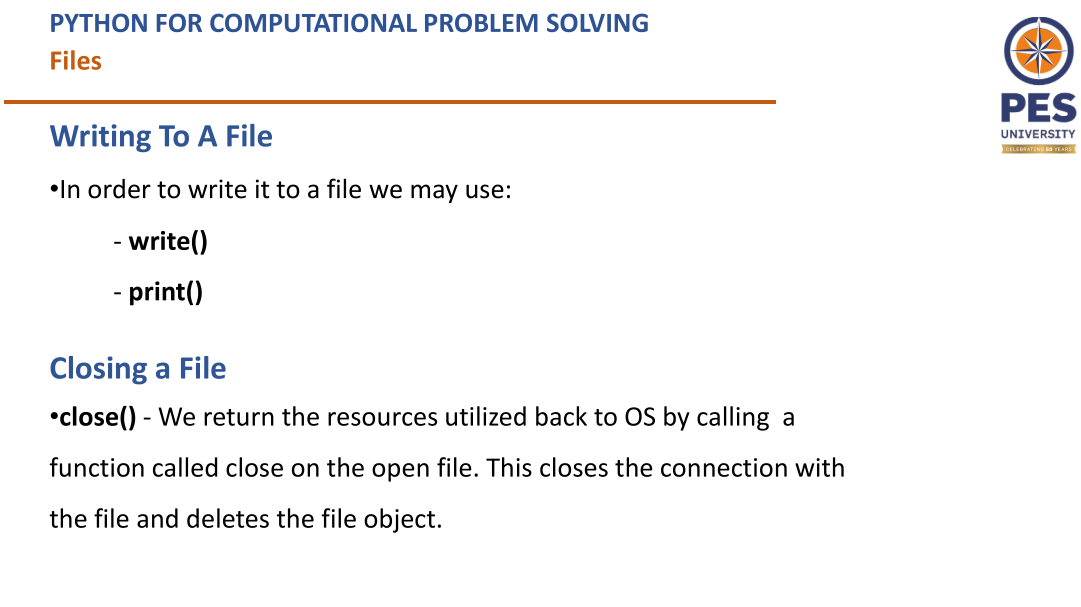


Figure G000063. UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files · source slide page 7.

EV0000265

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

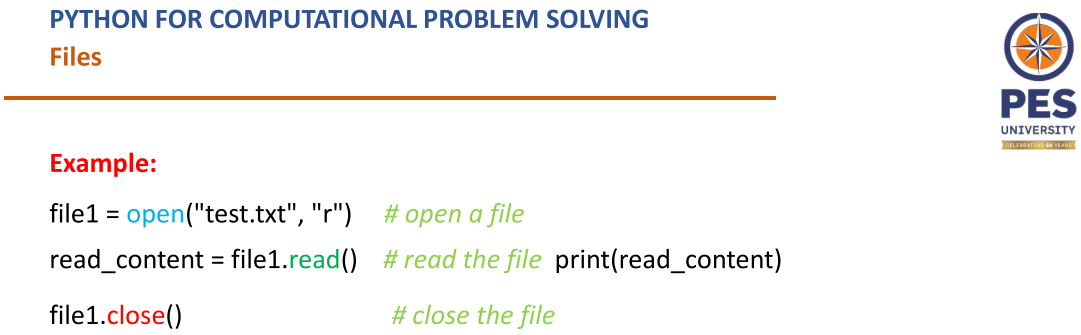


Figure G000064. UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files · source slide page 8.

EV0000268

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

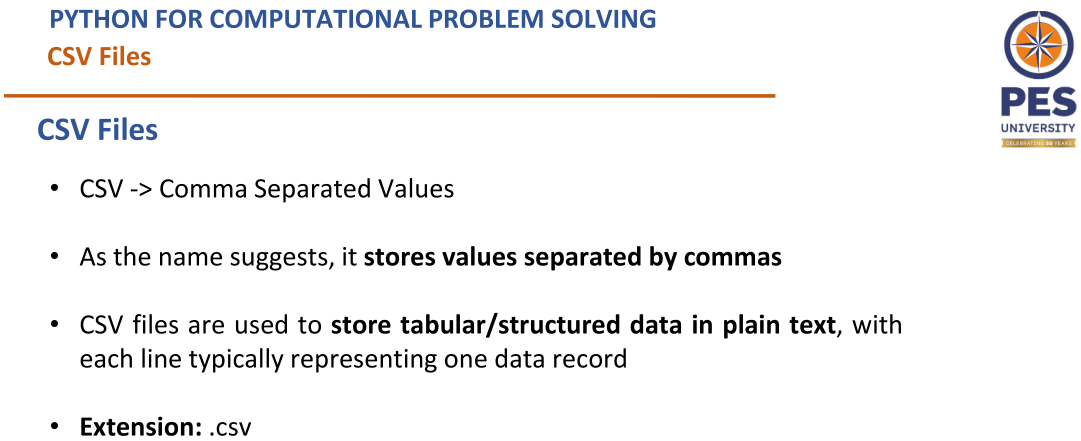


Figure G000065. UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files · source slide page 9.

EV0000271

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

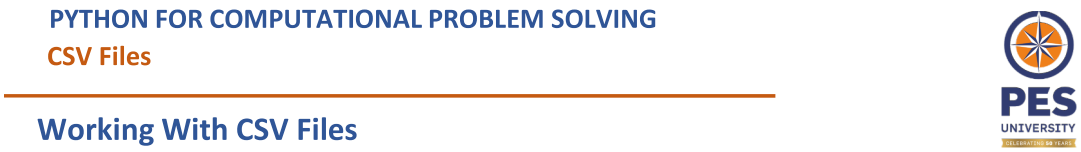


Figure G000066. UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files · source slide page 10.

EV0000274

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

CSV Files

Reading From CSV Files

- **csv.reader(csv_file_name):** Takes a CSV file name string as an argument and returns a CSV reader object
- Let's say your CSV reader object is stored in a variable called **csvreader**.
- The **CSV reader object is iterable**; i.e., we can use methods like **next()** and **looping constructs** to access information stored in it.
- **next(csvreader):** Returns the next row in the csvreader object as a list of strings
- You can also use **loops to access each row** in the csvreader object as a list of strings.

Figure G000067. UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files · source slide page 11.

EV0000277

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

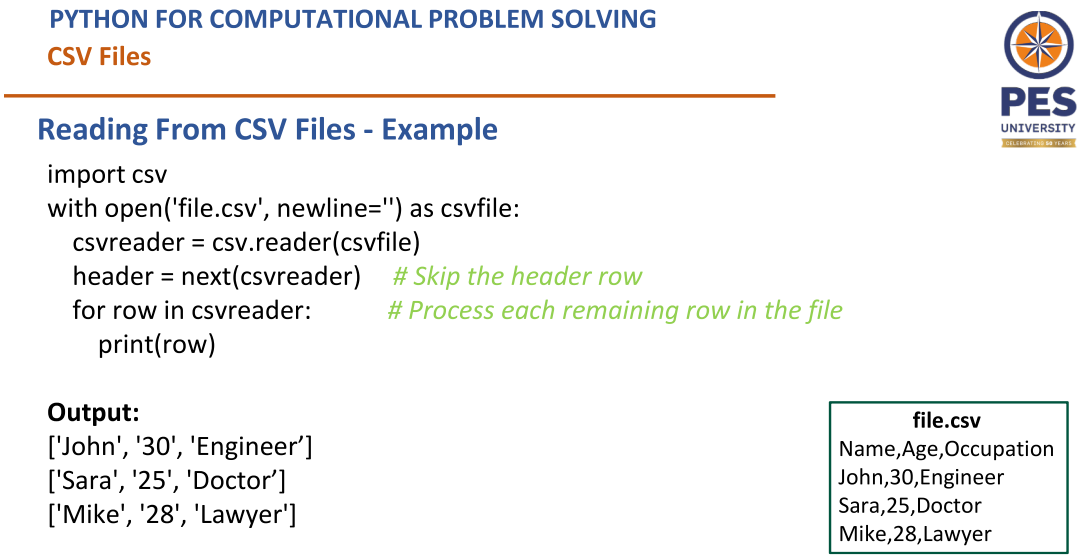


Figure G000068. UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files · source slide page 12.

EV0000280

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

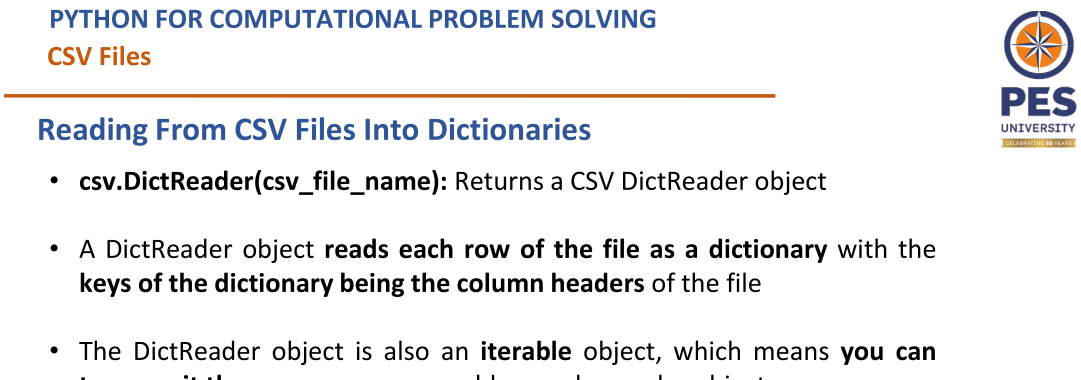


Figure G000069. UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files · source slide page 13.

EV0000283

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

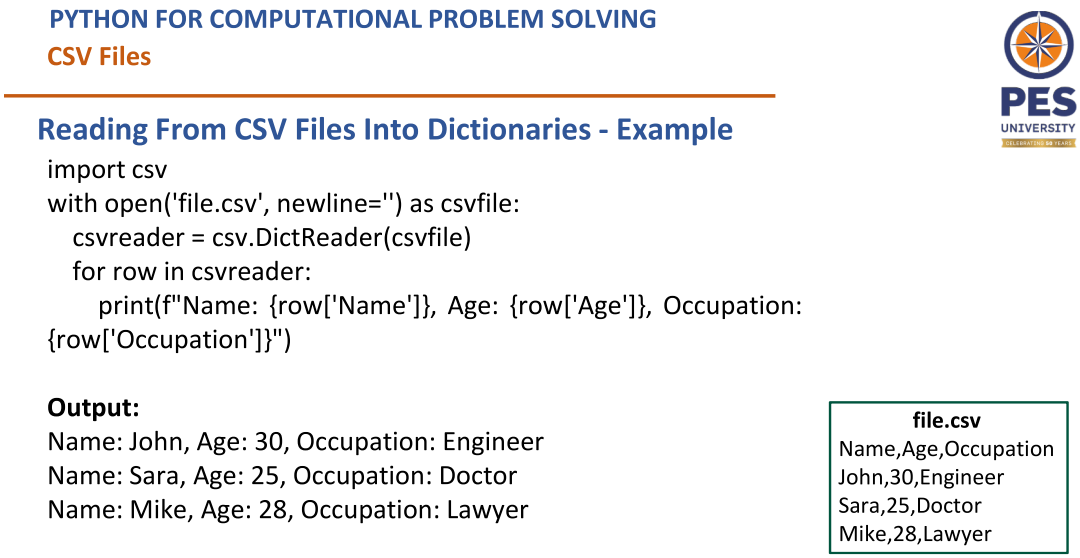


Figure G000070. UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files · source slide page 14.

EV0000286

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

CSV Files

Writing To CSV Files

- **csv.writer(csv_file_name):** Takes a CSV file name string as an argument and returns a CSV writer object
- The writer object writes rows of data to a CSV file, **with each row being a list/tuple in Python**
- Let's say your CSV writer object is stored in a variable called **csvwriter**.
- **csvwriter.writerow() –** Takes a single list/tuple and writes it as a row into the CSV file
- **csvwriter.writerows() –** Takes a list/tuples of lists/tuples and writes each of them as a row into the CSV file

Figure G000071. UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files · source slide page 15.

EV0000289

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

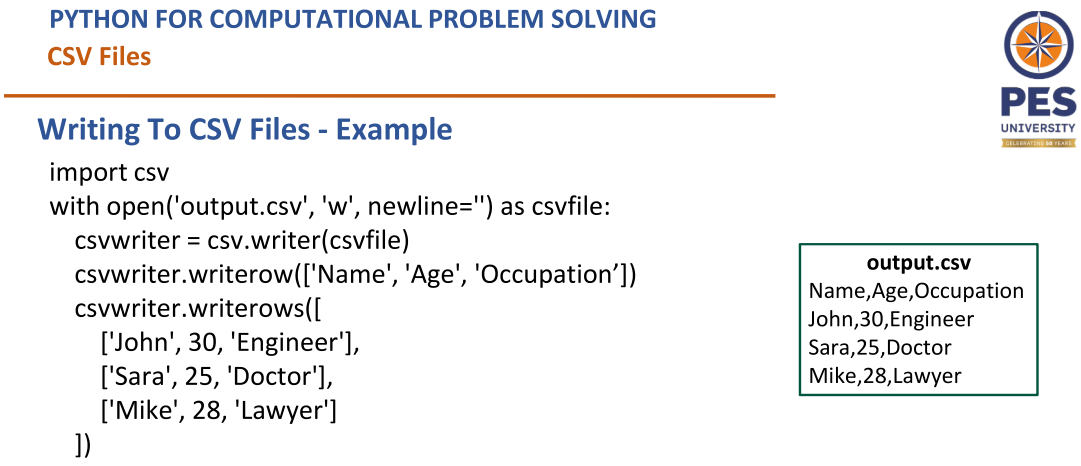


Figure G000072. UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files · source slide page 16.

EV0000292

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

CSV Files

Writing Into CSV Files Using Dictionaries

- **csv.DictWriter(csv_file_name, fieldnames=fieldnames):** Returns a CSV DictWriter object
- Allows you to write to a CSV file using dictionaries, where **each dictionary becomes a row in the CSV file**
- The **fieldnames parameter defines the headers** in the CSV file
- This allows you to **write data easily with named parameters** instead of relying on positional indexes

Figure G000073. UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files · source slide page 17.

EV0000295

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

CSV Files

Writing Into CSV Files Using Dictionaries

- Let's say your DictWriter object is stored in a variable called **csvDictWriter**
- **csvDictWriter.writeheader() –** Writes the headers using the content of fieldnames
- **csvDictWriter.writerows() –** Writes multiple rows of data; dictionaries provided here must have keys identical to the headers in fieldnames

Figure G000074. UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files · source slide page 18.

EV0000298

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

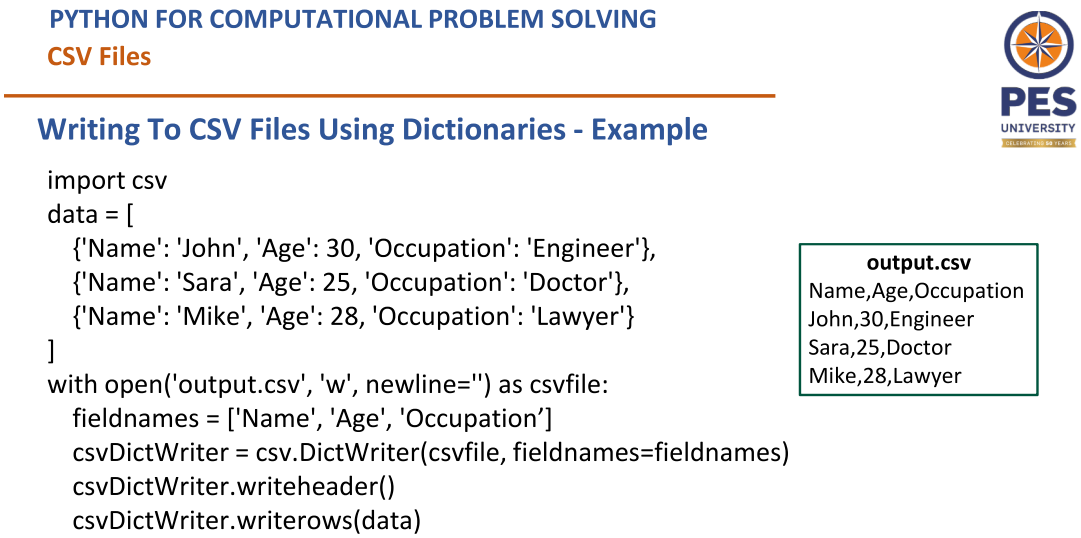


Figure G000075. UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files · source slide page 19.

EV0000301

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

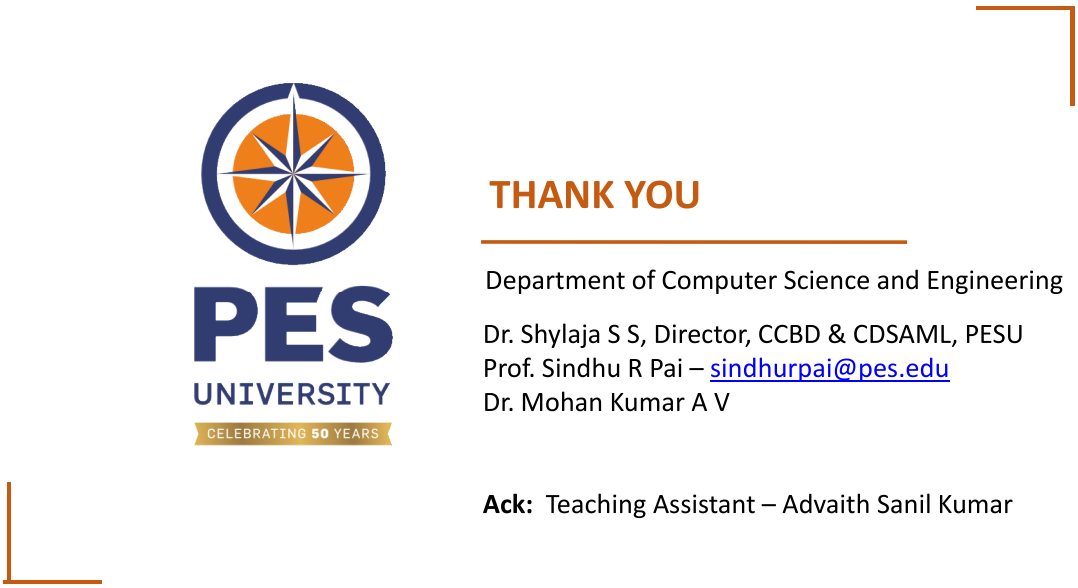


Figure G000076. UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files · source slide page 20.

EV0000304

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING File Handling in Python Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000246 · EV0000247 · EV0000248
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Introduction Different ways of giving input to the program: •Using command line arguments. •Through the keyboard using input() function. In both the cases the amount of input given would be minimal and also be prone to errors. • To store and deal with large data, Files care helpful. EV0000249 · EV0000250 · EV0000251
3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Files Advantages of Files: •Data is persistent even after the termination of the program. •Datasets used can be much larger. •The data can be inputp much more quicklyq y and with less chance of error. In Python, a file operation takes place in the following order: 1. Open a file 2. Read or write (perform operation) 3. Close the file EV0000252 · EV0000253 · EV0000254

4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Files Opening A File: open() Function open(file, mode='r', buffering=-1, encoding=None, errors=None, newline=None, closefd=True, opener=None) file is either a text or byte string giving the name of the file to be opened if it is in the current working directory, or the path to a file if it is in a different directory. mode is an optional string that specifies the mode in which the file is opened. It defaults to 'r' which means open for reading in text mode. Other modes are 'a', 'a+', etc. We don't need to worry about the other parameters right now. EV0000255 · EV0000256 · EV0000257
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Files Opening A File •A file opening may not always succeed. If it fails, it throws an exception. •A few possible exceptions: - openingopening aa nonnon-existentexistent filefile forfor readingreading. - opening a file for writing where directory does not allow us to create files. •If the required file exists, then a file object is returned. •OS provides the required resources when the file gets created. EV0000258 · EV0000259 · EV0000260
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Files Reading From A File After we open a file, we use the read() method to read its contents. File can be read in different ways: •read()•read() -- ReturnsReturns thethe contentscontents ofof thethe filefile asas aa stringstring •readline() - Reads one line from the file and returns it as a string. The string returned by readline will contain the newline character at the end. •readlines() - Returns a list containing each line in the file as a list item. EV0000261 · EV0000262 · EV0000263
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Files Writing To A File •In order to write it to a file we may use: - write() - print()print() Closing a File •close() - We return the resources utilized back to OS by calling a function called close on the open file. This closes the connection with the file and deletes the file object. EV0000264 · EV0000265 · EV0000266
8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Files Example: file1 = open("test.txt", "r") # open a file read_content = file1.read() # read the file print(read_content) file1.close() # close the file EV0000267 · EV0000268 · EV0000269
9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING CSV Files CSV Files • CSV -> Comma Separated Values • As the name suggests, it stores values separated by commas • CSV files are used to store tabular/structured data in plain text, with each line typically representing one data record • Extension: .csv EV0000270 · EV0000271 · EV0000272
10. Source slide page 10 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING CSV Files Working With CSV Files • Process remains the same: open, perform operations, close • However, to perform operations on a CSV file, we use a different PythonPython module:module: thethe csvcsv modulemodule. ThisThis hashas toto bebe importedimported intointo youryour program when you're working with CSV files. • The csv module allows you to create CSV reader and writer objects that will be used to interact with the CSV file without having to worry about maintaining or dealing with the formatting of CSV files. EV0000273 · EV0000274 · EV0000275

11. Source slide page 11 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING CSV Files Reading From CSV Files • csv.reader(csv_file_name): Takes a CSV file name string as an argument and returns a CSV reader object • Let's say your CSV reader object is stored in a variable called csvreader. • The CSV reader object is iterable; i.e., we can use methods like next() and looping constructs to access information stored in it. • next(csvreader): Returns the next row in the csvreader object as a list of strings • You can also use loops to access each row in the csvreader object as a list of strings. EV0000276 · EV0000277 · EV0000278

12. Source slide page 12 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING CSV Files Reading From CSV Files - Example import csv with open('file.csv', newline='') as csvfile: csvreader = csv.reader(csvfile) header = next(csvreader) # Skip the header row for row in csvreader: # Process each remaining row in the file print(row) Output: file.csv ['John', '30', 'Engineer'] Name,Age,Occupation ['Sara', '25', 'Doctor'] John,30,Engineer Sara,25,Doctor ['Mike', '28', 'Lawyer'] Mike,28,Lawyer EV0000279 · EV0000280 · EV0000281

13. Source slide page 13 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING CSV Files Reading From CSV Files Into Dictionaries • csv.DictReader(csv_file_name): Returns a CSV DictReader object • A DictReader object reads each row of the file as a dictionary with the keysy of the dictionaryy beingg the column headers of the file • The DictReader object is also an iterable object, which means you can traverse it the same way you would a regular reader object EV0000282 · EV0000283 · EV0000284

14. Source slide page 14 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING CSV Files Reading From CSV Files Into Dictionaries - Example import csv with open('file.csv', newline='') as csvfile: csvreader = csv.DictReader(csvfile) for row in csvreader: print(f"Name: {row['Name']}, Age: {row['Age']}, Occupation: {row['Occupation']}") Output: file.csv Name: John, Age: 30, Occupation: Engineer Name,Age,Occupation Name: Sara, Age: 25, Occupation: Doctor John,30,Engineer Name: Mike, Age: 28, Occupation: Lawyer Sara,25,Doctor Mike,28,Lawyer EV0000285 · EV0000286 · EV0000287

15. Source slide page 15 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING CSV Files Writing To CSV Files • csv.writer(csv_file_name): Takes a CSV file name string as an argument and returns a CSV writer object • The writer object writes rows of data to a CSV file, with each row being a list/tuple in Python • Let's say your CSV writer object is stored in a variable called csvwriter. • csvwriter.writerow() – Takes a single list/tuple and writes it as a row into the CSV file • csvwriter.writerows() – Takes a list/tuples of lists/tuples and writes each of them as a row into the CSV file EV0000288 · EV0000289 · EV0000290

16. Source slide page 16 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING CSV Files Writing To CSV Files - Example import csv with open('output.csv', 'w', newline='') as csvfile: csvwriter = csv.writer(csvfile) output.csv csvwriter.writerow(['Name', 'Age', 'Occupation']) NameName,Age,OccupationAge Occupation csvwriter.writerows([John,30,Engineer ['John', 30, 'Engineer'], Sara,25,Doctor ['Sara', 25, 'Doctor'], Mike,28,Lawyer ['Mike', 28, 'Lawyer']]) EV0000291 · EV0000292 · EV0000293

17. Source slide page 17 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING CSV Files Writing Into CSV Files Using Dictionaries • csv.DictWriter(csv_file_name, fieldnames=fieldnames): Returns a CSV DictWriter object • Allows you to write to a CSV file using dictionaries, where each dictionary becomes a row in the CSV file • The fieldnames parameter defines the headers in the CSV file • This allows you to write data easily with named parameters instead of relying on positional indexes EV0000294 · EV0000295 · EV0000296
18. Source slide page 18 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING CSV Files Writing Into CSV Files Using Dictionaries • Let's say your DictWriter object is stored in a variable called csvDictWriter • csvDictWriter.writeheader() – Writes the headers using the content of fifieldnamesld • csvDictWriter.writerows() – Writes multiple rows of data; dictionaries provided here must have keys identical to the headers in fieldnames EV0000297 · EV0000298 · EV0000299
19. Source slide page 19 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING CSV Files Writing To CSV Files Using Dictionaries - Example import csv data = [{'Name': 'John', 'Age': 30, 'Occupation': 'Engineer'}, output.csv {'Name':{ 'Sara',, 'Age':g 25,, 'Occupation':p 'Doctor'},}, NameName,Age,OccupationAge Occupation {'Name': 'Mike', 'Age': 28, 'Occupation': 'Lawyer'} John,30,Engineer] Sara,25,Doctor Mike,28,Lawyer with open('output.csv', 'w', newline='') as csvfile: fieldnames = ['Name', 'Age', 'Occupation'] csvDictWriter = csv.DictWriter(csvfile, fieldnames=fieldnames) csvDictWriter.writeheader() csvDictWriter.writerows(data) EV0000300 · EV0000301 · EV0000302
20. Source slide page 20 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shylaja S S, Director, CCBD & CDSAML, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Dr. Mohan Kumar A V Ack: Teaching Assistant – Advaith Sanil Kumar EV0000303 · EV0000304 · EV0000305

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:01.316] Hello, welcome back to Python for computational problem solving. In today's session, we are going to be discussing the concepts of files. So let's get started. [00:00:25.028] So till now what we have done is we have worked with so many examples where input to the program where input to the program was in terms of a function called as an input. [00:00:39.120] So we tried giving an input through the keyboard. So there is also one more way of giving an input, in which case I can also pass on the values as the inputs by making use of a command line arguments. Command line arguments. [00:01:02.020] So let's quickly look into how do we give an input through the command line arguments. So take a simple example in this case. So I will say import sys. [00:01:15.940] system. Okay. My main intention is that I want to perform addition of two numbers [00:01:25.796] addition of two numbers using command line arguments. Okay. So in this case I'll say it as import system and I will also say X is equal to in [00:01:46.020] of system. I'm gonna be explaining you what exactly these means and so don't worry. So I will also say okay. So similarly I will copy paste [00:02:03.972] Right? And this is my Y, but my argument in this case could be two. Okay. So what I really want to do, I want to find out the sum. So I will say sum. [00:02:21.604] Sum is equal to X plus Y. Okay. And I want to previously let me print the values how it was initially. Okay. So I will say print. [00:02:38.948] Okay, so I'll remove these things. Anyway, I don't need them. [00:02:45.572] Goodbye. [00:02:52.772] I will be doing it. [00:02:56.932] I want to find out what is the um initial value and how will I give the um how will I get the uh final result. So let's quickly

execute this. Fine, and uh when I do this, just look at what happens. [00:03:13.956] It says index list index out of range. The reason is when I am passing an input as a command line arguments, my input also should be a part of the command line itself. So in this case I'll say 20 followed by 30. So now [00:03:30.096] This is R g of zero, this is R g of one, and R V of two. So that is what we are trying to do. My final output I'm expecting is a fifty in this particular case. So if you can see [00:03:43.748] We have successfully executed giving an input through the command line arguments. Previously you would have already seen uh the other way of giving inputs. Okay. You want it to be an integer, so you say it as an int of inputs. Similarly [00:04:07.108] Okay, so this is Y and C print sum is equal to X comma Y. Now let's execute uh both of them. Okay, let's see how does the both of them execute. [00:04:23.972] Right. So it says fifty and your cursor is still blinking. I will give an input called as twenty, the other one is thirty again. So you're also getting fifty as an answer when both the programs are executed. Okay. So in both the cases [00:04:39.440] The inputs that we are trying to give is very minimal. What if I say to give an input using command line arguments where the number of input will be hundred. [00:04:49.840] humanly impossible to specify hundred different values on the command line. The reason may be we may have some errors while giving in the sense I may end up just giving 99 or I may end up giving one out one also. So it is not sure.

EV0000046 · EV0000047 · EV0000048 · EV0000049

Transcript block 2 (source interval 000300-000599 seconds): [00:05:04.704] Similarly, when I'm trying to give an input using the variables, maybe using hundred different variables may be uh tough, but at the same time uh you can also make use of a for loop and then specify the input also. But in both the cases it is not feasible. Okay [00:05:23.696] Uh there may be a situation where you want to work with a particular application where the amount of data that is required is more for you. The larger data is required. So in such case, it is always better to make use of files. [00:05:40.900] Hopefully uh it is clear to you why we need files. So what are the advantages of files? Data is persistent, even after your program terminates, the data is persistent. You're going to be working with larger amount of data. [00:05:56.228] And at the same time, it's chances of errors are very less when uh input is given through a particular file. Okay. So thus what is a file? [00:06:07.940] A file is some information or data which stays in a computer storage device, especially in computer memory. So files are basically divided into two different types. One is called as a binary, and the other one is called as a text file. [00:06:27.876] Okay. Binary files, it contains the sequence. Okay, it's in it's a collection of bytes. [00:06:34.084] And uh very difficult uh for human to read. So it is uh human readable format is not supported by the binary files. At the same time, when you have a text file, you can store data in the form of alphabets, digits and other special symbols. [00:06:50.192] Okay, uh where the information is stored in terms of the ASCII values. Okay, and at the same time it is uh human readable format. So it is very clear that we are going to be working with the text files itself. [00:07:09.220] In this um session we are going to be working with the text files. So [00:07:20.804] When I say we will be working with the files, a file may be used for a reading, or it can be used for writing. May be used for reading or it can be used for writing purpose. [00:07:38.340] When I see a file, I can use an existing file, or I can create a new file, or I can overwrite an existing file. [00:07:53.668] You can make use of an existing file, so overwrite, both means the same in this case, or you can create your own files. Okay at the same time, whenever you are trying to perform some operation. What operations? Reading and writing. You may also specify reading as R. [00:08:14.564] Writing as W, reading as R and writing as W. Where R and W represent modes. R and W represents modes. [00:08:26.928] There are other

modes also, but they are out of scope for our syllabus, so we are going to be discussing only a read and write operation. So when I want to work with the file, there are series of operations that I have to do with the file. One is open a file. [00:08:46.884] Perform operation. Okay. What operation? Reading or writing operation. At the same time you will also have to close the file. You will also have to close the file. [00:09:00.420] Hope it is uh clear. Okay. So let us look at uh uh where uh I'm going to be opening a file, writing few things and all those things that we will try to do it. Okay, so let's start with this. So when I say opening a file [00:09:19.940] Oh fine. [00:09:25.092] If a file that I want to open or before that, in order to open a file, I will make use of a function called as open. [00:09:33.008] Okay. So now what goes as a parameter to this particular function, I will have to specify which file that I want to open. So in this case, maybe I want to open t2.txt. I want to open it only for the reading purpose. [00:09:48.992] Okay. So this is what I will try to do. Now in this case if t2.txt is in existence, which means if it is present, then open function will return an object. Okay. Open function will return an object for this particular

EV0000050 · EV0000051 · EV0000052 · EV0000053

Transcript block 3 (source interval 000600-000899 seconds): [00:10:06.208] text file. So I will capture this object in a particular variable. So what is a f in this case? F is a file object. On success, so it gives me if t2.txt does not exist, it throws me an error. [00:10:23.296] Let me repeat. If one success, it returns a file object. If it is not present, it throws me an error. Okay. So let's quickly this. [00:10:37.732] So files dot BY and it says that oh sorry. [00:10:44.036] It's still executing. Maybe I haven't commented on it. So one second, sorry, let me comment. We don't need them. Okay, so I have uh commented. [00:10:59.268] So we will execute this files.py. In this case, what am I trying to do? I'm trying to open a particular file. So as said, if it is present, returns a a file object. If it is not present, it throws us in. [00:11:15.568] Error. So it says t2.txt does not exist. File not found error has been thrown. [00:11:23.556] Okay, so what we will do is we will have to create a particular file. Okay. When you create a file, make sure your dot txt and dot p y resides in a same part. [00:11:42.500] resides in a same path. Otherwise you will have to specify the entire path where your text file is available. Okay. So now what I will do is I have already created a TWin.txt [00:11:59.364] In and uh both my.py files and both.txt files are present on the desktop at this particular point. Okay so now what I will do is I will change it as tvin.txt. [00:12:12.864] Okay. So now if t1.txt is success, I just want to print [00:12:22.116] F. What is that output? Let's not worry about it. Let's not worry. So let me check with this. [00:12:30.756] And it says it is success because twin.txt was already available for us, so and I was successfully uh able to open the text file. Okay. So hope uh this is clear to you. This is how we try to open a particular file. [00:12:48.420] Okay, so when I say a read operation, when I say a read operation, there are different ways in which a reading can be done. [00:12:58.788] There are different ways to perform a read operation. One is the first one is by making use of something called as read line. [00:13:11.332] There is something called as a read line. The other one is read lines. [00:13:19.908] And the last one is actually read. Read comes with and without parameter. Okay. Read comes with and without parameters. So uh let's uh look into each one of them. [00:13:33.508] Okay, so what I have done is I've already entered some um information in the text file. [00:13:40.932] Okay, so if you can see this is what my text file basically contains. This is Python files dot txt and dot py should be in the same location. This is what the contents of my text uh sir. Yes, T1.txt file. Okay so [00:13:58.128] Let's go back here and uh we will perform these operations on that particular text file. Okay. So let's see to this. Let me execute again. I do not want this. [00:14:14.372] Comment so let's go ahead

and see F is equal to open T1 dot TXT. [00:14:27.876] in read mode. We are asking you to open. I want you to open it in a read mode. And I want to read [00:14:36.324] Okay, so what does this from now onwards whenever I want to work with t1.txt I would make use of f as an handler. Okay, f is an handle for t1.txt. So now I'll say on the t1.txt I want to perform a read line. [00:14:54.852] Okay, I want to read one line at a time. Okay. So what I will do is I will have to assign it to something which I'll say read line.

EV0000054 · EV0000055 · EV0000056 · EV0000057

Transcript block 4 (source interval 000900-001199 seconds): [00:15:05.060] Okay it could be anything of your choice variable name for the convenience I've used it as read line. So I'll say print read line. So print read line now look at the output. [00:15:28.836] If you can see it has read the first line from the file, but there is a huge gap between this and our prompt. [00:15:37.860] Is this fine? The reason is the function read line, it reads a line from a file, in the sense read line is reading this from a file along with reading, it also includes a new line character. [00:15:50.496] Okay. It also includes a new line characters at the end of the line in a file. Along with that. [00:15:58.096] Along with that, you do have a print statement. By default, print statement already has a new line within it, because of which I am getting two lines as an extra line here. [00:16:10.596] Fine. So is that okay with you? No, we do not want any extra lines to be available. So what do I need to do? I need to get rid of this particular line. [00:16:20.704] Okay so in order to get rid of this particular line what I will do is I will make use of a function called as a strip function. So I'll say read line, it has given you already an answer. So read line in that strip. Say strip. [00:16:42.596] read line dot strip. Make sure you assign it to the same variable. Okay? A sign it to the same variable. Let's look into this now and find out the difference. [00:16:56.100] Okay, if you can see we have got rid of one line from this to this because print statement already has a new line because of which we are getting this particular line. If you still want to get rid of that also, what you can do is you can specify [00:17:11.792] And is equal to nothing. So in this case [00:17:16.836] We will not have that extra line also in the output. Okay. So hope this is clear to you. So wherever you have an extra line, make sure that you are going to be using a strip function in order to remove that particular extra line. [00:17:34.564] Okay so hope it is clear. Fine. So similarly we will go to the next one and I will just copy paste a few things so that I want to make my life easy. [00:17:51.908] Okay, control V. In this case I will use read. Okay. So I will make use of uh read. Read is a function which reads your entire file. [00:18:05.220] Read is a function which basically reads your entire file. Okay, whole file is read in this particular case. So let's execute. [00:18:18.148] And see the entire file has been read. Okay? Or if you are interested, you're also free to read only certain bytes of information from your particular file. [00:18:31.268] This is also possible. So read comes with two different options where you can specify the number of bytes that you want to read. At the same time, you can also read the entire file. [00:18:45.124] Okay. So this is the usage of reading a file. Okay. There are other ways of reading a file where you can uh iterate over uh using your while loop and then uh get each line [00:19:00.484] using your read line and uh print the same. Uh any extra lines you can make use of your strip function and get rid of a particular uh line. Okay so that uh code is also being uh given as a part of the notes please uh kindly go through it. [00:19:19.140] Okay, so the next one is we went through read line, we also had a read option and there is one more which is called as read lines. Okay, so let me make use of this. [00:19:38.340] I will use these two lines. I don't want the previous one, so I will comment it. So I have commented. Now what I will do is I will say read the lines. [00:19:58.468] A read lines reads the whole file into a list.

EV0000058 · EV0000059 · EV0000060 · EV0000061

Transcript block 5 (source interval 001200-001499 seconds): [00:20:04.004] Please remember read file reads the entire or a whole file into a list. Okay? Where each element of the list is a line of a file. Where each element of a list is a [00:20:20.548] line of a particular file. So let's also go through this. Print read lines. [00:20:34.276] So let's execute and see you're getting every line as an element of a particular list. So if you want to access what you can do, you can definitely write a for loop and then access each element and then try to print. [00:20:54.372] Okay, so hopefully uh this is also clear for you. Okay, so read read lines followed by read line. These are the few things that we have learned. Okay next one is [00:21:15.204] to perform writing operation to a file. So let's find out how do I write it to a file. [00:21:24.676] write to a file. So if I want to write to a file, I have to open it in a right mode. Okay so I will say f is equal to open [00:21:41.284] Some file where we will say or I will make use of the same file t1.txt itself I will make use. [00:21:51.940] Okay? But I have to open it in a write mode. The moment you open it in a write mode, your previous content will be lost. Okay? So the new content you can write. [00:22:02.660] Okay, so what I will do is I have opened it in a write mode. If you can remember my previous contents were this is Python files, dot txt and dot py should be in the same location. So this was the content of my particular file. Okay. [00:22:21.284] What we will do is we will try to print. Okay, print hello. Right? I just want to print hello. To which one? To a file. [00:22:34.116] Even.txt whose handle is F. If you remember, I just want to show it to you. When we did your print statement. When we did your print statement. [00:22:50.852] Okay, if you can see the print statements, give me time. Okay. File is equal to system dot standard output. So value and number of values that you give, which are separated by comma, separators field was space, and field was slash. [00:23:09.828] You wanted your output to be available. All this time your output was available on the standard output. But right now, whatever you are writing, you want your output to to be available in a particular file, which is t win.txt. So now what we will do is we will write. [00:23:28.324] to that. Okay. F sorry. File is equal to F. Okay. So I am trying to write it N. [00:23:40.900] uh t1.txt. So just to show you this is the content of my current file, t1.txt. Now let me execute [00:23:54.532] Exits I'll say python files.py and it has successfully executed. Now go back to your file. [00:24:10.372] Okay, file t win.txt in a right mode. So let's close this and uh try to open even.txt. [00:24:25.604] Fine, and if you can see Twin.txt at this point has something called as an hello. Okay why previously it was not shown. [00:24:34.276] We had opened it long back, so I just closed the file and then opened it for viewing the content. So this is how you also write into a file. In the notes, we have also given you using a write statement, okay, write as a function. [00:24:50.672] Um we strictly um tell you to just make use of a print statement in order to write it to a file. Okay. So in the uh coming sessions we will also try to work with a few example programs. Hope you have understood the concept of uh files. Thank you.

EV0000062 · EV0000063 · EV0000064 · EV0000065

Full transcript: 3330 normalized words, 104 timestamped segments, 1512.085333 seconds duration. Low-confidence segments flagged: 0.

EV0000046 · EV0000047 · EV0000048 · EV0000049

5 Function_Call_Definition_Variable_arguments

This lesson is indexed from the source video titled Function_Call_Definition_Variable_arguments. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence.

EV0000066 · EV0000067

Graphs and diagrams

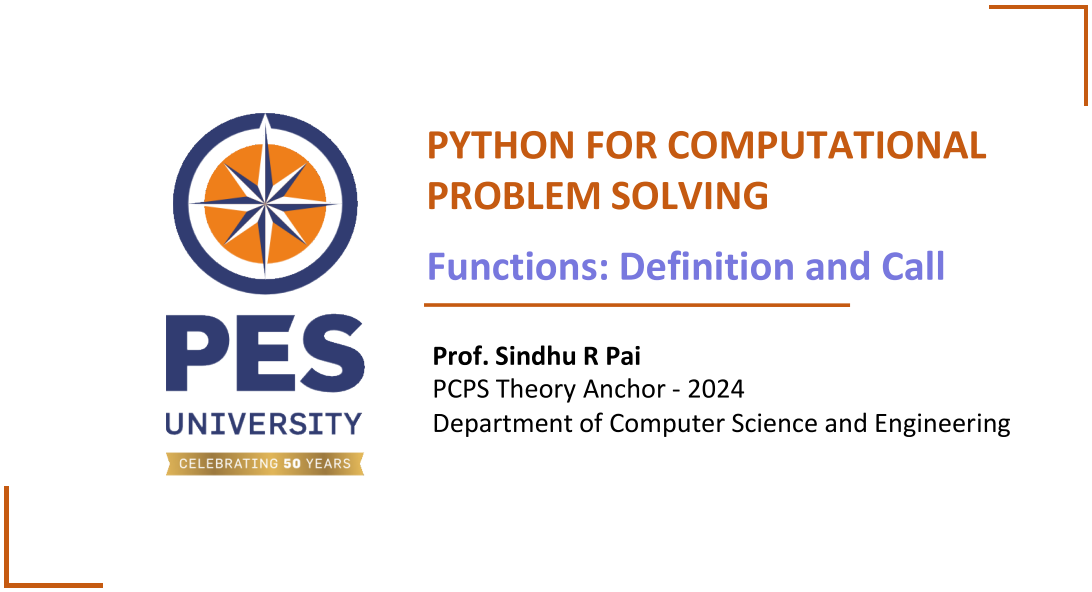


Figure G000090. UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call · source slide page 1.

EV0000346

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

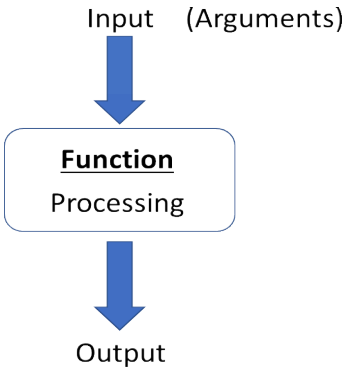
PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Course Overview

PES UNIVERSITY

What are functions?

- A function is a **self contained block of code** that performs a specific task
- Functions ideally **take input**, performs a set of **operations** and **returns an output** to the program that invoked it.



1

Figure G000091. UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call · source slide page 2.

EV0000349

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

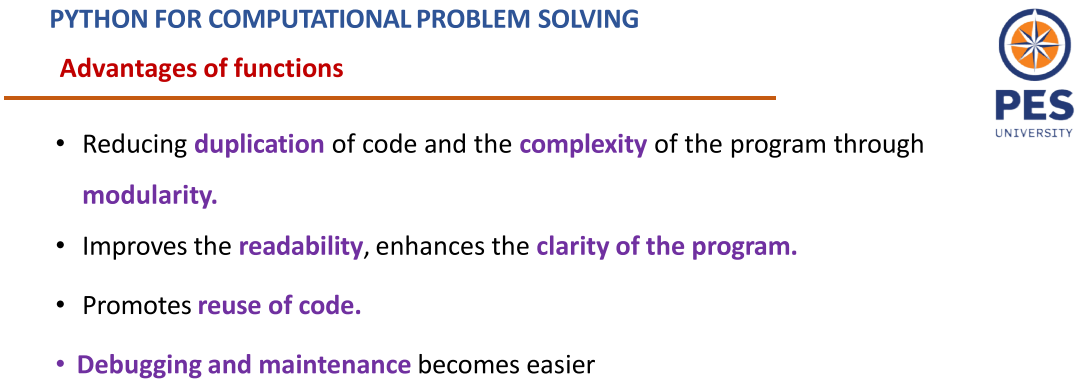


Figure G000092. UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call · source slide page 3.

EV0000352

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

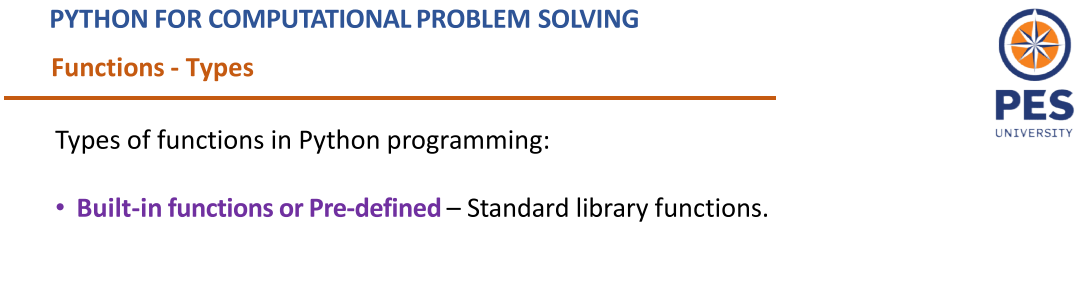


Figure G000093. UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call · source slide page 4.

EV0000355

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

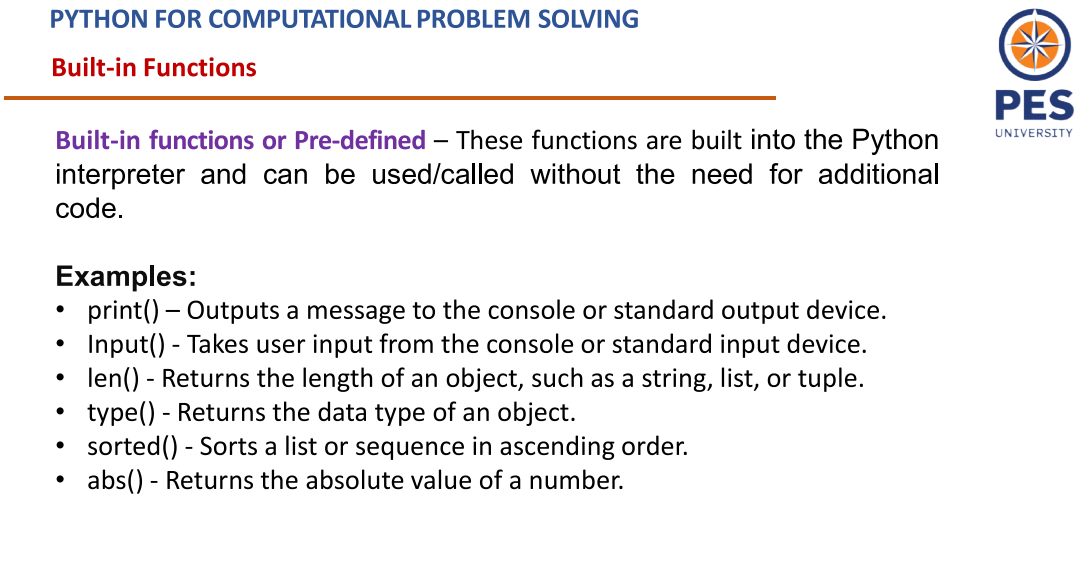


Figure G000094. UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call · source slide page 5.

EV0000358

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

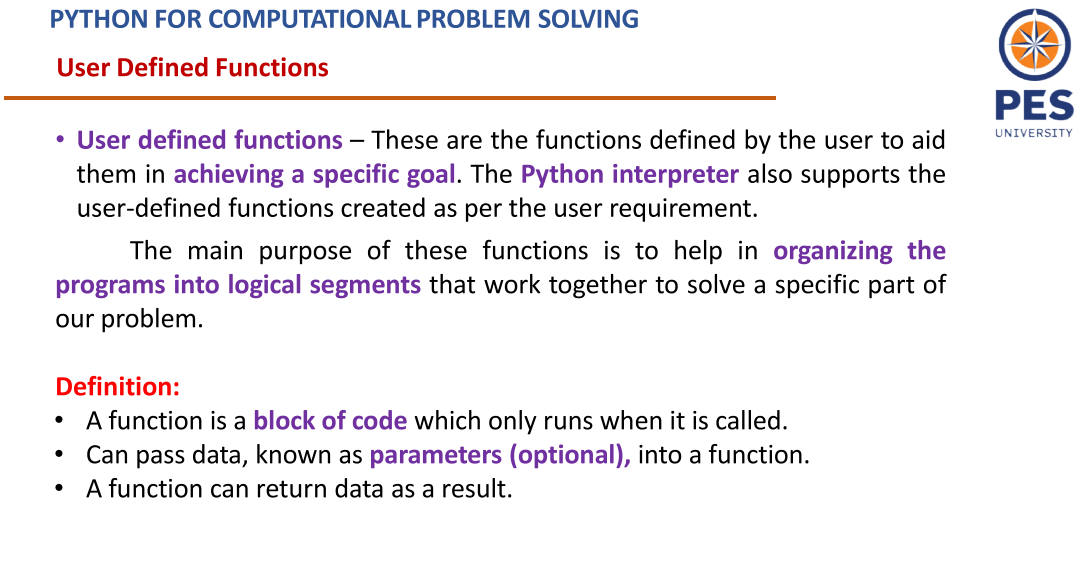


Figure G000095. UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call · source slide page 6.

EV0000361

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

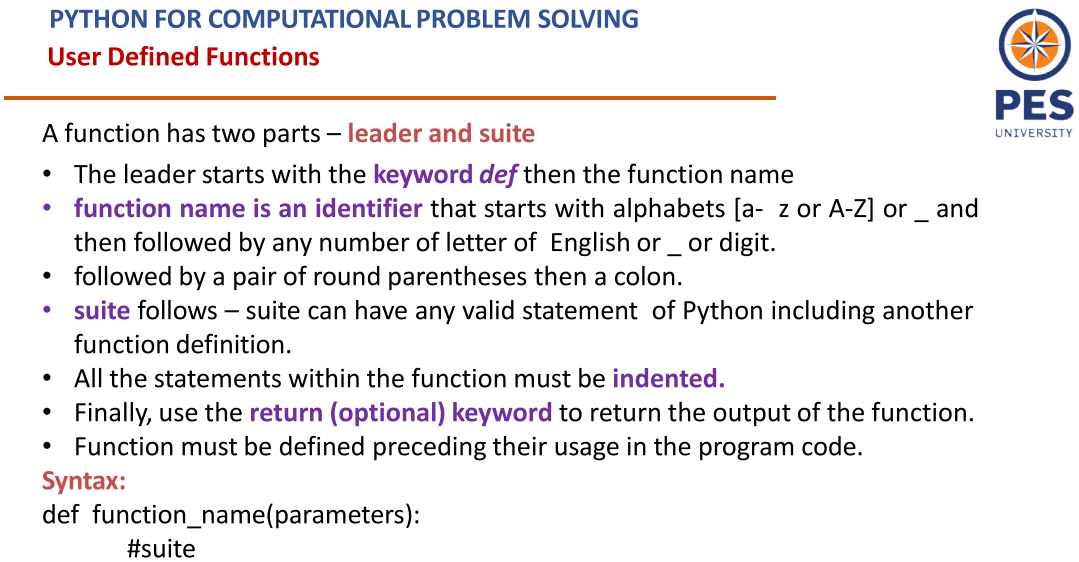


Figure G000096. UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call · source slide page 7.

EV0000364

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Processing of Functions

PES UNIVERSITY

- When the function is defined, header or the leader is processed; the user is provided a name or a handle which is the same as the function name.
- A function entity with the function name in the definition along with the suite is stored.
- Each entity in Python has a reference count.
- At this point of translation, only leader is processed and the suite is not processed.

Figure G000097. UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call · source slide page 8.

EV0000367

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

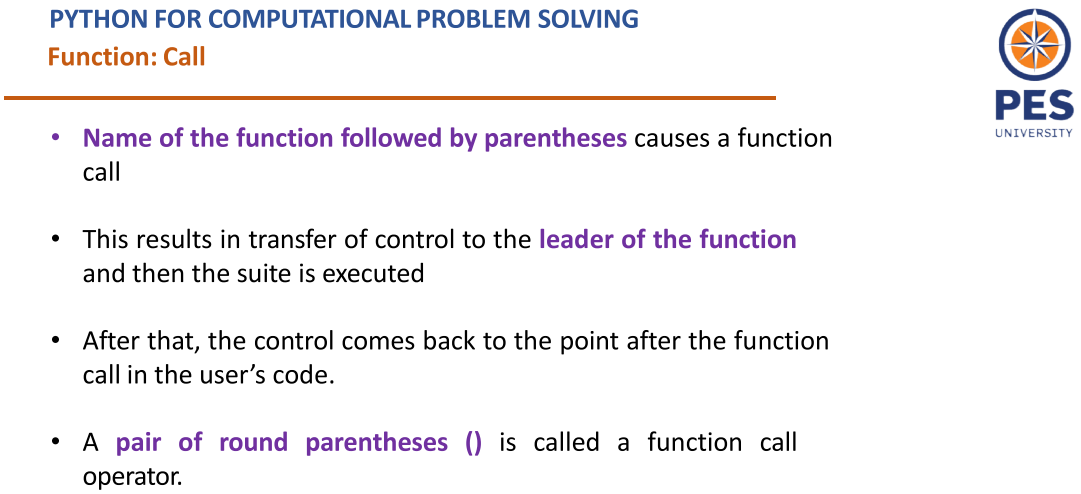


Figure G000098. UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call · source slide page 9.

EV0000370

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

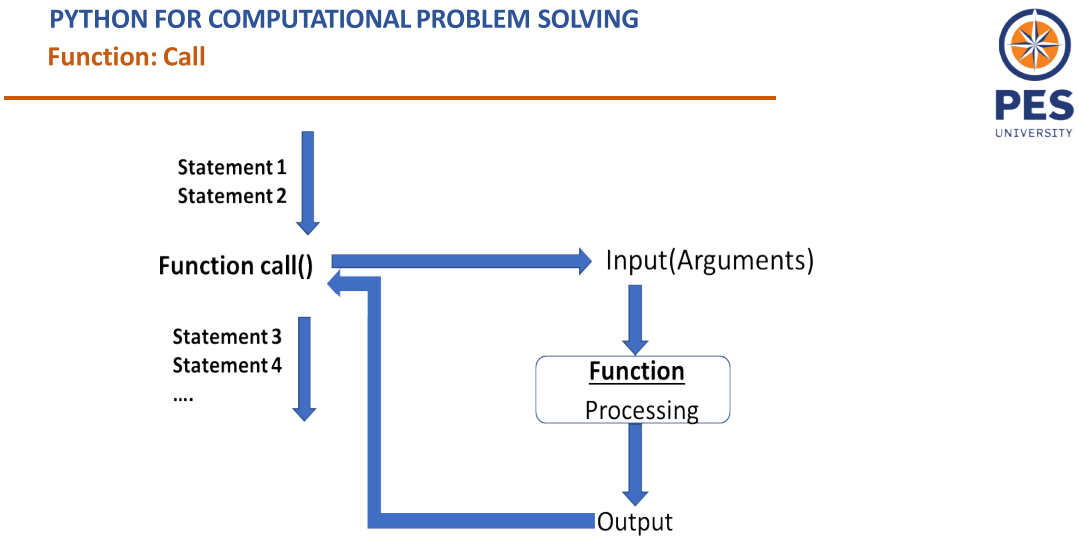


Figure G000099. UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call · source slide page 10.

EV0000373

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

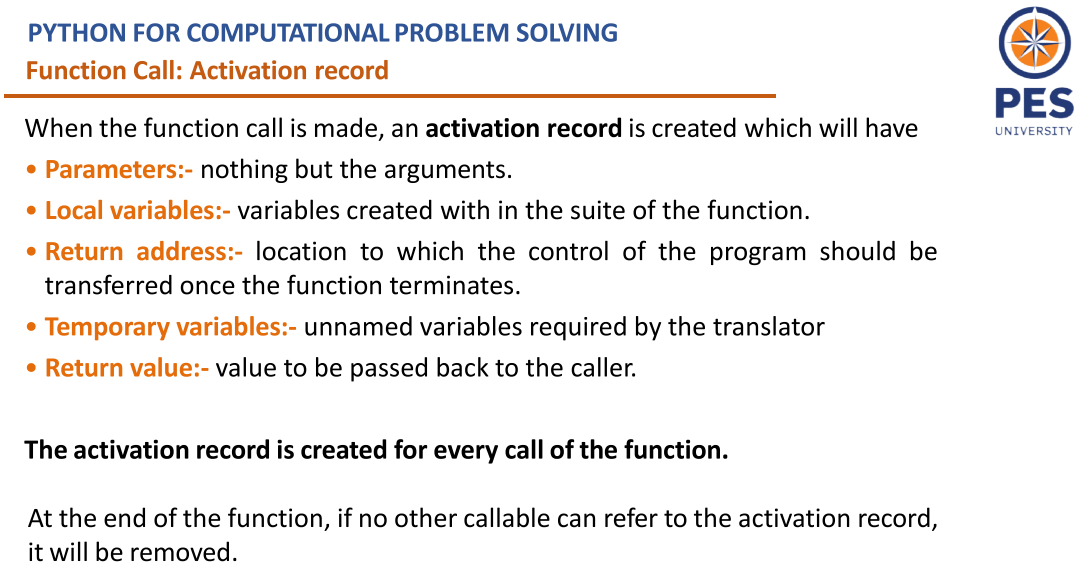


Figure G000100. UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call · source slide page 11.

EV0000376

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

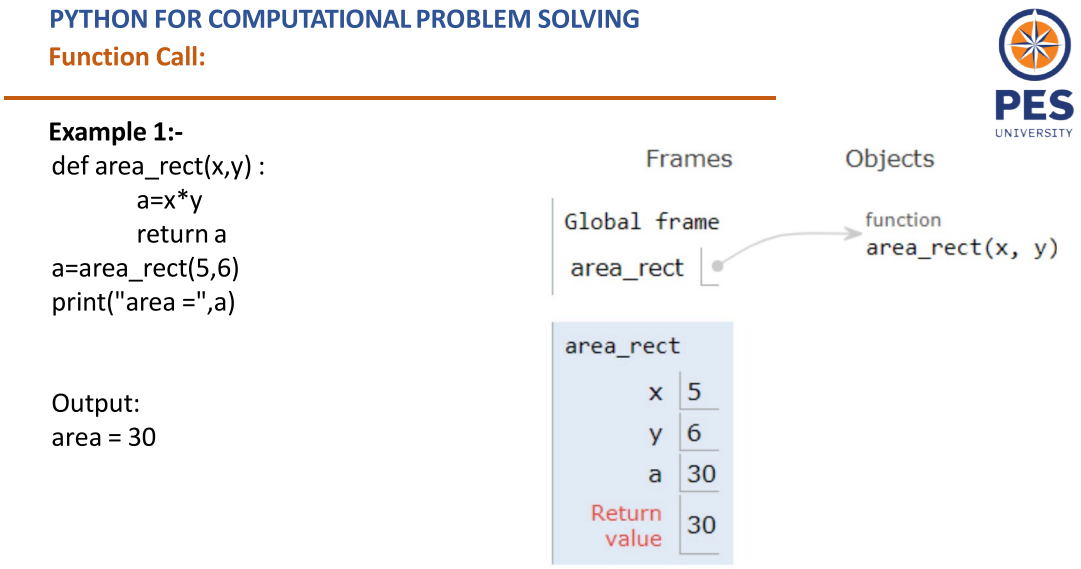


Figure G000101. UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call · source slide page 12.

EV0000379

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

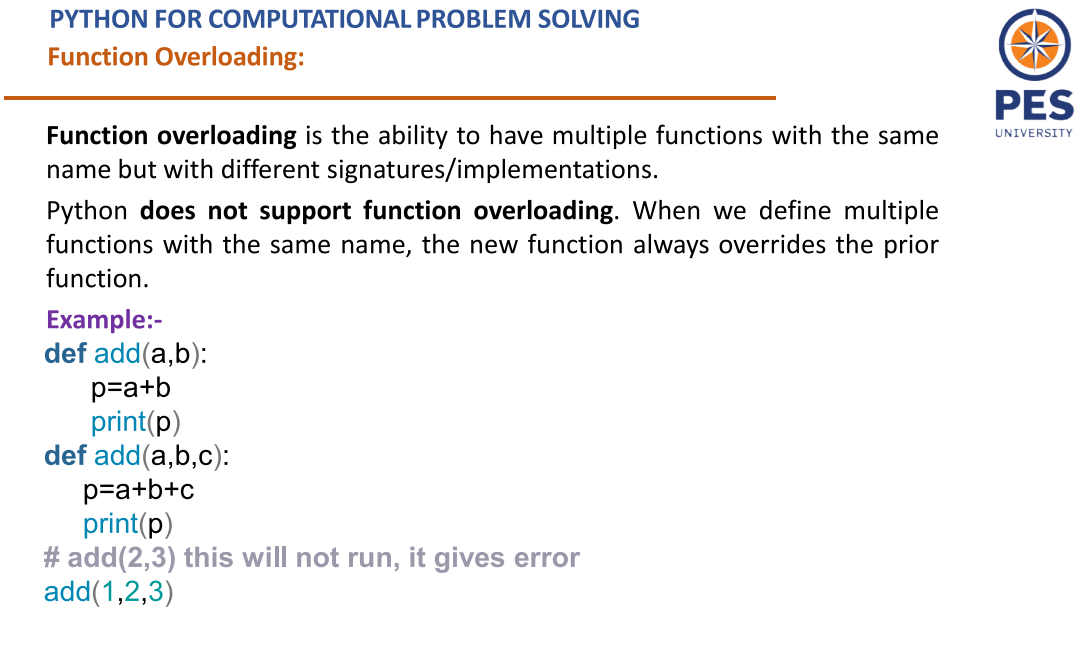


Figure G000102. UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call · source slide page 13.

EV0000382

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

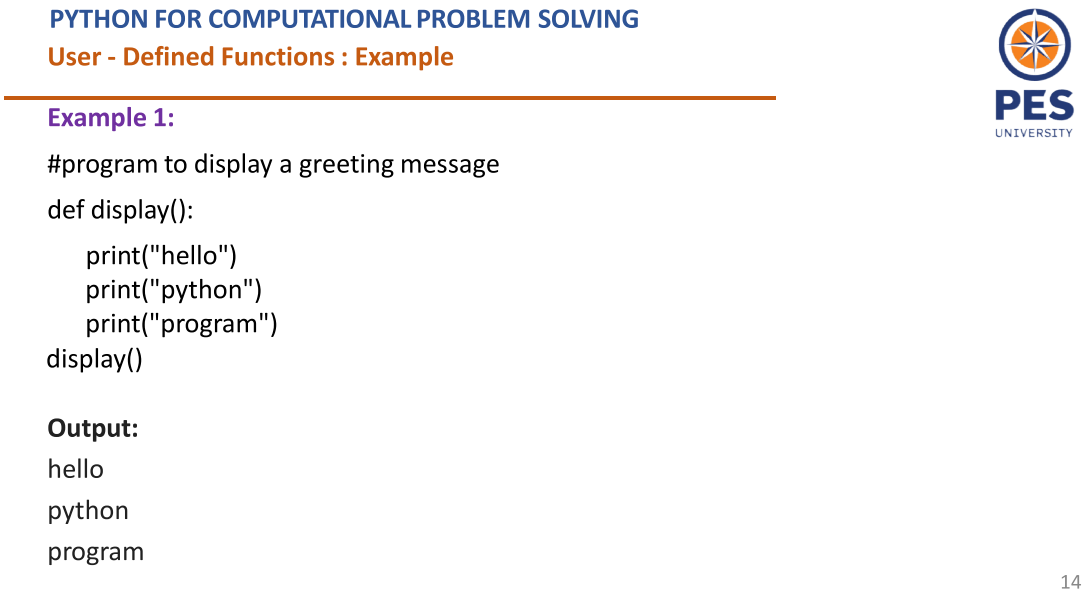


Figure G000103. UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call · source slide page 14.

EV0000385

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

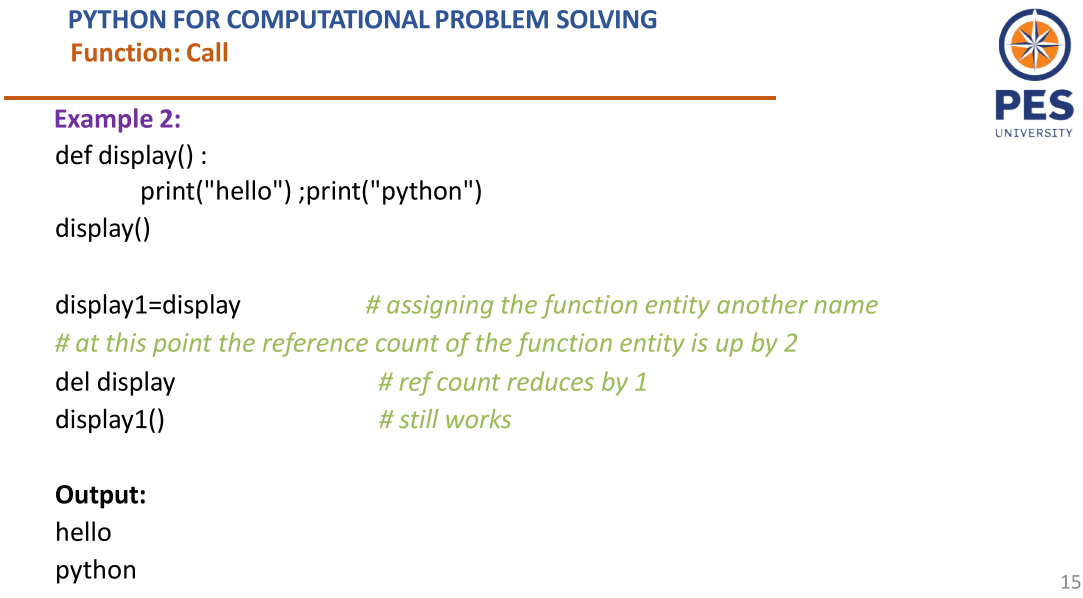


Figure G000104. UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call · source slide page 15.

EV0000388

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

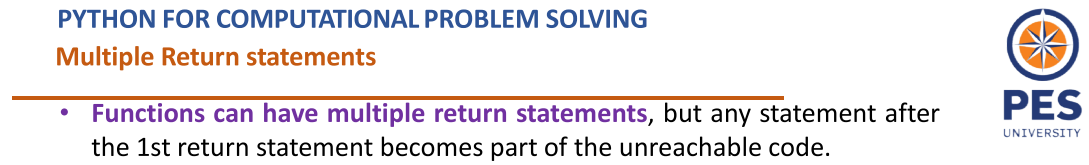


Figure G000105. UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call · source slide page 16.

EV0000391

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

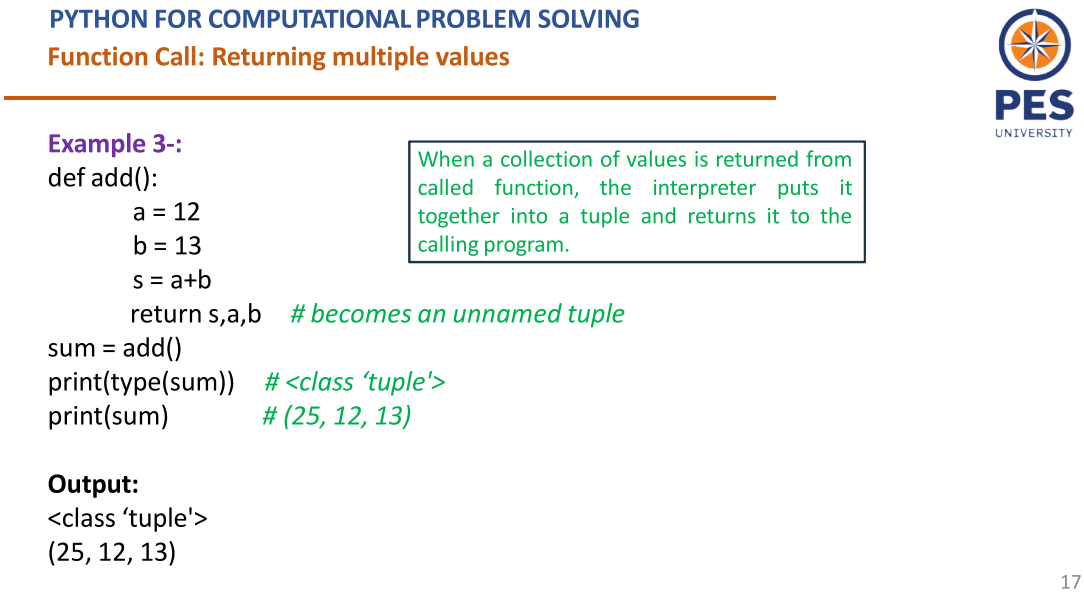


Figure G000106. UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call · source slide page 17.

EV0000394

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

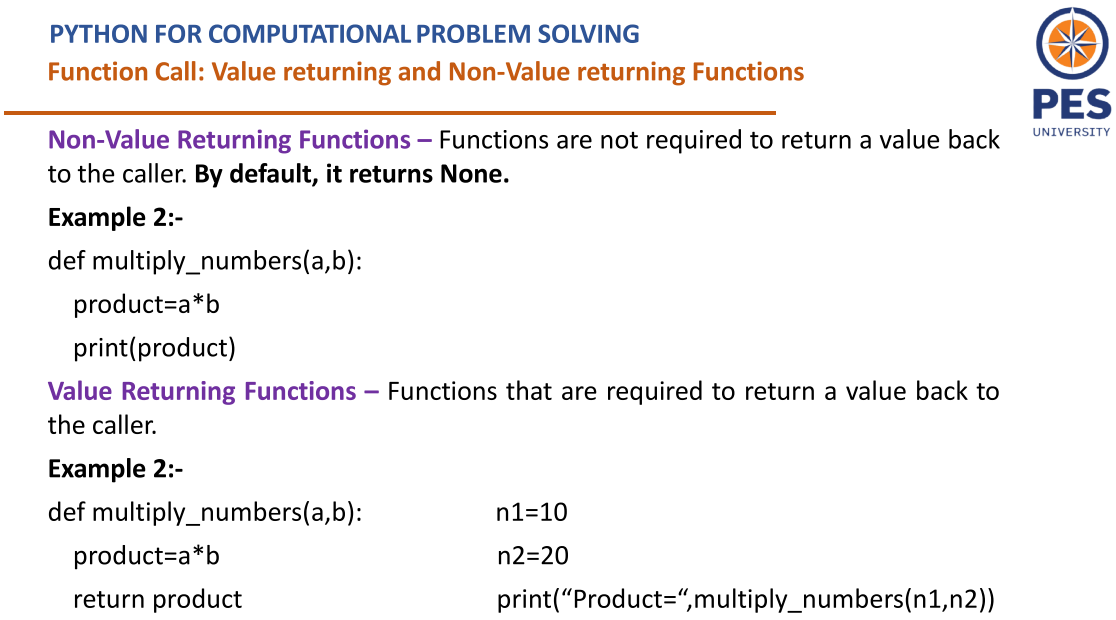


Figure G000107. UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call · source slide page 18.

EV0000397

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

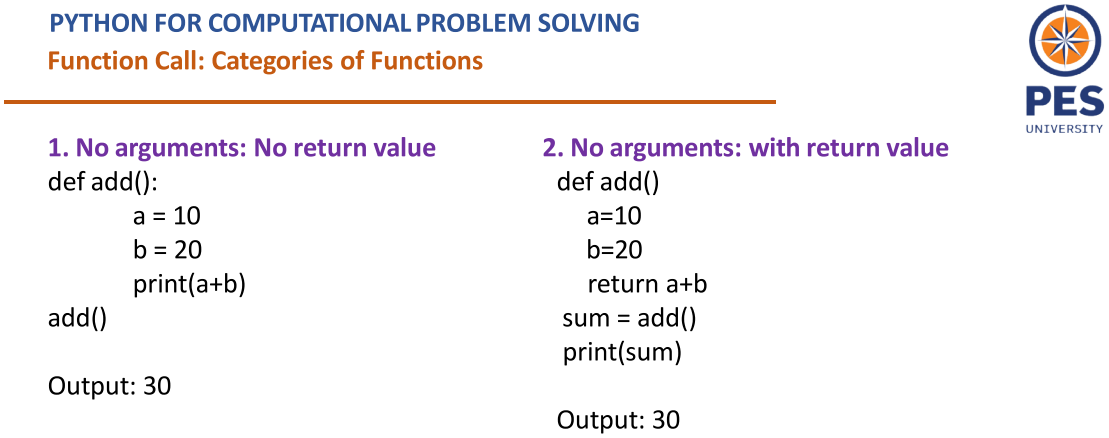


Figure G000108. UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call · source slide page 19.

EV0000400

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

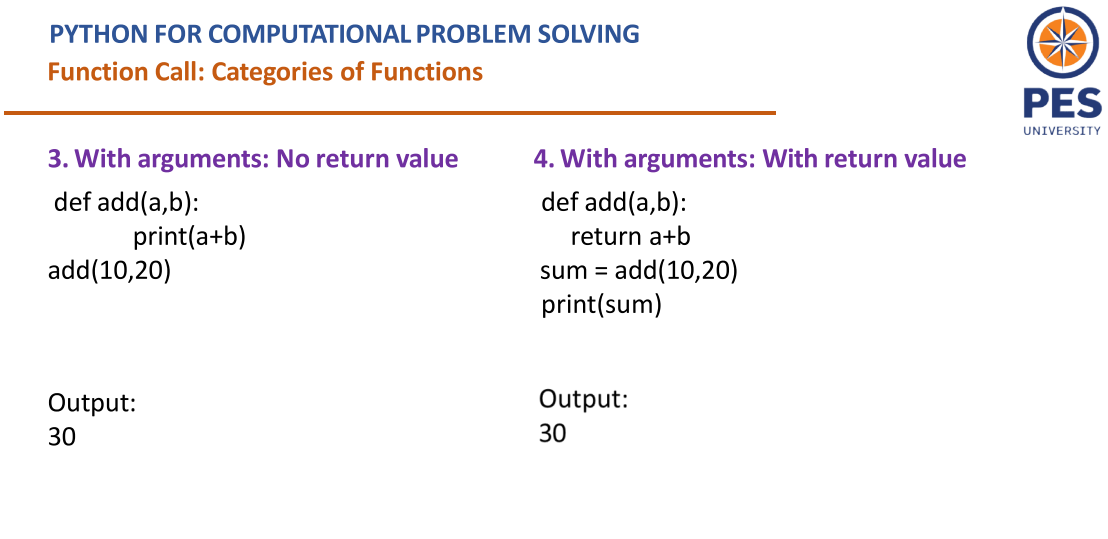


Figure G000109. UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call · source slide page 20.

EV0000403

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

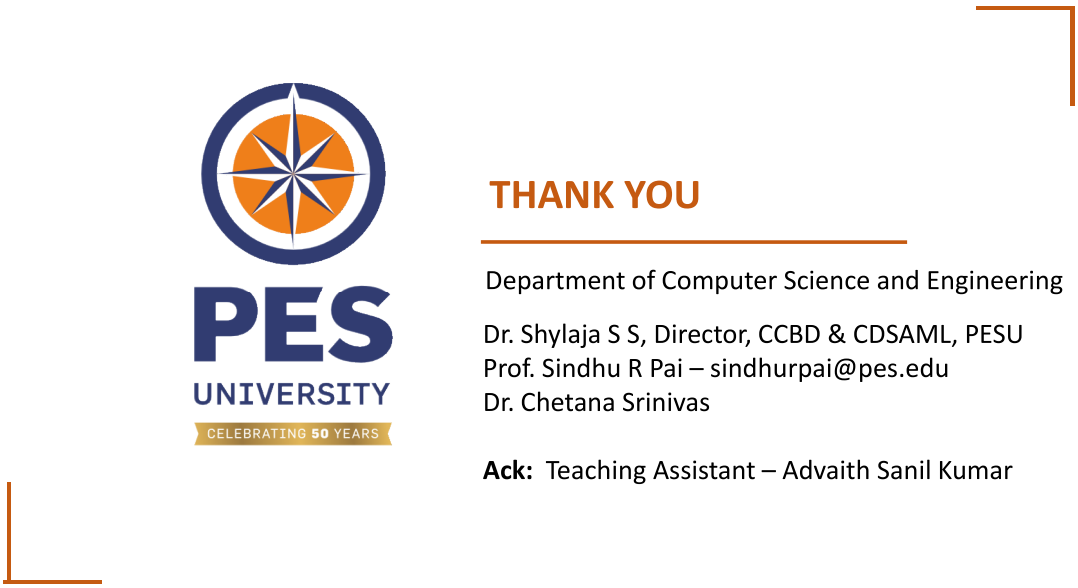


Figure G000110. UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call · source slide page 21.

EV0000406

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Functions: Definition and Call Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000345 · EV0000346 · EV0000347
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Course Overview What are functions? • A function is a self contained block of code that performsperforms aa specificspecific tasktask • Functions ideally take input, performs a set of operations and returns an output to the program that invoked it. 1 EV0000348 · EV0000349 · EV0000350
3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Advantages of functions • Reducing duplication of code and the complexity of the program through modularity. • Improves the readability, enhances the clarity of the program. • Promotes reuse of code. • Debugging and maintenance becomes easier EV0000351 · EV0000352 · EV0000353

4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Functions - Types Types of functions in Python programming: • Built-in functions or Pre-defined – Standard library functions. •• UserUser defineddefined functionsfunctions - DefinedDefined byby useruser forfor specificspecific applicationapplication inin orderorder toto reduce complexity of large programs. EV0000354 · EV0000355 · EV0000356
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Built-in Functions Built-in functions or Pre-defined – These functions are built into the Python interpreter and can be used/called without the need for additional code. Examples: • print() – Outputs a message to the console or standard output device. • Input() - Takes user input from the console or standard input device. • len() - Returns the length of an object, such as a string, list, or tuple. • type() - Returns the data type of an object. • sorted() - Sorts a list or sequence in ascending order. • abs() - Returns the absolute value of a number. EV0000357 · EV0000358 · EV0000359
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING User Defined Functions • User defined functions – These are the functions defined by the user to aid them in achieving a specific goal. The Python interpreter also supports the user-defined functions created as per the user requirement. The main purpose of these functions is to help in organizing the programsp g into logicalg segmentsg that work togetherg to solve a specificp partp of our problem. Definition: • A function is a block of code which only runs when it is called. • Can pass data, known as parameters (optional), into a function. • A function can return data as a result. EV0000360 · EV0000361 · EV0000362
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING User Defined Functions A function has two parts – leader and suite • The leader starts with the keyword def then the function name • function name is an identifier that starts with alphabets ⌊a- z or A-Z⌋ or _ and then followed by any number of letter of English or _ or digit. • followed by a pair of round parentheses then a colon. • suite follows – suite can have any valid statement of Python including another function definition. • All the statements within the function must be indented. • Finally, use the return (optional) keyword to return the output of the function. • Function must be defined preceding their usage in the program code. Syntax: def function_name(parameters): #suite EV0000363 · EV0000364 · EV0000365
8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Processing of Functions • When the function is defined, header or the leader is processed; the user is provided a name or a handle which is the same as the function name. • A function entity with the function name in the definition along with the suite is stored. • Each entity in Python has a reference count. • At this point of translation, only leader is processed and the suite is not processed. EV0000366 · EV0000367 · EV0000368
9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Function: Call • Name of the function followed by parentheses causes a function call • This results in transfer of control to the leader of the function and then the suite is executed • After that, the control comes back to the point after the function call in the user's code. • A pair of round parentheses () is called a function call operator. EV0000369 · EV0000370 · EV0000371
10. Source slide page 10 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Function: Call EV0000372 · EV0000373 · EV0000374

11. Source slide page 11 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function Call: Activation record When the function call is made, an activation record is created which will have • Parameters:- nothing but the arguments. • Local variables:- variables created with in the suite of the function. • Return address:- location to which the control of the program should be transferredtransferred onceonce thethe functionfunction terminates.terminates. • Temporary variables:- unnamed variables required by the translator • Return value:- value to be passed back to the caller. The activation record is created for every call of the function. At the end of the function, if no other callable can refer to the activation record, it will be removed. EV0000375 · EV0000376 · EV0000377

12. Source slide page 12 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function Call: Example 1:- def area_rect(x,y) : a=x*y return a a=area_rect(5,6) print("areai t(" =",a)") Output: area = 30 EV0000378 · EV0000379 · EV0000380

13. Source slide page 13 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function Overloading: Function overloading is the ability to have multiple functions with the same name but with different signatures/implementations. Python does not support function overloading. When we define multiple functions with the same name, the new function always overrides the prior function. Example:- def add(a,b): p=a+b print(p) def add(a,b,c): p=a+b+c print(p) # add(2,3) this will not run, it gives error add(1,2,3) EV0000381 · EV0000382 · EV0000383

14. Source slide page 14 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING User - Defined Functions : Example Example 1: #program to display a greeting message def display(): print("hello") print("python")print(python) print("program") display() Output: hello python program 14 EV0000384 · EV0000385 · EV0000386

15. Source slide page 15 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Function: Call Example 2: def display() : print("hello") ;print("python") display() display1=displayl l # assigning theh ffunction entity anotherh name # at this point the reference count of the function entity is up by 2 del display # ref count reduces by 1 display1() # still works Output: hello python 15 EV0000387 · EV0000388 · EV0000389

16. Source slide page 16 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Multiple Return statements • Functions can have multiple return statements, but any statement after the 1st return statement becomes part of the unreachable code. • The python interpreter is forgiving in terms of letting its programmers make these kind of errors. •• Example:Example:- • def example(): print(“an example function”) return #function ends here print(“after return”) #unreachable return ‘Hello’ #unreachable EV0000390 · EV0000391 · EV0000392

17. Source slide page 17 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function Call: Returning multiple values Example 3-: When a collection of values is returned from def add(): called function, the interpreter puts it a = 12 together into a tuple and returns it to the b = 13 calling program. s = a+b return s,a,b # becomes an unnamed tuple sum = add() print(type(sum)) # <class ‘tuple'> print(sum) # (25, 12, 13) Output: <class ‘tuple'> (25, 12, 13) 17 EV0000393 · EV0000394 · EV0000395

18. Source slide page 18 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function Call: Value returning and Non-Value returning Functions Non-Value Returning Functions – Functions are not required to return a value back to the caller. By default, it returns None. Example 2:- def multiply_numbers(a,b): product=a*bproduct a b print(product) Value Returning Functions – Functions that are required to return a value back to the caller. Example 2:- def multiply_numbers(a,b): n1=10 product=a*b n2=20 return product print("Product=",multiply_numbers(n1,n2)) EV0000396 · EV0000397 · EV0000398
19. Source slide page 19 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function Call: Categories of Functions 1. No arguments: No return value 2. No arguments: with return value def add(): def add() a = 10 a=10 b = 20 b=20 print(a+b) return a+b add() sum = add() print(sum) Output: 30 Output: 30 EV0000399 · EV0000400 · EV0000401
20. Source slide page 20 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function Call: Categories of Functions 3. With arguments: No return value 4. With arguments: With return value def add(a,b): def add(a,b): print(a+b) return a+b add(10,20) sum = add(10,20) print(sum)print(sum) Output: 30 EV0000402 · EV0000403 · EV0000404
21. Source slide page 21 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shylaja S S, Director, CCBD & CDSAML, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Dr. Chetana Srinivas Ack: Teaching Assistant – Advaith Sanil Kumar EV0000405 · EV0000406 · EV0000407

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:00.420] Hello everyone, welcome back to the lecture series on Python for computational problem solving. I am Nita Anjaikan from the Department of Computer Science and Engineering. [00:00:11.748] In this lecture, we will look at the concept of functions. We will look at how to define functions, what exactly functions are, and how we could call or use them. [00:00:25.956] So first let's explore at what a function really is. So as displayed in the slide, you can see it says that a function is a self-contained block of code that performs a specific task. It may seem like a bunch of [00:00:41.360] jargon to you, but let me put it in a simpler way. When you have a set of [00:00:47.684] code that say would be used multiple times in your program, rather than copying or repeating those lines of code multiple times, I would rather club all those reusable lines together, put a name for it. [00:01:05.328] So that every time I want to use that same functionality, I just call it by its name. [00:01:12.644] So what else does it mean that it performs a specific task? Functions ideally would take an input, performs a specific set of operations, and computes and results gives you an output. [00:01:28.132] We are actually familiar with this concept as we have seen in the first unit when we talked about your input and output functions. So what does function do? It takes an input, processes your input, does a couple of steps, and then returns an output. [00:01:46.916] There are different kinds of functions available in Python. You have built in an existing functions like the print and input function, and there are user defined functions which as a programmer we can create based on the needs of our application. [00:02:05.540] What are the advantages of using functions? They are, like I've already said, it reduces the duplication of code by just call the name of the function. It supports modularity, which means that every feature of the application that you build could be a function. [00:02:25.668] So if I want to modify just one feature, I can just modify that specific function. It improves the clarity of code because it's so neatly structured as you will see in our examples that you can easily debug the

code. [00:02:43.492] It promotes reuse, it enhances readability, and the debugging and maintenance also becomes easier because when you know that there is a problem with a specific feature, you can directly jump to the lines of code where the function is defined and debug the code appropriately. [00:03:03.940] So how exactly do we create a function? So in our programming term when we say function definition, it means that we're going to try and create the function. So a function has two parts, like you have seen in your control structures. [00:03:19.204] there would be a leader and a suite because we are going to have multiple lines of code that are going to be clubbed and given a single name. So the leader starts with the keyword def, D E F. [00:03:33.700] Followed by the name of the function. A function name is an identifier, which means it should start with alphabets, then followed by a number or letter in English, or a digit, followed by a pair of round parentheses and then a colon. [00:03:51.428] So why would we need these round parentheses? It is to give an input to the function if required. We will look at all of that in the upcoming lectures. Once we have [00:04:04.708] and the leader with a colon, you can then use an indented block called the suite, which can have any valid statement in Python. It could be one or more statements, or it could also be another function, as we will see. [00:04:22.244] So let's look at the syntax. As you can see, there is define the name of a function followed by a pair of parentheses and the parameters are optional. So if you don't want to give an input, then you don't have to list the parameters. [00:04:39.332] Then you have an indented block which would be the suite of the program. We must note that a function must be defined before we use or call the function. [00:04:53.572] So let's just look at an example. So I would like to now create show you a hands-on example of how exactly we do create your function. Okay so I'm going to create a simple display function almost like your print, right?

EV0000066 · EV0000067 · EV0000068 · EV0000069

Transcript block 2 (source interval 000300-000599 seconds): [00:05:12.708] So I'm going to say define display. I know this is a redundant example, but it's just to show you how exactly it works. Hello? I'm going to just say hello from a function. [00:05:30.596] Okay. So as you can see, you start with the keyword DEF followed by a valid identifier which acts as the name of the function and a pair of round parentheses. Now this is a function that's not going to take any parameter, so I have left it blank. [00:05:49.764] Every leader must end with a semicolon. Now the suite could be any valid Python statement. So I've just given a simple print. Now this is a valid function. [00:06:03.684] Right? So if I were to run the program, am I going to get any result? So I'm heading over to the [00:06:12.964] Command prompt and I'm going to execute functions.py, which is the file in which I have written the code. You can see there is no output because all I have done is create the function. I have not brought the function to life. So your function just [00:06:29.440] has a piece of code and a name associated with it. So if I want to use this function anywhere in the program, I need to use the name of the function [00:06:39.876] Followed by what we would call a function call operator. So a pair of round bracket is called a function call operator, and this is where we would actually invoke the function. Okay. [00:06:57.348] Invoke or we would activate the function. [00:07:04.036] Okay, so I'm going to save the code and head back to the interpreter, I mean uh the command prompt and execute it. So now you can see that it does print hello from a function. So now that we've understood how a function definition works, let's [00:07:20.656] start to look at what exactly happens when I create a function and when I would call the function. So heading back to the slides. [00:07:35.396] So when a function is defined, as you saw that when I executed, I did not get any output because the header or the leader is processed. Okay, so the user is just provided a name or a handle, which is the same as a function name. Okay, and the function entity

will be created in the [00:07:55.012] global uh memory okay and a function entity along with the suite is stored. Okay and each entity in Python has a reference count as you have seen with many variables. So since only the leader is processed, we don't get any output. [00:08:10.660] So how do we get the output? By using the function call. So we use the name of the function, followed by a set of parentheses, and you get the function call. [00:08:20.484] So this results in the transfer of control to the leader, and then it goes into the suite and starts to execute it. So once the execution is over, it comes back to the point where the function call. [00:08:34.756] was made. Right? So let's look at it in detail. So say we have a set of statements and then we encounter a function call. At that point of time the control is shifted to the entire function block. So if there are any arguments is given as an input, [00:08:53.252] The function is processed. So in our case, it was a display function that just printed hello from the function. And there was no output. So what it does is it goes back to the same place where the function call was made and the program executes as [00:09:12.452] expected or as follows. It goes on to the next set of statements. Right? So let's look at another example where I'm going to like we've shown in this particular PPT, I'm going to create the same uh [00:09:27.204] display function and have multiple print statements to show you what exactly would happen if I have other statements and what exactly the name of the function actually means. So let's move on back to our hands on session. So [00:09:46.660] Before display, I'm going to say print before function call. [00:09:56.036] Okay? And after display print after function call. So this example is just to prove that

EV0000070 · EV0000071 · EV0000072 · EV0000073

Transcript block 3 (source interval 000600-000899 seconds): [00:10:06.884] The first two lines are not executed in the beginning. It's always going to be before and in between the before and after function call. So I've saved it, heading to the command prompt, and you can see hello from a function comes in between the before and after function call. [00:10:24.932] Now what exactly does the name of the function mean? So in my print statement, I'm just going to give the name of the function without the function call parameter. Right? So let's look at the result. [00:10:41.988] You can see it says it is a function display at a particular location. This is because it's stored in the global memory, right? Which means your function is actually. [00:10:54.436] just another object which has a property to be reused. Right? So I can so write display 1 is equal to display. [00:11:08.548] What does this statement mean? Since the function is an object, I can assign one function object to another. So this is basically creating another alias for the function display. Okay [00:11:26.052] So let me just experiment and see what happens. I'm going to write display1 and the function call parameter. So what would our output be? Heading over to the interpreter. You can see it says hello from a function, right? So let's also display [00:11:47.300] the uh value of display one. [00:11:57.636] So if I were to let me clear the screen. [00:12:02.596] So when you look at print display and print display one, you can see that both of them [00:12:09.412] are functions at the same location, which means the name of a function is a function object or a variable which is basically a whose value is a function and that can be activated using a pair of round parentheses. [00:12:26.784] Right? So right now display one and display are both names for the same function which is display and when invoked it will just execute the suite of the function. Now what would happen if I were to delete display the original value? So I'm going to write del [00:12:46.212] display. Does this delete my function altogether? So if I were to now just call display in the next line, okay I'm going to save it. Head over to the terminal [00:12:58.820] and execute it, you can say I get an error saying the name display is not defined. That's because I have deleted the display

function here. So heading over to the interpreter. After this I'm going to write display1 with the function call. [00:13:16.452] Will this work? Let's look at it. It works as expected. So what you can [00:13:24.068] Infer from this particular example is that display and display one are just names for that function object in the memory. I can call it using either one of them. So when I delete display, it only deletes the name. [00:13:43.748] Associated with the function. It doesn't delete the function itself, which means I have display one also pointing to the same function. Right? So only when the reference count of the function becomes zero will the [00:14:01.860] function become inaccessible. So as long as I have aliases pointing to the function object, the function would still remain active and available in your [00:14:14.340] Memory. Okay. So with this video, I'm going to end the concept of how to define a function and call it. I hope you have understood the concept. In the upcoming videos, we will look at what exactly happens when a function is called, how we could give input to the function. [00:14:34.180] and process these values as well. So you can also head over to the assignment section in PESO Academy and try and solve some of the works, examples or worksheets that have been specified. Thank you everyone [00:14:54.404] So let's look at the next set of information in the upcoming class. Thank you.

EV0000074 · EV0000075 · EV0000076 · EV0000077

Full transcript: 2117 normalized words, 60 timestamped segments, 902.698667 seconds duration. Low-confidence segments flagged: 0.

EV0000066 · EV0000067 · EV0000068 · EV0000069

6 Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets

This entry preserves the complete slide deck titled UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets as searchable extracted page text and page-faithful rendered images. Its association with a particular video is not confirmed; it remains a separate topic under Python for computational Problem Solving, Unit_02, to avoid an unsupported match. EV0000174 · EV0000175

Graphs and diagrams

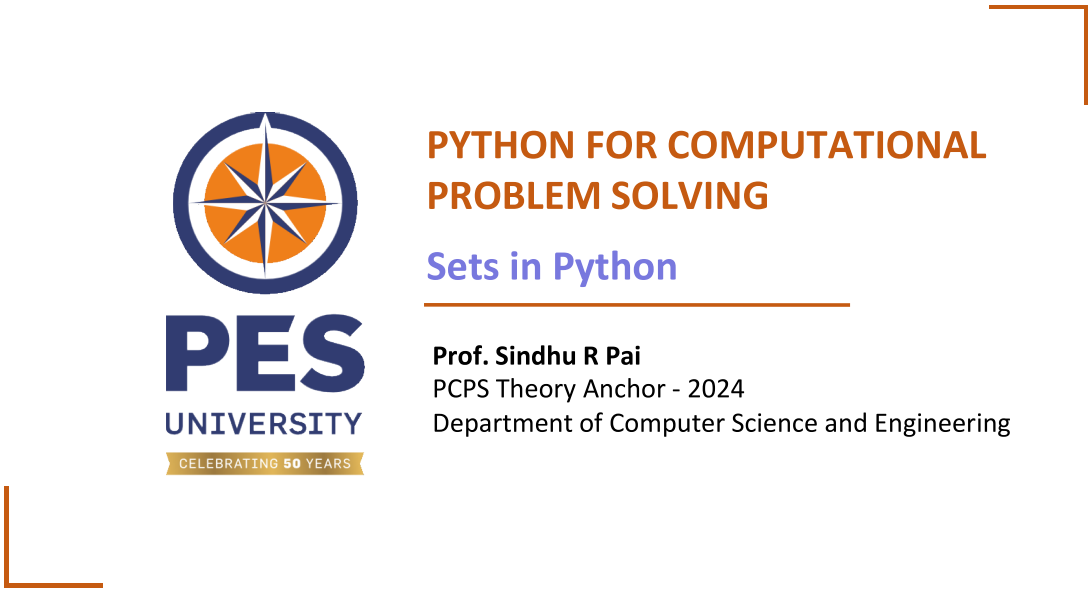


Figure G000033. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 1.

EV0000175

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Introduction

Definition:- A set is an unordered collection of Unique elements with zero or more elements present with the following attributes.

- In Python **sets are written with curly braces.**
- **Constructor set()** is used to create an empty set.

Examples :

- Empty_set=set() # Use the **constructor set()**.
- fruitset = { “apple”, “orange”, “kiwi”, “banana”, “cherry”}
- set1 = {(1, 'a', 'hi'), 2, 'hello', 5.56, 3+5j, True}

Figure G000034. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 2.

EV0000178

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

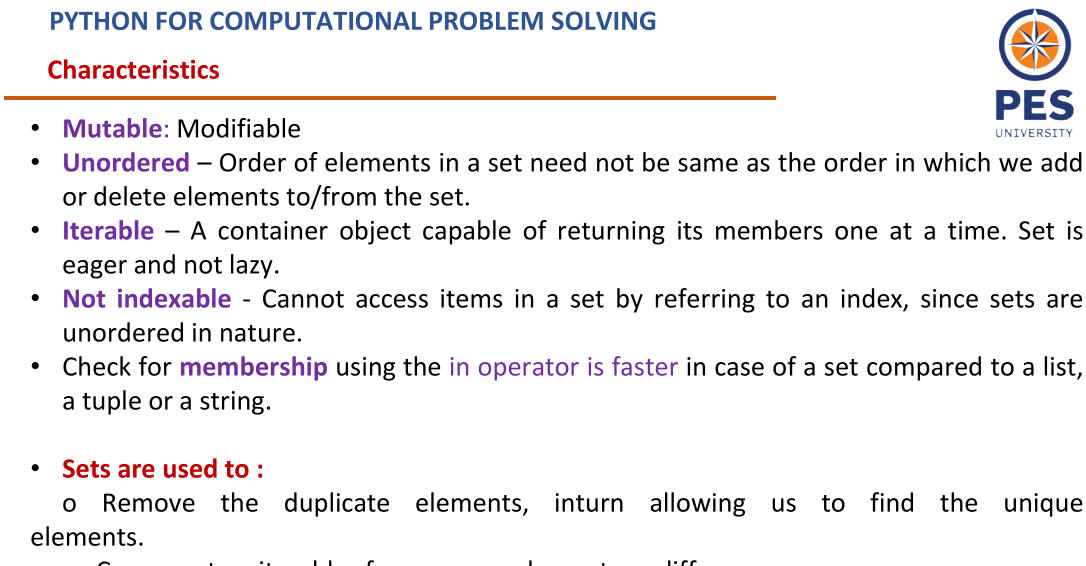


Figure G000035. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 3.

EV0000181

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Characteristics

- Elements are **unique** – does not support repeated elements.
- Elements should be **hashable.**
- Hashing is a mechanism to convert the given element into an integer.
- An object is hashable if it has a hash value which never changes during its lifetime (it needs a __hash__() method).

Examples of hashable objects:

- int, float, complex, bool, string, tuple, range, frozenset, bytes, decimal.

Examples of Unhashable objects:

- list, dict, set, bytearray

Figure G000036. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 4.

EV0000184

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

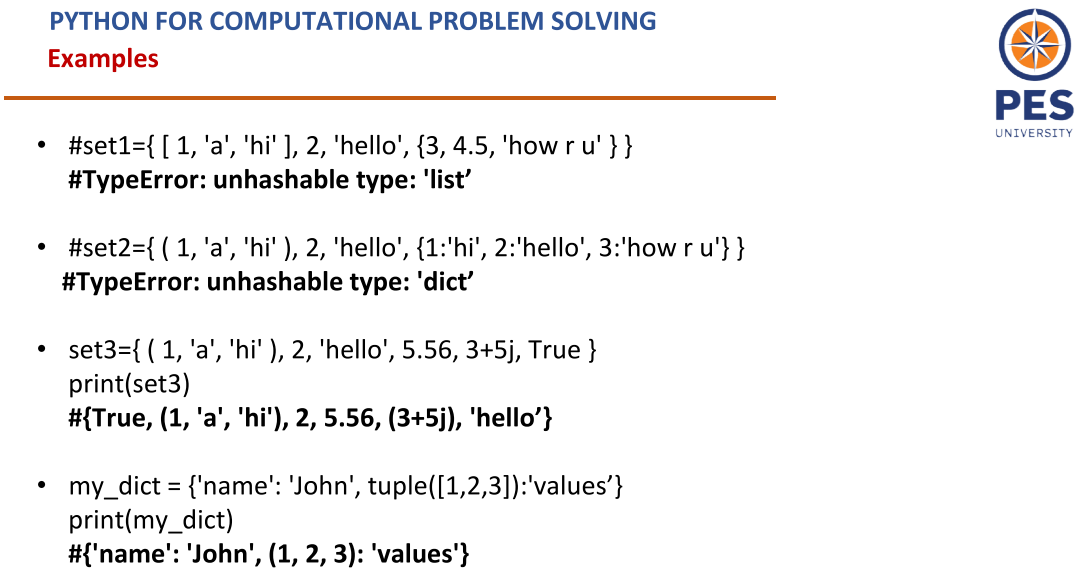


Figure G000037. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 5.

EV0000187

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

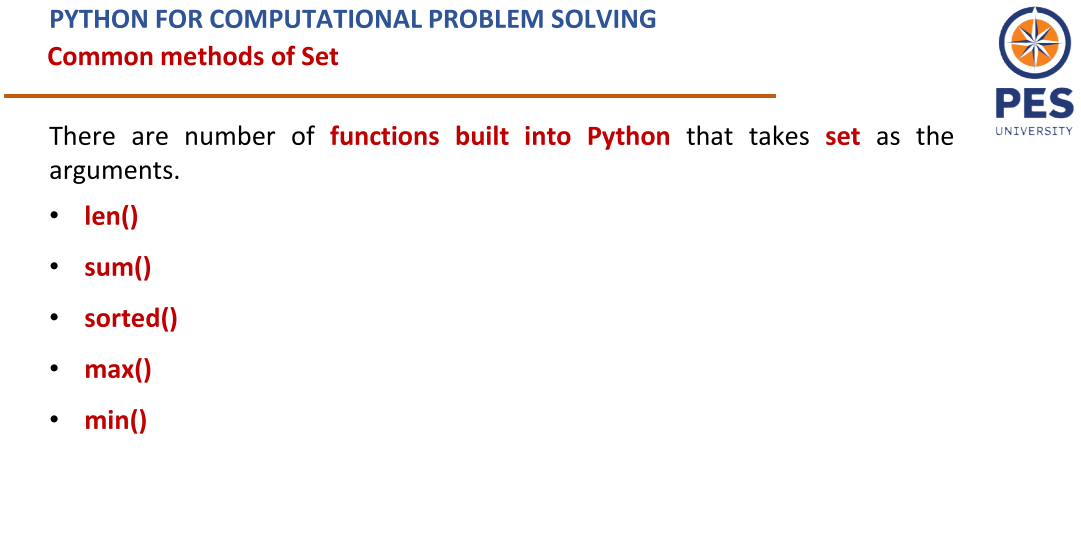


Figure G000038. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 6.

EV0000190

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

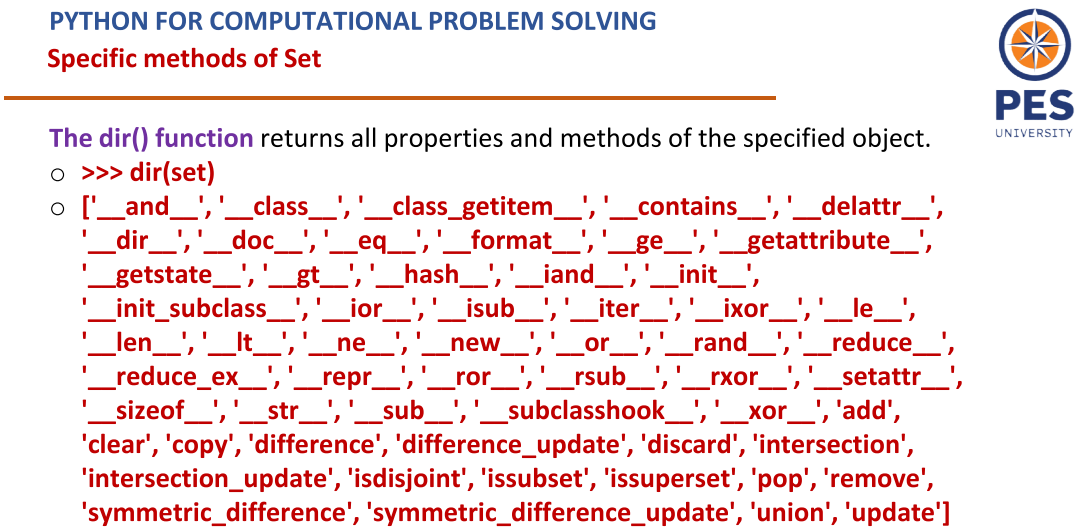
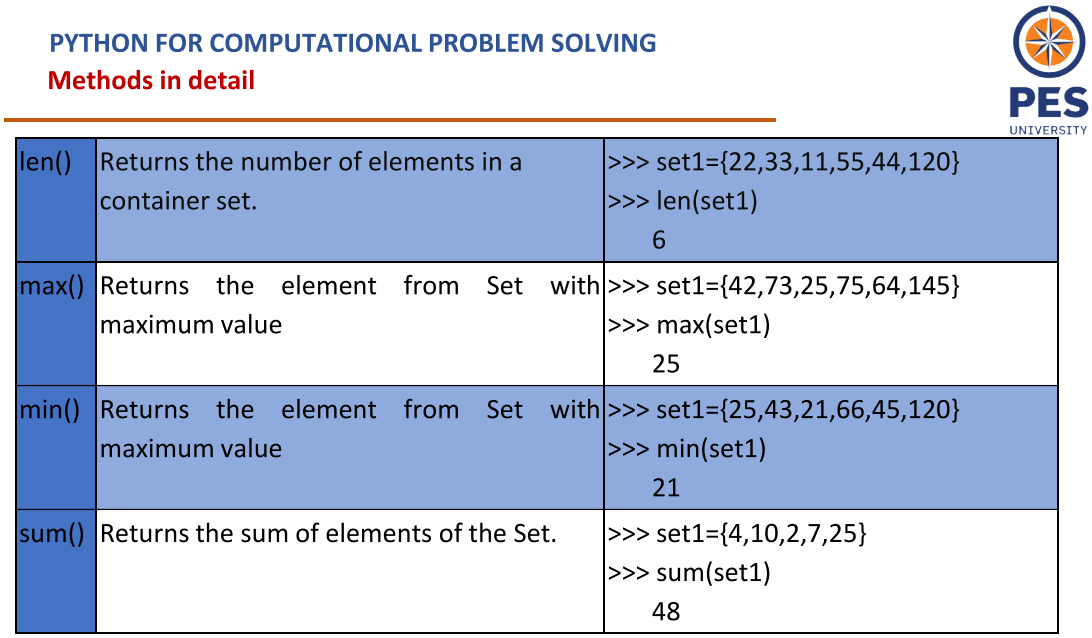


Figure G000039. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 7.

EV0000193

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.



len()	Returns the number of elements in a container set.	>>> set1={22,33,11,55,44,120}
		>>> len(set1)
		6

max()	Returns the element from Set with maximum value	>>> set1={42,73,25,75,64,145}
		>>> max(set1)
		25

min()	Returns the element from Set with maximum value	>>> set1={25,43,21,66,45,120}
		>>> min(set1)
		21

sum()	Returns the sum of elements of the Set.	>>> set1={4,10,2,7,25}
		>>> sum(set1)
		48

Figure G000040. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 8.

EV0000196

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

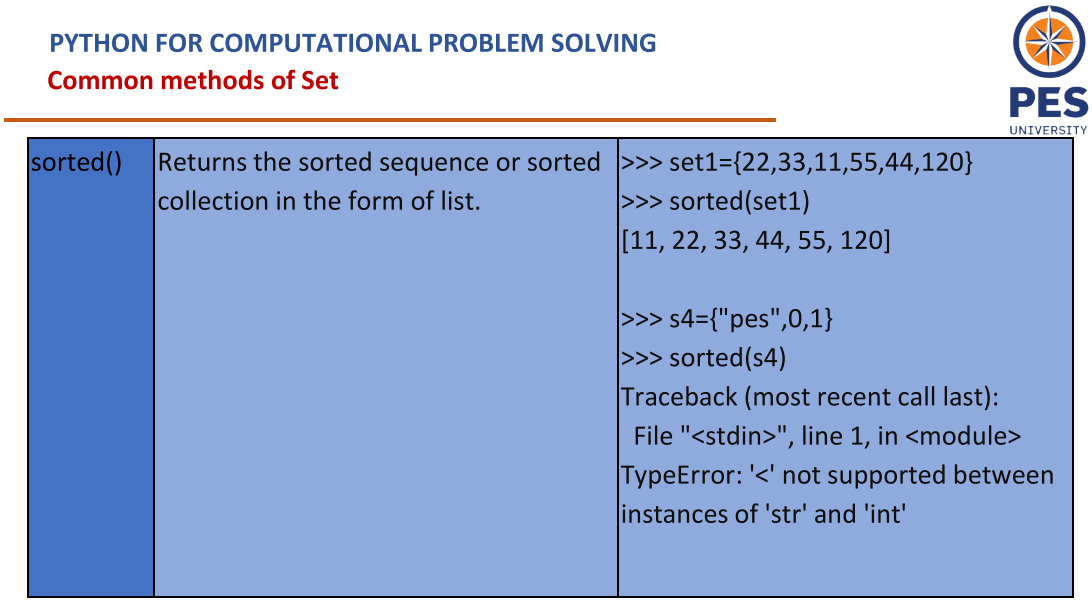


Figure G000041. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 9.

EV0000199

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

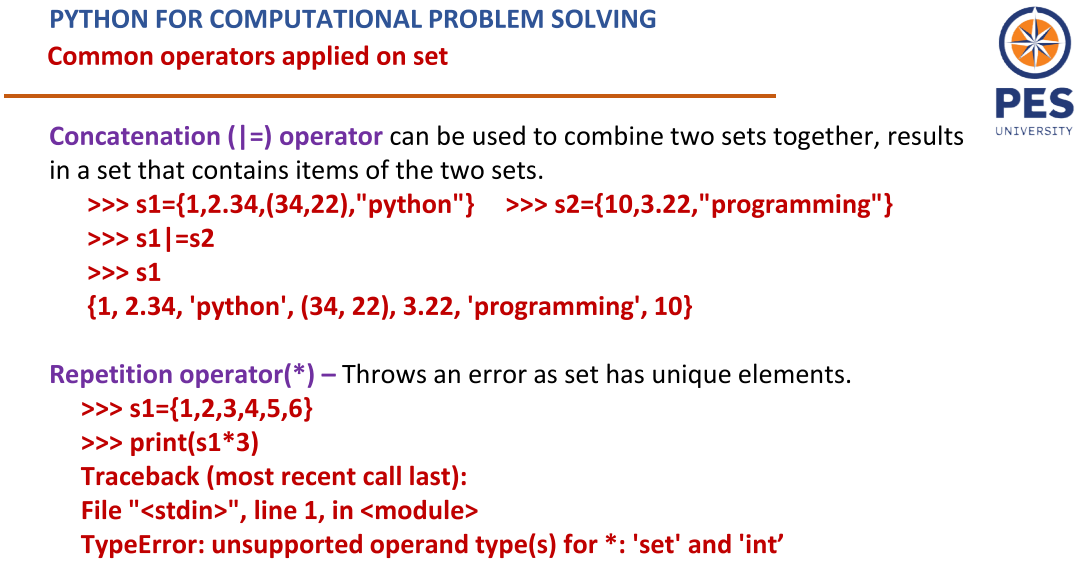


Figure G000042. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 10.

EV0000202

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

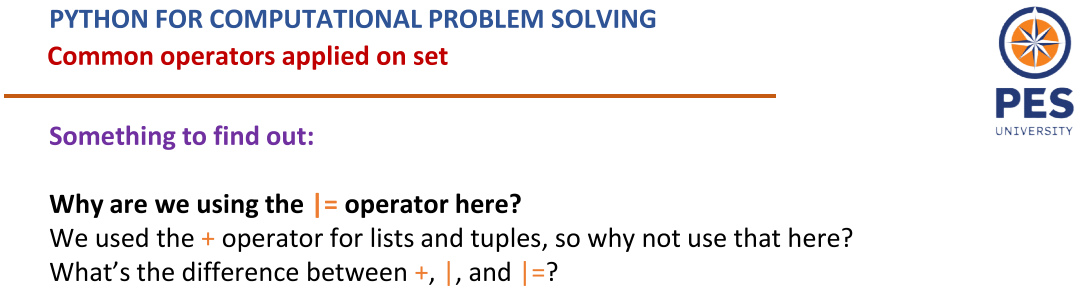


Figure G000043. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 11.

EV0000205

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

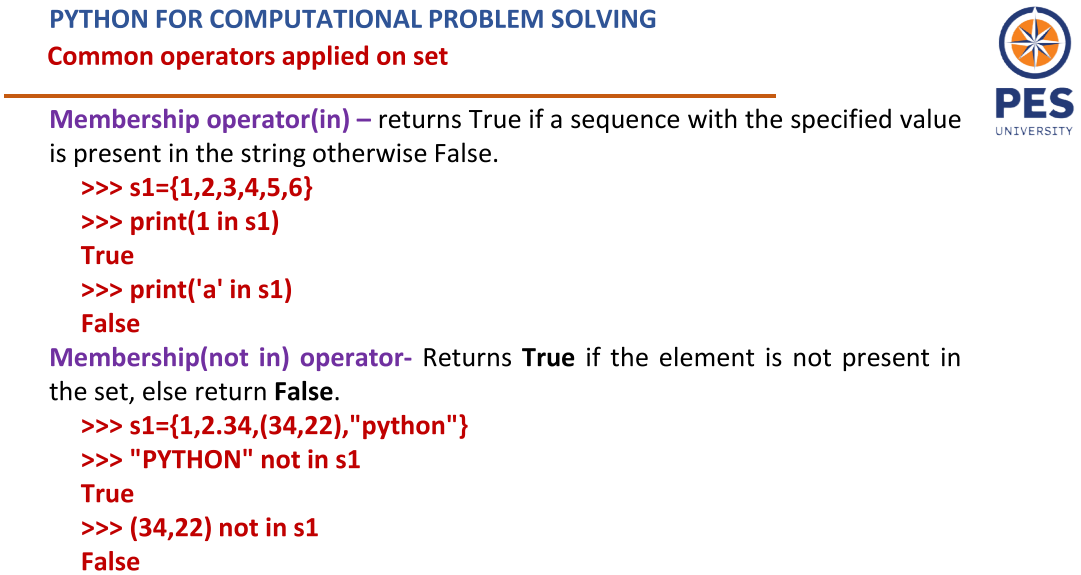


Figure G000044. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 12.

EV0000208

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

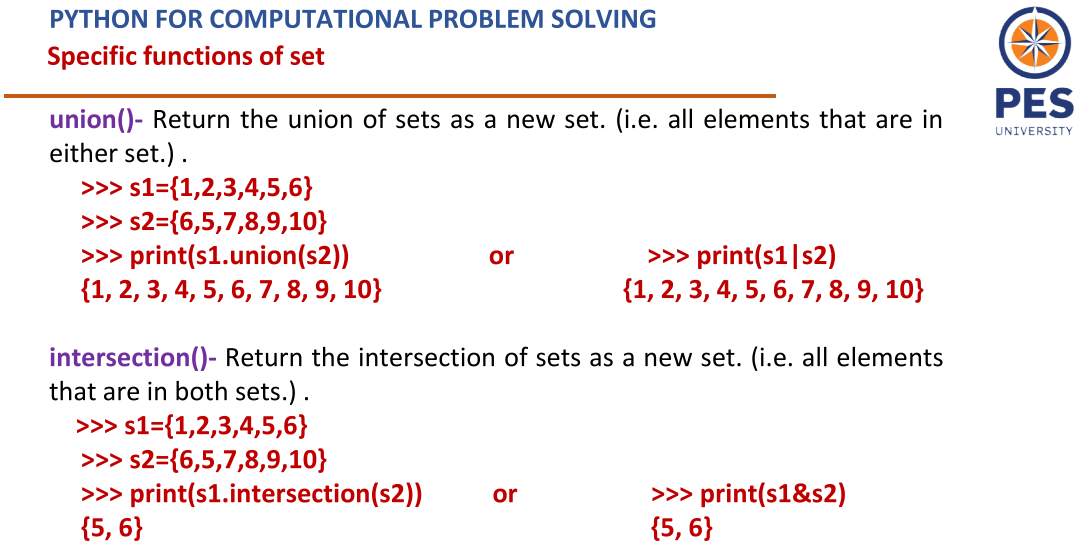


Figure G000045. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 13.

EV0000211

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

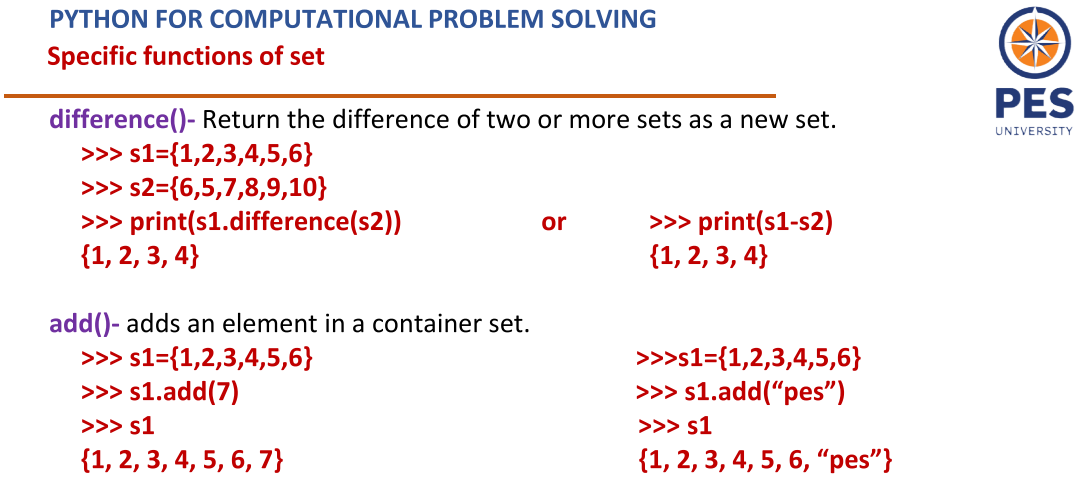


Figure G000046. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 14.

EV0000214

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

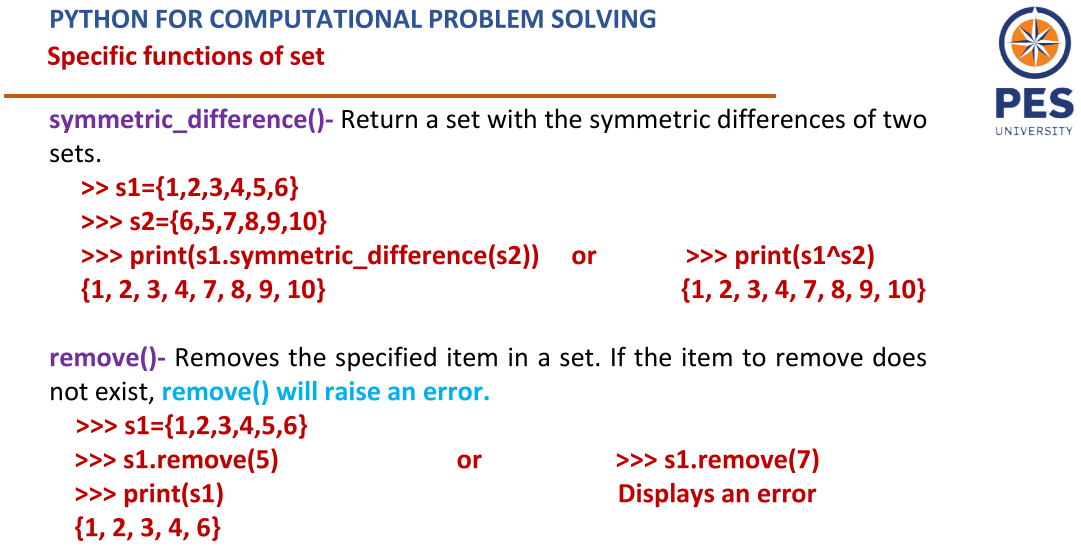


Figure G000047. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 15.

EV0000217

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

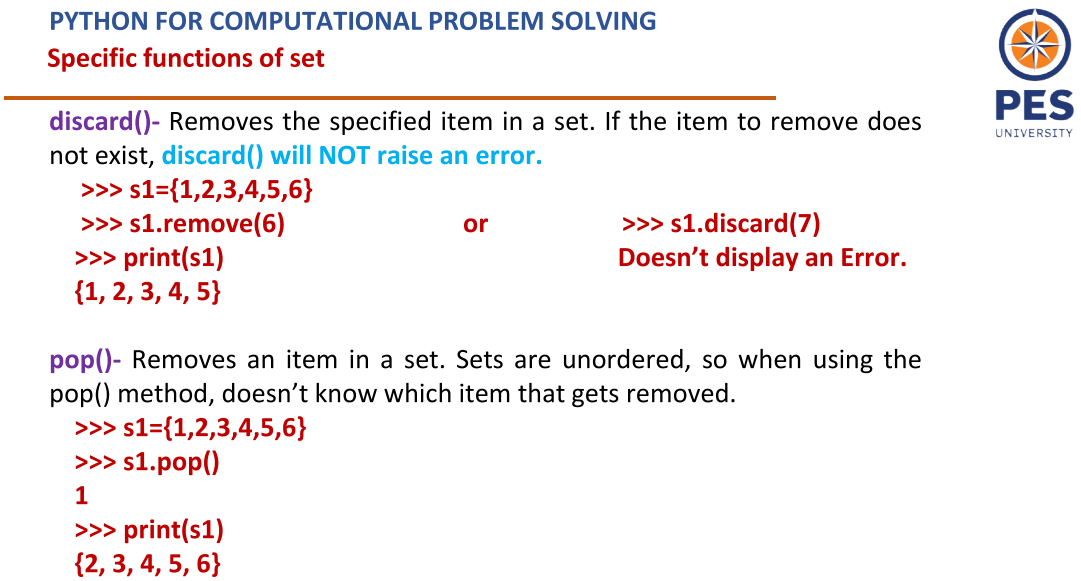


Figure G000048. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 16.

EV0000220

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Specific functions of set

PES UNIVERSITY

update()- updates the current set, by adding items from another set (or any other iterable). If the element is present in both the sets, only one appearance of element present in updated set

```
>>> s1={120,12,23}
>>> s1.update([22,15,67,"pes"])
>>> s1
{67, 12, 15, 22, 23, 120, 'pes'}
```

intersection_update()- Removes the items that is not present in both sets.

```
>>> set1={"apple", "banana", "cherry"}
>>> set2={"google", "microsoft", "apple"}
>>> set1.intersection_update(set2)
>>> set1
{'apple'}
```

Figure G000049. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 17.

EV0000223

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

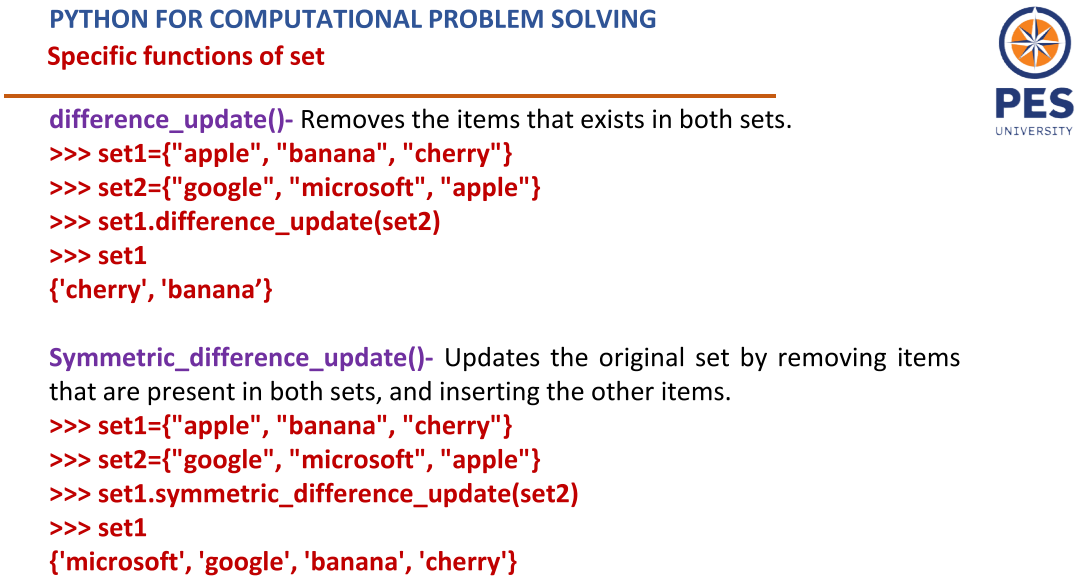


Figure G000050. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 18.

EV0000226

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

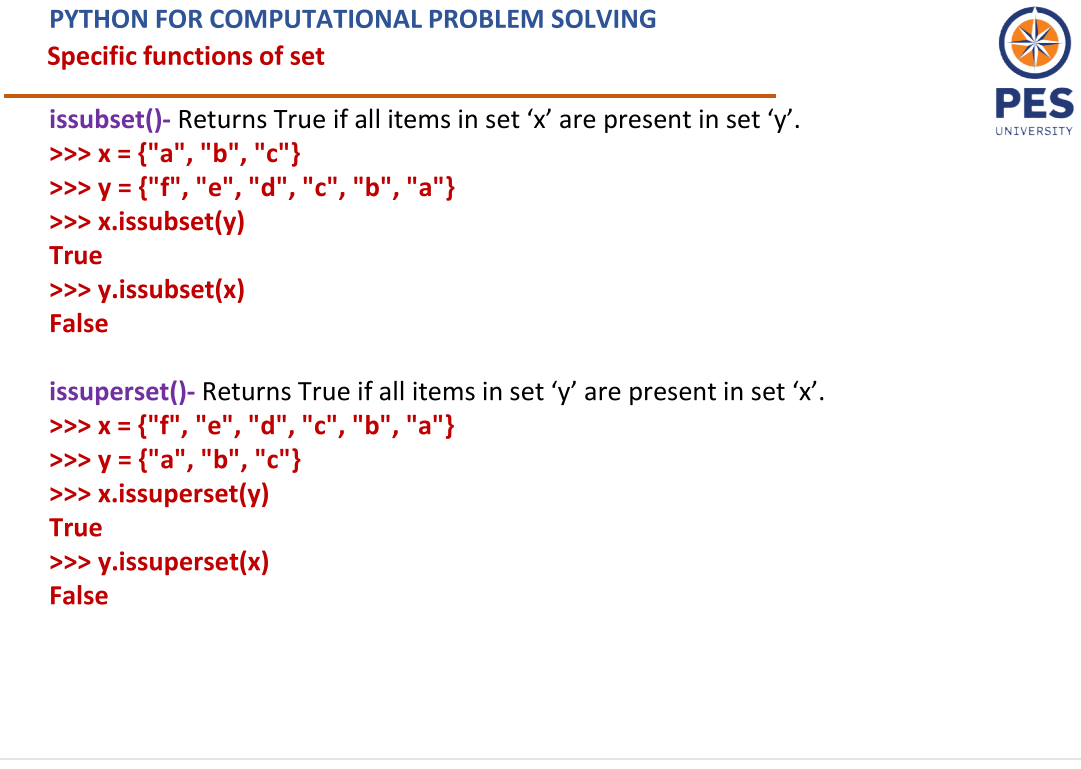


Figure G000051. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 19.

EV0000229

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

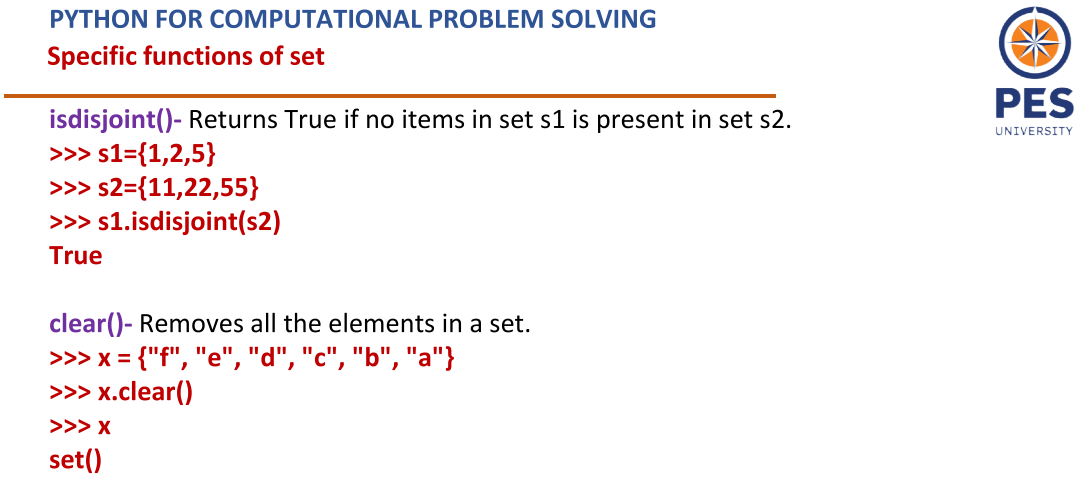


Figure G000052. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 20.

EV0000232

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

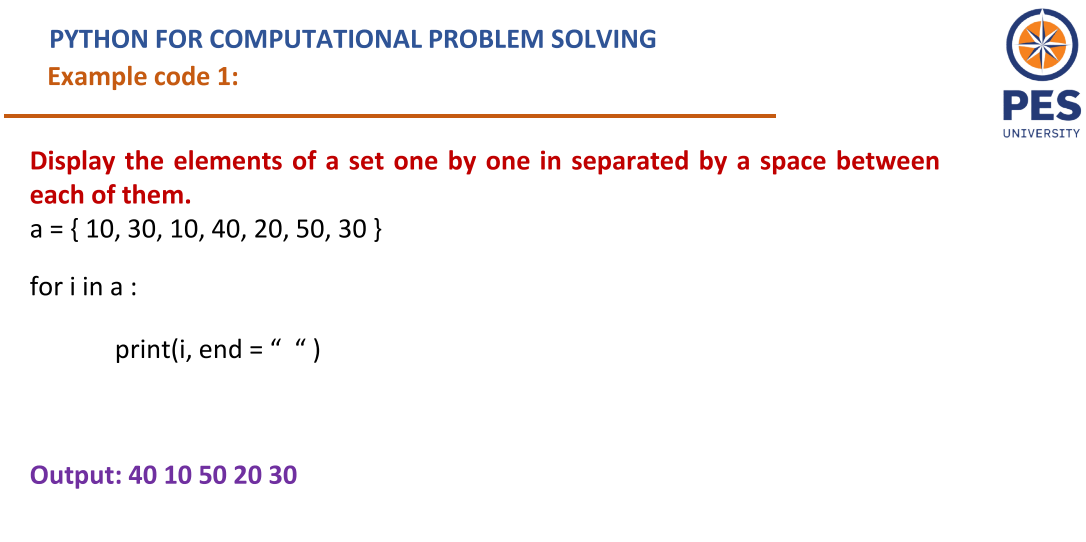


Figure G000053. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 21.

EV0000235

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Example code 2:

PES UNIVERSITY

Create a set of numbers from 2 to n.

Steps: create an empty set and add the elements to it.

```
s = set()
n=int(input("enter the value of n")) #10
for i in range(2,n+1):
    s.add(i)
print(s)
```

Output: {2, 3, 4, 5, 6, 7, 8, 9, 10}

Figure G000054. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 22.

EV0000238

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

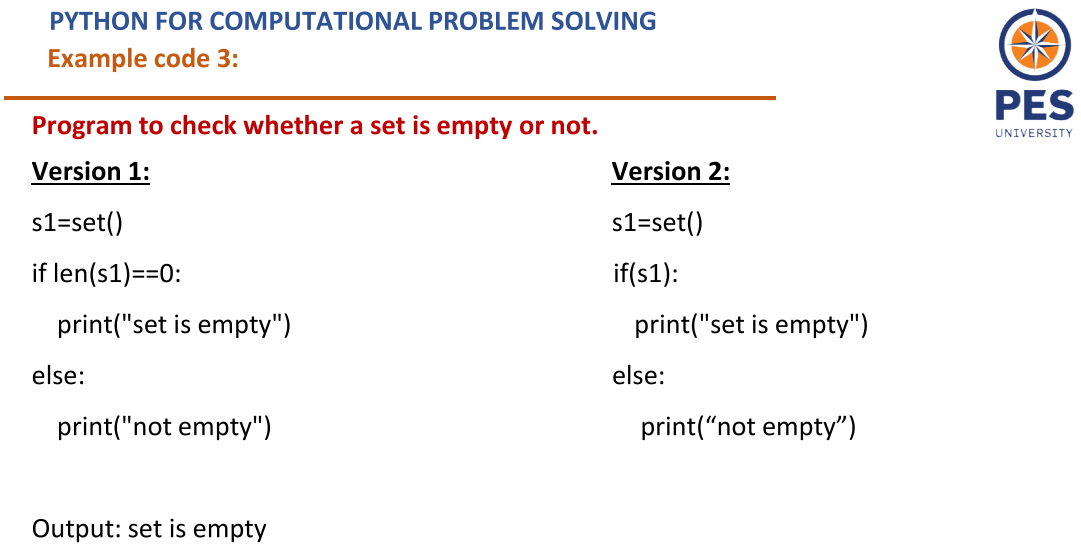


Figure G000055. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 23.

EV0000241

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

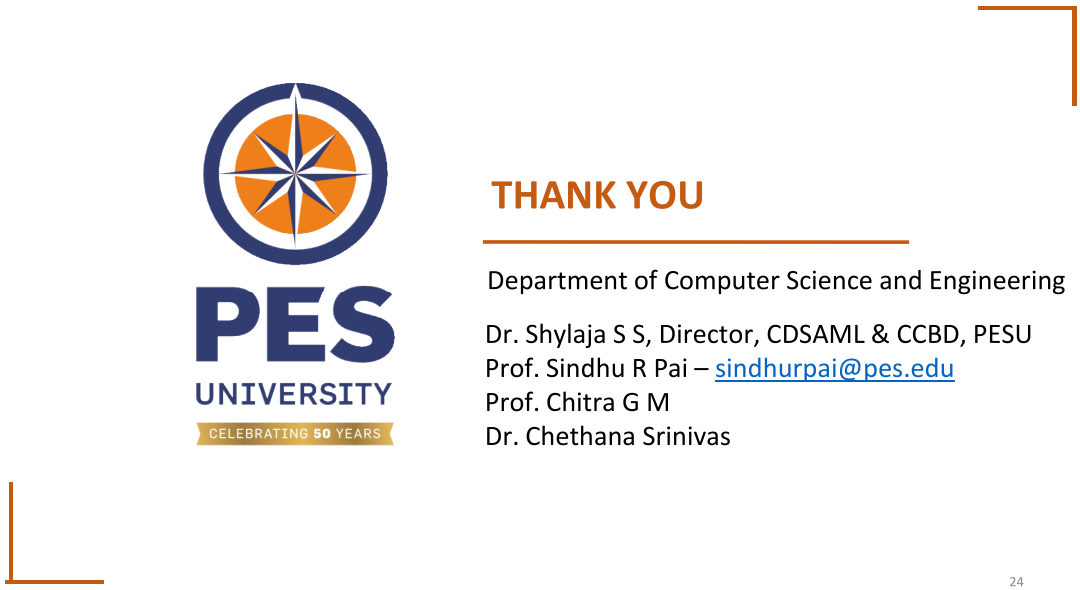


Figure G000056. UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets · source slide page 24.

EV0000244

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Sets in Python Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000174 · EV0000175 · EV0000176
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Introduction Definition:- A set is an unordered collection of Unique elements with zero or more elements present with the following attributes. • In Python sets are written with curly braces. • ConstructorConstructor set()set() isis usedused totoo createcreate anan emptyempty setset. Examples : • Empty_set=set() # Use the constructor set(). • fruitset = { “apple”, “orange”, “kiwi”, “banana”, “cherry”} • set1 = {(1, 'a', 'hi'), 2, 'hello', 5.56, 3+5j, True} EV0000177 · EV0000178 · EV0000179

3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Characteristics • Mutable: Modifiable • Unordered – Order of elements in a set need not be same as the order in which we add or delete elements to/from the set. • Iterable – A container object capable of returning its members one at a time. Set is eager and not lazy. •• NotNot indexableindexable - CannotCannot accessaccess itemsitems inin aa setset bby referringreferring toto anan indeindex, sincesince setssets areare unordered in nature. • Check for membership using the in operator is faster in case of a set compared to a list, a tuple or a string. • Sets are used to : o Remove the duplicate elements, inturn allowing us to find the unique elements. o Compare two iterables for common elements or difference. EV0000180 · EV0000181 · EV0000182
4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Characteristics • Elements are unique – does not support repeated elements. • Elements should be hashable. • Hashing is a mechanism to convert the given element into an integer. • An object is hashable if it has a hash value which never changes during its lifetime (it needs a __hash__() method). Examples of hashable objects: • int, float, complex, bool, string, tuple, range, frozenset, bytes, decimal. Examples of Unhashable objects: • list, dict, set, bytearray EV0000183 · EV0000184 · EV0000185
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Examples • #set1={ [1, 'a', 'hi'], 2, 'hello', {3, 4.5, 'how r u' } } #TypeError: unhashable type: 'list' • #set2={ (1, 'a', 'hi'), 2, 'hello', {1:'hi', 2:'hello', 3:'how r u'} } #TypeError:yp unhashable type:yp 'dict' • set3={ (1, 'a', 'hi'), 2, 'hello', 5.56, 3+5j, True } print(set3) #{True, (1, 'a', 'hi'), 2, 5.56, (3+5j), 'hello'} • my_dict = {'name': 'John', tuple([1,2,3]):'values'} print(my_dict) #{'name': 'John', (1, 2, 3): 'values'} EV0000186 · EV0000187 · EV0000188
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Common methods of Set There are number of functions built into Python that takes set as the arguments. • len() • sum() • sorted() • max() • min() Note: Check reversed() EV0000189 · EV0000190 · EV0000191
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific methods of Set The dir() function returns all properties and methods of the specified object. o >>> dir(set) o ['__and__', '__class__', '__class_getitem__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getstate__', '__gt__', '__hash__', '__iand__', '__init__', '__init_subclass__', '__ior__', '__isub__', '__iter__', '__ixor__', '__le__', '__len__', '__lt__', '__ne__', '__new__', '__or__', '__rand__', '__reduce__', '__reduce_ex__', '__repr__', '__ror__', '__rsub__', '__rxor__', '__setattr__', '__sizeof__', '__str__', '__sub__', '__subclasshook__', '__xor__', 'add', 'clear', 'copy', 'difference', 'difference_update', 'discard', 'intersection', 'intersection_update', 'isdisjoint', 'issubset', 'issuperset', 'pop', 'remove', 'symmetric_difference', 'symmetric_difference_update', 'union', 'update'] EV0000192 · EV0000193 · EV0000194

8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Methods in detail len() Returns the number of elements in a >>> set1={22,33,11,55,44,120} container set. >>> len(set1) 6 max() Returns the element from Set with >>> set1= {42,73,25,75,64,145} maximumi valuel >>> max(set1)(1) 25 min() Returns the element from Set with >>> set1={25,43,21,66,45,120} maximum value >>> min(set1) 21 sum() Returns the sum of elements of the Set. >>> set1={4,10,2,7,25} >>> sum(set1) 7 48 EV0000195 · EV0000196 · EV0000197
9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Common methods of Set sorted() Returns the sorted sequence or sorted >>> set1= {22,33,11,55,44,120} collection in the form of list. >>> sorted(set1) [11, 22, 33, 44, 55, 120] >>>>>> s4={"pes"s4={ pes ,0,1}0 1} >>> sorted(s4) Traceback (most recent call last): File " <stdin>", line 1, in <module> TypeError: '<' not supported between instances of 'str' and 'int' 7 EV0000198 · EV0000199 · EV0000200
10. Source slide page 10 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Common operators applied on set Concatenation (|=) operator can be used to combine two sets together, results in a set that contains items of the two sets. >>> s1={1,2.34,(34,22),"python"} >>> s2={10,3.22,"programming"} >>> s1|=s2 >>> s1 {1, 2.34, 'python', (34, 22), 3.22, 'programming', 10} Repetition operator(*) – Throws an error as set has unique elements. >>> s1= {1,2,3,4,5,6} >>> print(s1*3) Traceback (most recent call last): File "<stdin>", line 1, in <module> TypeError: unsupported operand type(s) for *: 'set' and 'int' • EV0000201 · EV0000202 · EV0000203
11. Source slide page 11 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Common operators applied on set Something to find out: Why are we using the |= operator here? We used the + operator for lists and tuples, so why not use that here? What's the difference between +, |, and |=? EV0000204 · EV0000205 · EV0000206
12. Source slide page 12 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Common operators applied on set Membership operator(in) – returns True if a sequence with the specified value is present in the string otherwise False. >>> s1={1,2,3,4,5,6} >>> print(1 in s1) True >>>>>> print('a'print(a inin s1)s1) False Membership(not in) operator- Returns True if the element is not present in the set, else return False. >>> s1={1,2.34,(34,22),"python"} >>> "PYTHON" not in s1 True >>> (34,22) not in s1 False EV0000207 · EV0000208 · EV0000209
13. Source slide page 13 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of set union()- Return the union of sets as a new set. (i.e. all elements that are in either set.) . >>> s1={1,2,3,4,5,6} >>> s2={6,5,7,8,9,10} >>> print(s1.union(s2)) or >>> print(s1|s2) {1{1, 22, 33, 44, 55, 66, 77, 88, 99, 10}10} {1{1, 22, 33, 44, 55, 66, 77, 88, 99, 10}10} intersection()- Return the intersection of sets as a new set. (i.e. all elements that are in both sets.) . >>> s1={1,2,3,4,5,6} >>> s2={6,5,7,8,9,10} >>> print(s1.intersection(s2)) or >>> print(s1&s2) {5, 6} {5, 6} EV0000210 · EV0000211 · EV0000212
14. Source slide page 14 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of set difference()- Return the difference of two or more sets as a new set. >>> s1={1,2,3,4,5,6} >>> s2={6,5,7,8,9,10} >>> print(s1.difference(s2)) or >>> print(s1-s2) {1, 2, 3, 4} {1, 2, 3, 4} add()- adds an element in a container set. >>> s1={1,2,3,4,5,6} >>>s1= {1,2,3,4,5,6} >>> s1.add(7) >>> s1.add("pes") >>> s1 >>> s1 {1, 2, 3, 4, 5, 6, 7} {1, 2, 3, 4, 5, 6, "pes"} EV0000213 · EV0000214 · EV0000215

15. Source slide page 15 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of set symmetric_difference()- Return a set with the symmetric differences of two sets. >> s1={1,2,3,4,5,6} >>> s2={6,5,7,8,9,10} >>> print(s1.symmetric_difference(s2)) or >>> print(s1^s2) {1{1, 22, 33, 44, 77, 88, 99, 10}10} {1{1, 22, 33, 44, 77, 88, 99, 10}10} remove()- Removes the specified item in a set. If the item to remove does not exist, remove() will raise an error. >>> s1={1,2,3,4,5,6} >>> s1.remove(5) or >>> s1.remove(7) >>> print(s1) Displays an error {1, 2, 3, 4, 6} EV0000216 · EV0000217 · EV0000218

16. Source slide page 16 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of set discard()- Removes the specified item in a set. If the item to remove does not exist, discard() will NOT raise an error. >>> s1={1,2,3,4,5,6} >>> s1.remove(6) or >>> s1.discard(7) >>> print(s1) Doesn't display an Error. {1{1, 22, 33, 44, 5}5} pop()- Removes an item in a set. Sets are unordered, so when using the pop() method, doesn't know which item that gets removed. >>> s1={1,2,3,4,5,6} >>> s1.pop() 1 >>> print(s1) {2, 3, 4, 5, 6} EV0000219 · EV0000220 · EV0000221

17. Source slide page 17 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of set update()- updates the current set, by adding items from another set (or any other iterable). If the element is present in both the sets, only one appearance of element present in updated set >>> s1={120,12,23} >>> s1.update([22,15,67,"pes"]) >>>>>> s1s1 {67, 12, 15, 22, 23, 120, 'pes'} intersection_update()- Removes the items that is not present in both sets. >>> set1={"apple", "banana", "cherry"} >>> set2={"google", "microsoft", "apple"} >>> set1.intersection_update(set2) >>> set1 {'apple'} EV0000222 · EV0000223 · EV0000224

18. Source slide page 18 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of set difference_update()- Removes the items that exists in both sets. >>> set1= {"apple", "banana", "cherry"} >>> set2={"google", "microsoft", "apple"} >>> set1.difference_update(set2) >>> set1 {'cherry'{ cherry , 'banana'}banana } Symmetric_difference_update()- Updates the original set by removing items that are present in both sets, and inserting the other items. >>> set1={"apple", "banana", "cherry"} >>> set2= {"google", "microsoft", "apple"} >>> set1.symmetric_difference_update(set2) >>> set1 {'microsoft', 'google', 'banana', 'cherry'} EV0000225 · EV0000226 · EV0000227

19. Source slide page 19 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of set issubset()- Returns True if all items in set 'x' are present in set 'y'. >>> x = {"a", "b", "c"} >>> y = {"f", "e", "d", "c", "b", "a"} >>> x.issubset(y) True >>>>>> yy.issubset(x)issubset(x) False issuperset()- Returns True if all items in set 'y' are present in set 'x'. >>> x = {"f", "e", "d", "c", "b", "a"} >>> y = {"a", "b", "c"} >>> x.issuperset(y) True >>> y.issuperset(x) False EV0000228 · EV0000229 · EV0000230

20. Source slide page 20 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of set isdisjoint()- Returns True if no items in set s1 is present in set s2. >>> s1= {1,2,5} >>> s2={11,22,55} >>> s1.isdisjoint(s2) True clear()- Removes all the elements in a set. >>> x = {"f", "e", "d", "c", "b", "a"} >>> x.clear() >>> x set() EV0000231 · EV0000232 · EV0000233

21. Source slide page 21 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Example code 1: Display the elements of a set one by one in separated by a space between each of them. a = { 10, 30, 10, 40, 20, 50, 30 } for i in a : print(i, end = " ") Output: 40 10 50 20 30 EV0000234 · EV0000235 · EV0000236

22. Source slide page 22 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Example code 2: Create a set of numbers from 2 to n. Steps: create an empty set and add the elements to it. s = set() n=int(input("enter the value of n")) #10 for i in range(2,n+1): s.add(i) print(s) Output: {2, 3, 4, 5, 6, 7, 8, 9, 10} EV0000237 · EV0000238 · EV0000239
23. Source slide page 23 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Example code 3: Program to check whether a set is empty or not. Version 1: Version 2: s1=set() s1=set() if len(s1)==0: if(s1): print("set is empty") print("set is empty") else: else: print("not empty") print("not empty") Output: set is empty Note: Empty data structure is False EV0000240 · EV0000241 · EV0000242
24. Source slide page 24 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shylaja S S, Director, CDSAML & CCBD, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Prof. Chitra G M Dr. Chethana Srinivas 24 EV0000243 · EV0000244 · EV0000245

SHORT-NOTE RECAP

Source deck preserved: UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets. Its extracted page text is listed as searchable points and all rendered source pages are attached as figures. Match to a specific video is candidate-based; this index does not claim a manual content review. EV0000174

7 Source slide deck: JSON

This entry preserves the complete slide deck titled JSON as searchable extracted page text and page-faithful rendered images. Its association with a particular video is not confirmed; it remains a separate topic under Python for computational Problem Solving, Unit_02, to avoid an unsupported match. EV0000306 · EV0000307

Graphs and diagrams

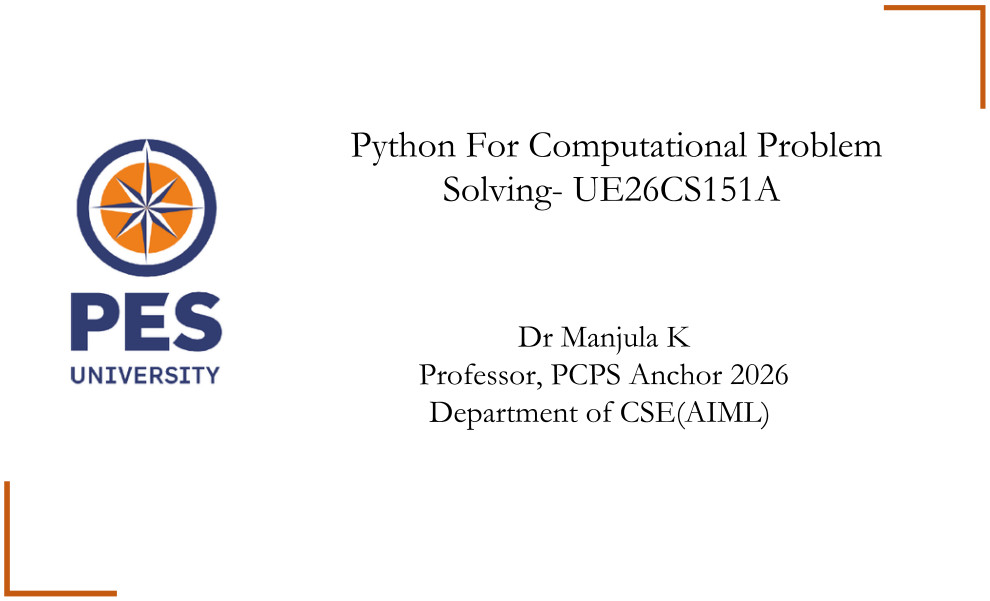


Figure G000077. JSON · source slide page 1.

EV0000307

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

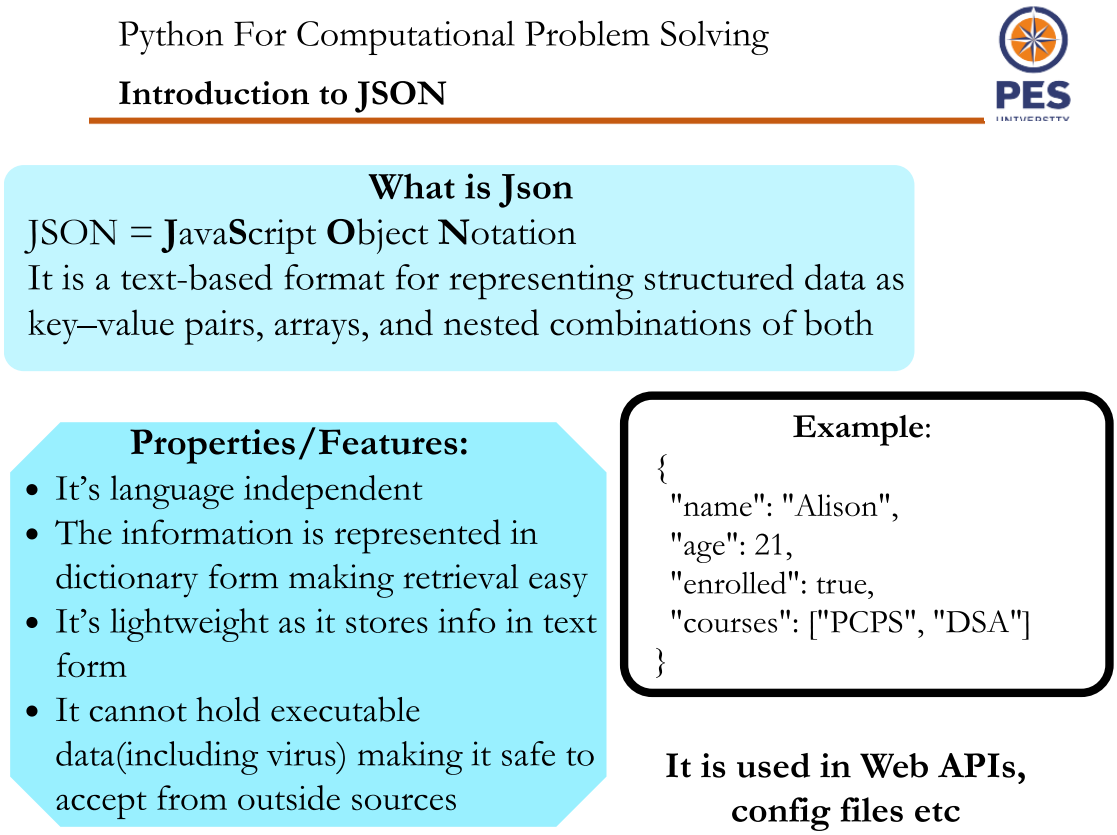


Figure G000078. JSON · source slide page 2.

EV0000310

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

Introduction to JSON

Only **6** data types exist in JSON. Every one has a direct Python equivalent, which is why json.load() can hand you back native Python objects with zero manual conversion:

JSON type	Python type	Notes
object {}	dict	unordered key–value pairs
array []	list	ordered, can mix types
"string"	str	**must** use double quotes
number	int or float	JSON doesn't distinguish; Python infers from format
true / false	True / False	lowercase in JSON, capitalized in Python
null	None	represents "no value"

Figure G000079. JSON · source slide page 3.

EV0000313

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

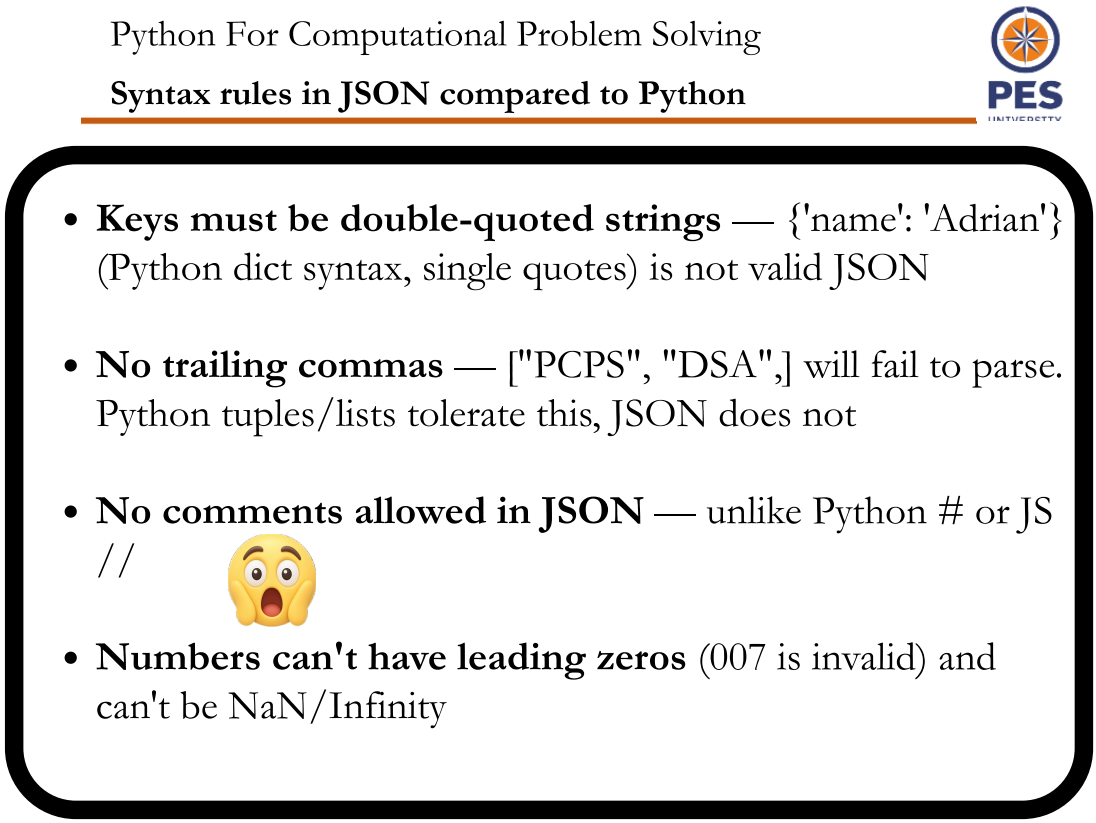


Figure G000080. JSON · source slide page 4.

EV0000316

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

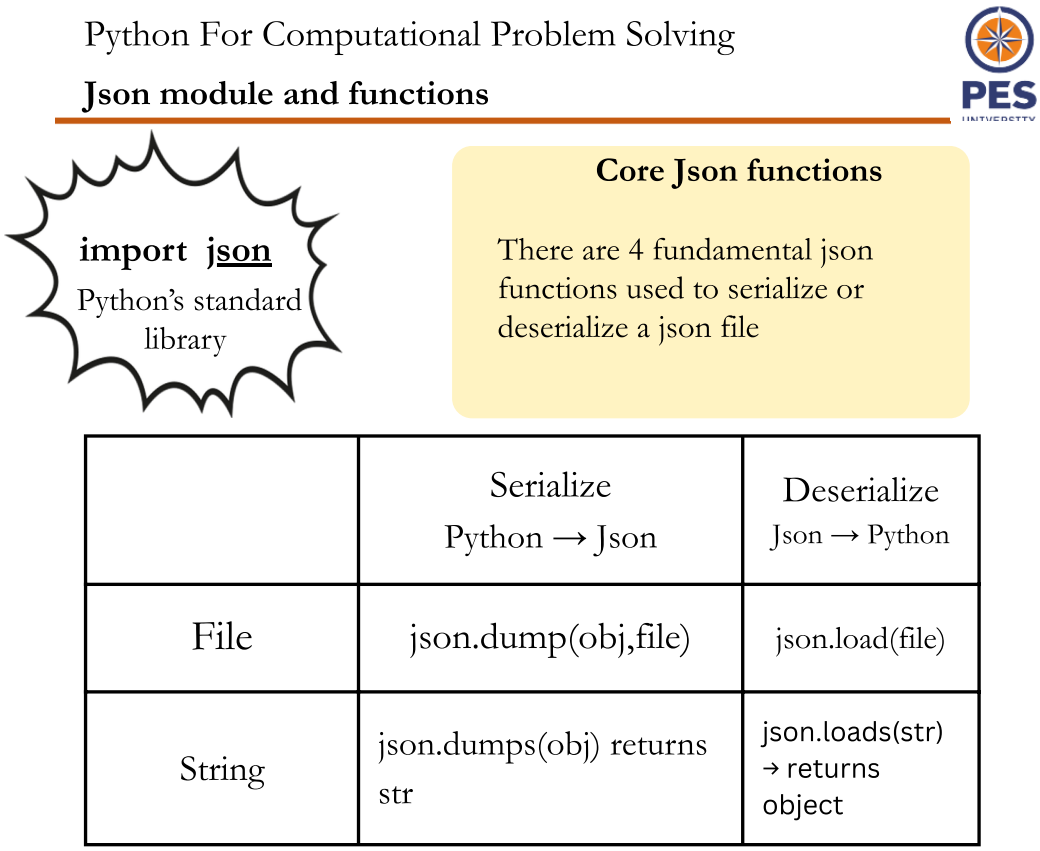


Figure G000081. JSON · source slide page 5.

EV0000319

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

Json.dump()

- json.dump() serializes a Python object and writes the resulting JSON text directly into an open file.
- dump() returns None
- Because it streams the translated data straight to the disk, it is highly efficient
- It takes your Python data (like a dictionary or a list) and writes it directly into a .json file on your hard drive.

```
import json

student = {"name": "Rigel", "age": 20, "department": "CS"}

with open("student.json", "w") as file:
    json.dump(student, file, indent=4)
```

dump() writes incrementally
If the program crashes mid-write without the file being properly closed, you can end up with a truncated, unparseable JSON file.
The with block guarantees the file closes (and flushes) even on error

Figure G000082. JSON · source slide page 6.

EV0000322

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

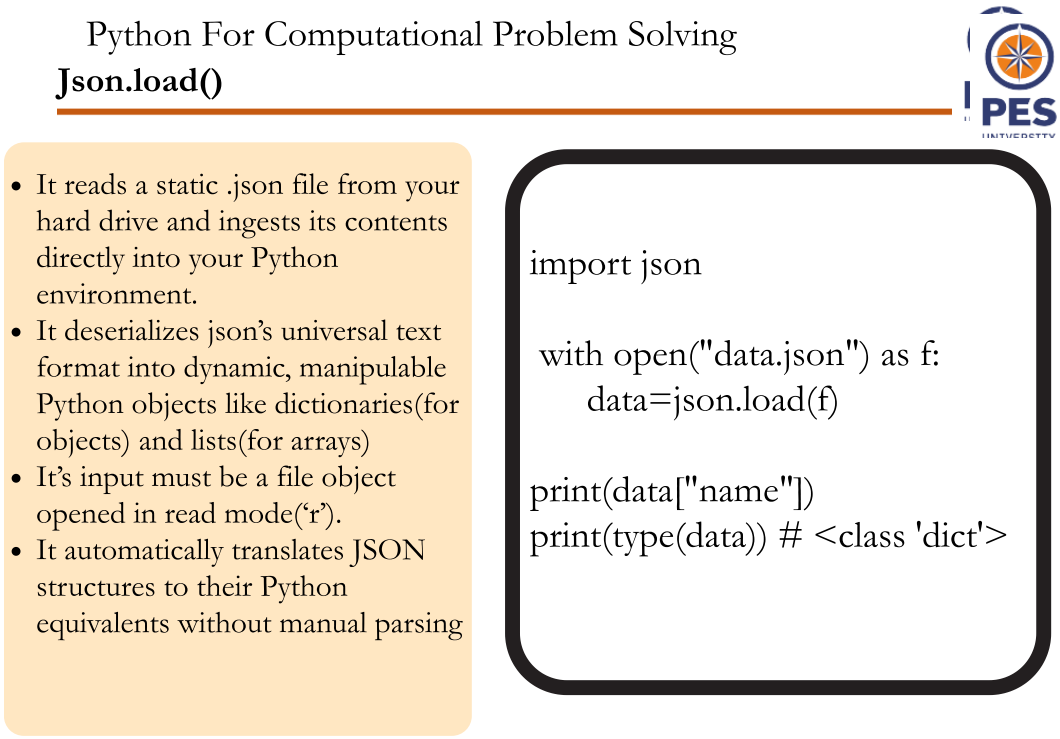


Figure G000083. JSON · source slide page 7.

EV0000325

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

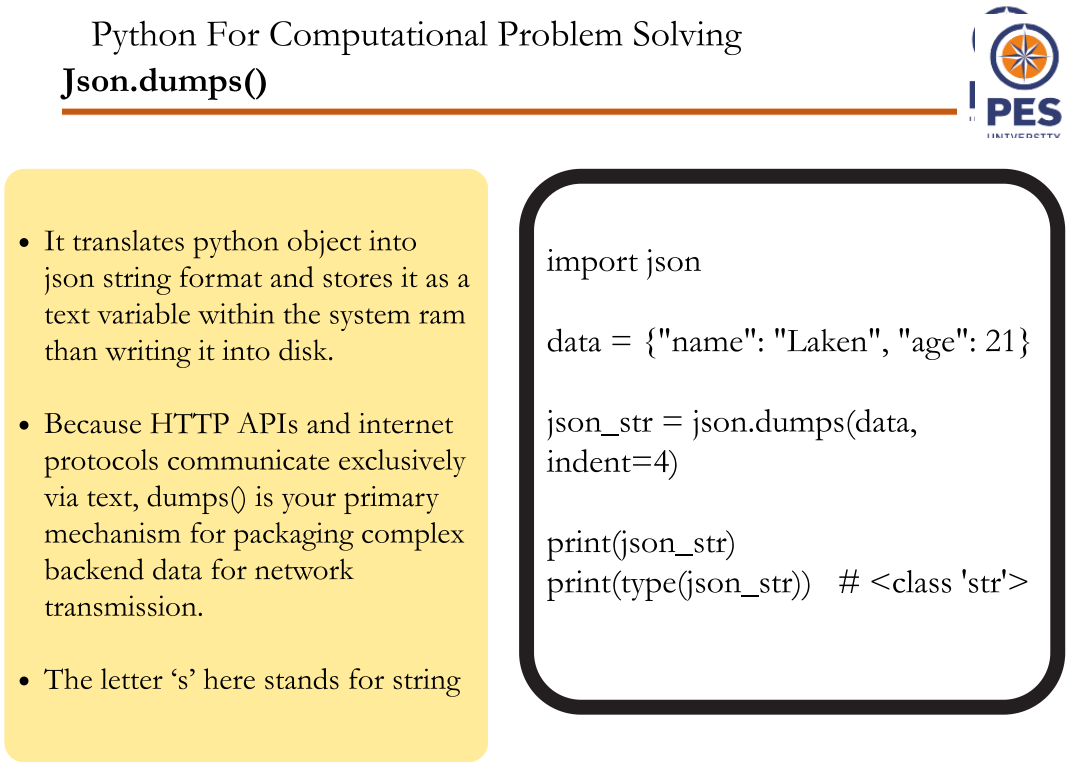


Figure G000084. JSON · source slide page 8.

EV0000328

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

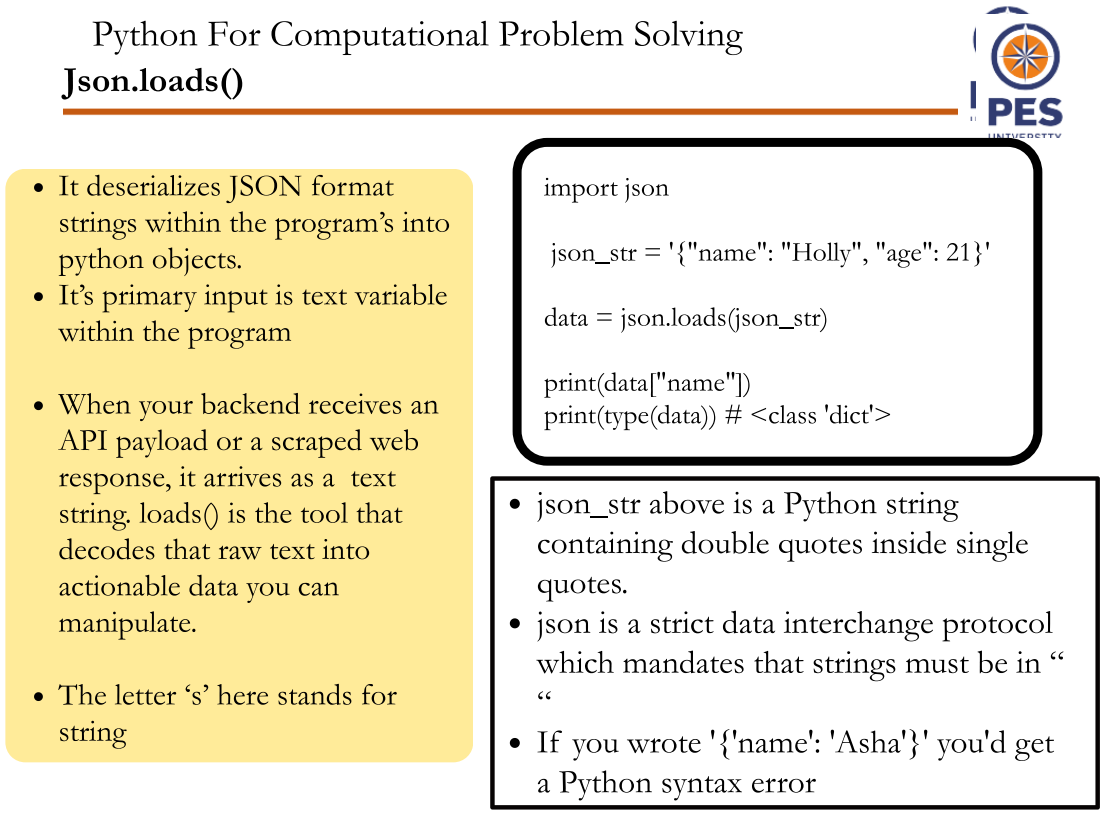


Figure G000085. JSON · source slide page 9.

EV0000331

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving
Common errors and good practices

- **The Translation Crash:** JSON is very basic and only understands simple things like text, numbers, and lists. It has no idea what a Python date, set, or custom object is. If you try to save them directly, your code will crash. You must simplify them first: turn dates into text strings, and turn sets into normal lists.
- **The Sneaky Key Change**: Python lets you use numbers as dictionary keys (like {1: "apple"}). JSON absolutely refuses this and only allows text keys. It will quietly change your number 1 into a text string "1" without warning you. When you load your data back into Python, you might get bugs because your numbers are now text.
- **The Broken Data Trap**: Never blindly trust a JSON file from the internet or a user. If the file is missing even a single comma or quote, your entire program will instantly crash when it tries to read it. Always wrap your code in a try/except block so it catches the error instead of breaking your app.
- **Ugly Text Syndrome**: By default, JSON is terrified of accents and special characters. It will take a word like "café" and scramble it into ugly machine code like caf\u00e9. To force it to leave your text alone and keep it readable, you have to tell it ensure_ascii=False when you save the data.
- **The Windows vs. Mac Glitch**: Different computers assume different default text formats when opening files. If you do not specify a format, moving a JSON file between computers can scramble the text. Always explicitly write encoding="utf-8" inside your open() function to guarantee it works flawlessly everywhere.

Figure G000086. JSON · source slide page 10.

EV0000334

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

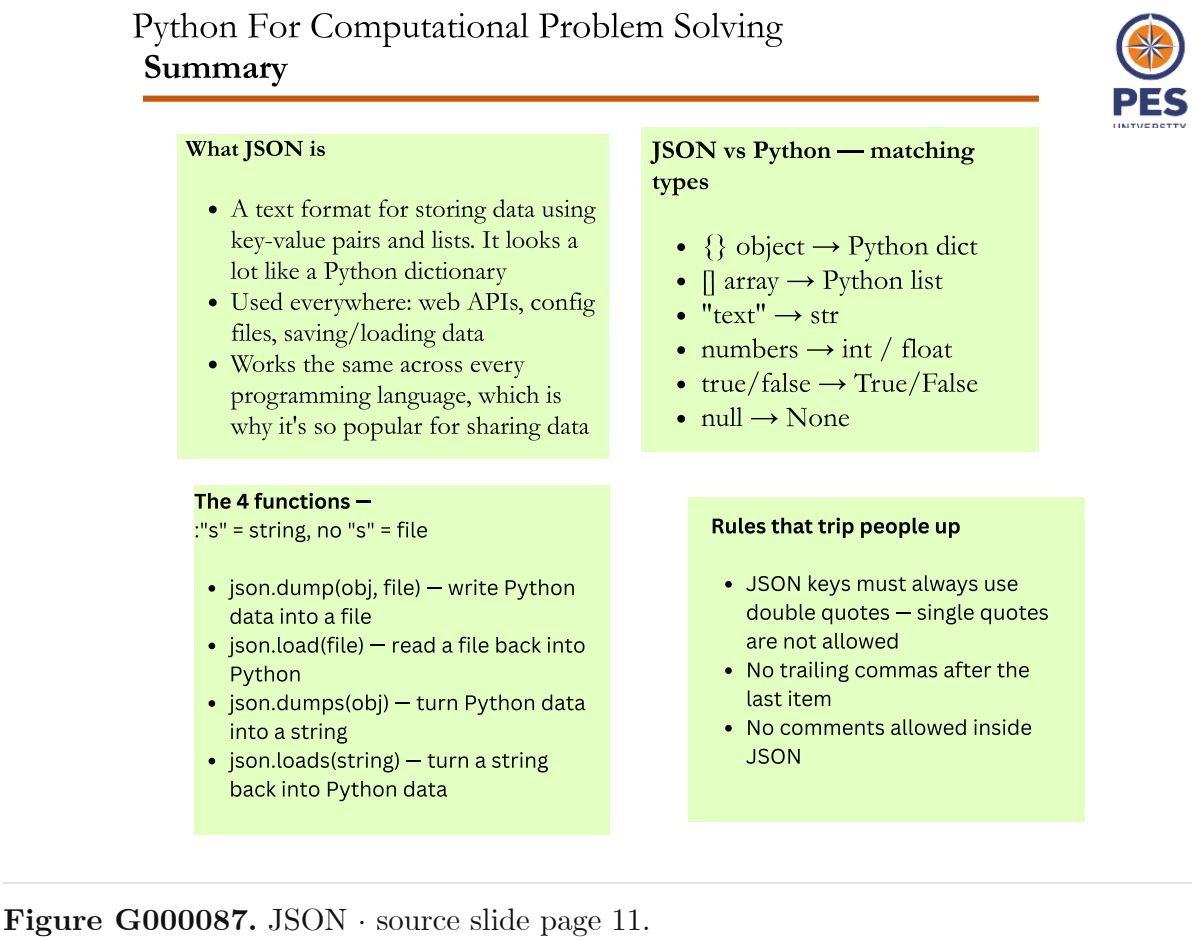


Figure G000087. JSON · source slide page 11.

EV0000337

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving
Time to be smart!

Read about TOON(data representation format born out of json) format

Project: The Campus Coffee Backend
The Mission: Build a simple Flask or FastAPI server that acts as a digital barista. It must receive a raw text coffee order from a student's phone, translate it into Python, calculate the bill, and send a text receipt back.

The Rules of the Game:

- The Internet is a Text Pipe: The phone app cannot send Python objects. It will send a raw JSON string.
- Your Code Only Speaks Python: You cannot do math on a text string. You must translate the incoming text into a live Python dictionary before calculating the price.
- The Return Trip: The phone app cannot read Python dictionaries. You must translate your final receipt back into a JSON string before sending it over the network.

Your Three Tasks:

1. Catch the Order (Deserialize): Create a web address (an endpoint) called /order. When the phone sends a JSON string like {"student": "John", "drink": "Latte", "quantity": 2}, your server must catch it and convert it into a Python dictionary.
2. Process the Data (Logic): Using Python, look at the dictionary. If a Latte costs $4, multiply that by the quantity to find the total price. Add a "status" tag that says "Order Confirmed".
3. Send the Receipt (Serialize): Create a new dictionary with the final details (student name, drink, total price, and status). Convert this live dictionary back into a flat JSON string and send it back to the phone.

The Test (How to know you passed):

- Input from Phone: {"student": "Alice", "drink": "Espresso", "quantity": 3}
- Required Output from Server: {"receipt_for": "Alice", "total_due": 12, "status": "Order Confirmed"}

Figure G000088. JSON · source slide page 12.

EV0000340

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

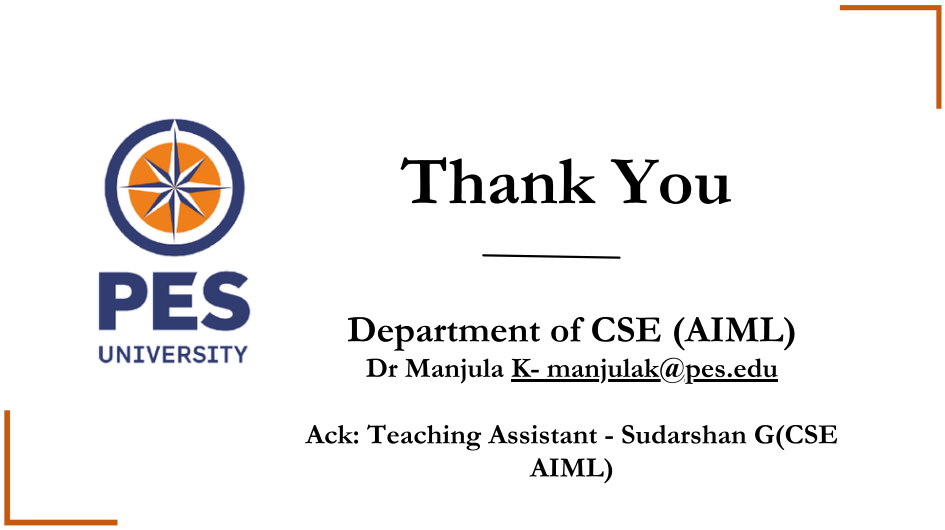


Figure G000089. JSON · source slide page 13.

EV0000343

> ***How to read it.*** Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): Python For Computational Problem Solving- UE26CS151A Dr Manjula K Professor, PCPS Anchor 2026 Department of CSE(AIML) EV0000306 · EV0000307 · EV0000308
2. Source slide page 2 (extracted text): Python For Computational Problem Solving Introduction to JSON What is Json JSON = JavaScript Object Notation It is a text-based format for representing structured data as key–value pairs, arrays, and nested combinations of both Example: Properties/Features: { It's language independent "name": "Alison", The information is represented in "age": 21, dictionary form making retrieval easy "enrolled": true, It's lightweight as it stores info in text "courses": ["PCPS", "DSA"] form } It cannot hold executable data(including virus) making it safe to It is used in Web APIs, accept from outside sources config files etc EV0000309 · EV0000310 · EV0000311

3. Source slide page 3 (extracted text): Python For Computational Problem Solving Introduction to JSON Only 6 data types exist in JSON. Every one has a direct Python equivalent, which is why json.load() can hand you back native Python objects with zero manual conversion: JSON type Python type Notes object {} dict unordered key–value pairs array [] list ordered, can mix types "string" str must use double quotes JSON doesn't distinguish; number int or float Python infers from format lowercase in JSON, true / false True / False capitalized in Python null None represents "no value" EV0000312 · EV0000313 · EV0000314
4. Source slide page 4 (extracted text): Python For Computational Problem Solving Syntax rules in JSON compared to Python Keys must be double-quoted strings — {'name': 'Adrian'} (Python dict syntax, single quotes) is not valid JSON No trailing commas — ["PCPS", "DSA",] will fail to parse. Python tuples/lists tolerate this, JSON does not No comments allowed in JSON — unlike Python # or JS // Numbers can't have leading zeros (007 is invalid) and can't be NaN/Infinity EV0000315 · EV0000316 · EV0000317
5. Source slide page 5 (extracted text): Python For Computational Problem Solving Json module and functions Core Json functions import json There are 4 fundamental json functions used to serialize orPython's standard deserialize a json file library Serialize Deserialize Python → Json Json → Python File json.dump(obj,file) json.load(file) json.loads(str) json.dumps(obj) returns String → returns str object EV0000318 · EV0000319 · EV0000320
6. Source slide page 6 (extracted text): Python For Computational Problem Solving Json.dump() import jsonjson.dump() serializes a Python object and writes the resulting student = {"name": "Rigel", "age": 20, JSON text directly into an open "department": "CS"} file. with open("student.json", "w") as file:dump() returns None json.dump(student, file, indent=4)Because it streams the translated data straight to the disk, it is highly efficient dump() writes incrementally It takes your Python data (like a If the program crashes mid-write without the file being properly closed,dictionary or a list) and writes it you can end up with a truncated,directly into a .json file on your unparseable JSON file. hard drive. The with block guarantees the file closes (and flushes) even on error EV0000321 · EV0000322 · EV0000323
7. Source slide page 7 (extracted text): Python For Computational Problem Solving Json.load() It reads a static .json file from your hard drive and ingests its contents directly into your Python import json environment. It deserializes json's universal text with open("data.json") as f:format into dynamic, manipulable Python objects like dictionaries(for data=json.load(f) objects) and lists(for arrays) It's input must be a file object print(data["name"]) opened in read mode('r'). print(type(data)) # <class 'dict'>It automatically translates JSON structures to their Python equivalents without manual parsing EV0000324 · EV0000325 · EV0000326
8. Source slide page 8 (extracted text): Python For Computational Problem Solving Json.dumps() It translates python object into import jsonjson string format and stores it as a text variable within the system ram data = {"name": "Laken", "age": 21}than writing it into disk. Because HTTP APIs and internet json_str = json.dumps(data, protocols communicate exclusively indent=4) via text, dumps() is your primary mechanism for packaging complex print(json_str) backend data for network print(type(json_str)) # <class 'str'> transmission. The letter 's' here stands for string EV0000327 · EV0000328 · EV0000329

9. Source slide page 9 (extracted text): Python For Computational Problem Solving Json.loads() It deserializes JSON format import json strings within the program's into json_str = '{"name": "Holly", "age": 21}'python objects. It's primary input is text variable data = json.loads(json_str)within the program print(data["name"]) When your backend receives an print(type(data)) # <class 'dict'> API payload or a scraped web response, it arrives as a text json_str above is a Python stringstring. loads() is the tool that containing double quotes inside singledecodes that raw text into actionable data you can quotes. manipulate. json is a strict data interchange protocol which mandates that strings must be in " The letter 's' here stands for " string If you wrote '{'name': 'Asha'}' you'd get a Python syntax error EV0000330 · EV0000331 · EV0000332
10. Source slide page 10 (extracted text): Python For Computational Problem Solving Common errors and good practices The Translation Crash: JSON is very basic and only understands simple things like text, numbers, and lists. It has no idea what a Python date, set, or custom object is. If you try to save them directly, your code will crash. You must simplify them first: turn dates into text strings, and turn sets into normal lists. The Sneaky Key Change: Python lets you use numbers as dictionary keys (like {1: "apple"}). JSON absolutely refuses this and only allows text keys. It will quietly change your number 1 into a text string "1" without warning you. When you load your data back into Python, you might get bugs because your numbers are now text. The Broken Data Trap: Never blindly trust a JSON file from the internet or a user. If the file is missing even a single comma or quote, your entire program will instantly crash when it tries to read it. Always wrap your code in a try/except block so it catches the error instead of breaking your app. Ugly Text Syndrome: By default, JSON is terrified of accents and special characters. It will take a word like "café" and scramble it into ugly machine code like caf\u00e9. To force it to leave your text alone and keep it readable, you have to tell it ensure_ascii=False when you save the data. The Windows vs. Mac Glitch: Different computers assume different default text formats when opening files. If you do not specify a format, moving a JSON file between computers can scramble the text. Always explicitly write encoding="utf-8" inside your open() function to guarantee it works flawlessly everywhere. EV0000333 · EV0000334 · EV0000335
11. Source slide page 11 (extracted text): Python For Computational Problem Solving Summary What JSON is JSON vs Python — matching types A text format for storing data using key-value pairs and lists. It looks a {} object → Python dict lot like a Python dictionary [] array → Python list Used everywhere: web APIs, config "text" → str files, saving/loading data numbers → int / float Works the same across every true/false → True/False programming language, which is null → None why it's so popular for sharing data The 4 functions — :"s" = string, no "s" = file Rules that trip people up json.dump(obj, file) — write Python JSON keys must always use data into a file double quotes — single quotes json.load(file) — read a file back into are not allowed Python No trailing commas after the json.dumps(obj) — turn Python data last item into a string No comments allowed inside json.loads(string) — turn a string JSON back into Python data EV0000336 · EV0000337 · EV0000338

12. Source slide page 12 (extracted text): Python For Computational Problem Solving Time to be smart! Read about TOON(data representation format born out of json) format Project: The Campus Coffee Backend The Mission: Build a simple Flask or FastAPI server that acts as a digital barista. It must receive a raw text coffee order from a student's phone, translate it into Python, calculate the bill, and send a text receipt back. The Rules of the Game: The Internet is a Text Pipe: The phone app cannot send Python objects. It will send a raw JSON string. Your Code Only Speaks Python: You cannot do math on a text string. You must translate the incoming text into a live Python dictionary before calculating the price. The Return Trip: The phone app cannot read Python dictionaries. You must translate your final receipt back into a JSON string before sending it over the network. Your Three Tasks: 1.Catch the Order (Deserialize): Create a web address (an endpoint) called /order. When the phone sends a JSON string like {"student": "John", "drink": "Latte", "quantity": 2}, your server must catch it and convert it into a Python dictionary. 2.Process the Data (Logic): Using Python, look at the dictionary. If a Latte costs $4, multiply that by the quantity to find the total price. Add a "status" tag that says "Order Confirmed". 3.Send the Receipt (Serialize): Create a new dictionary with the final details (student name, drink, total price, and status). Convert this live dictionary back into a flat JSON string and send it back to the phone. The Test (How to know you passed): Input from Phone: {"student": "Alice", "drink": "Espresso", "quantity": 3} Required Output from Server: {"receipt_for": "Alice", "total_due": 12, "status": "Order Confirmed"} EV0000339 · EV0000340 · EV0000341
13. Source slide page 13 (extracted text): Thank You Department of CSE (AIML) Dr Manjula K-manjulak@pes.edu Ack: Teaching Assistant - Sudarshan G(CSE AIML) EV0000342 · EV0000343 · EV0000344

SHORT-NOTE RECAP

Source deck preserved: JSON. Its extracted page text is listed as searchable points and all rendered source pages are attached as figures. Match to a specific video is candidate-based; this index does not claim a manual content review.

EV0000306

8 Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args

This entry preserves the complete slide deck titled UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args as searchable extracted page text and page-faithful rendered images. Its association with a particular video is not confirmed; it remains a separate topic under Python for computational Problem Solving, Unit_02, to avoid an unsupported match. EV0000408 · EV0000409

Graphs and diagrams

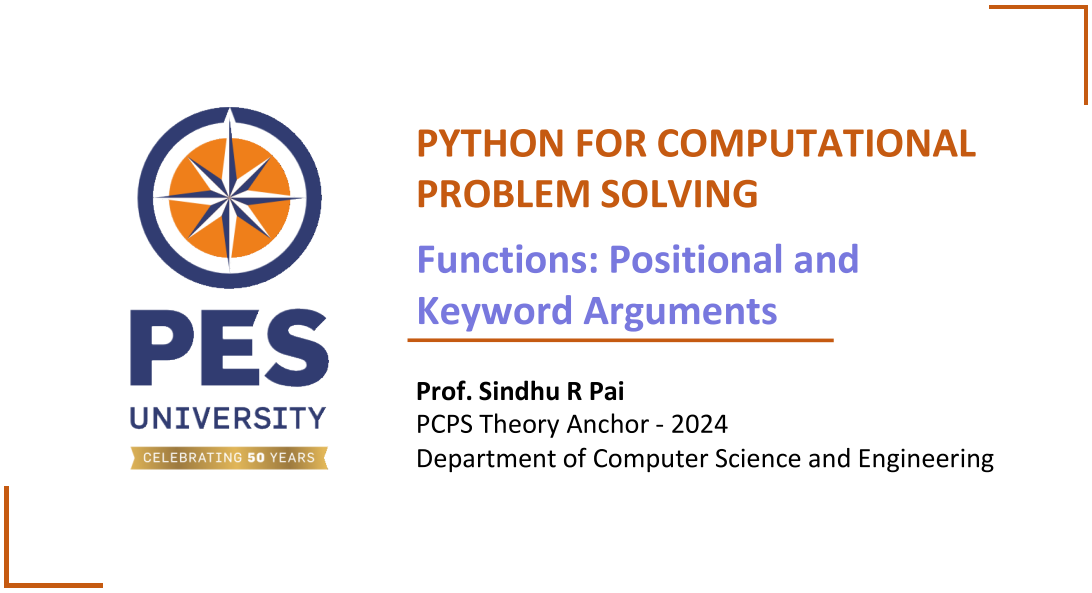


Figure G000111. UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args · source slide page 1.

EV0000409

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Function Call: Keyword and Positional Arguments

PES UNIVERSITY

When we call a function in Python, we can pass in two different types of arguments:

1. **Keyword arguments (named arguments) –** Arguments passed to a function or method which is preceded by a keyword (parameter_name) and an equals sign. Argument name must be same as the parameter name in the function definition.

2. **Positional arguments –** Arguments that need to be included in the proper position or order.

Note: If there are both keyword and positional arguments, **keyword arguments must follow positional arguments.**

Figure G000112. UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args · source slide page 2.

EV0000412

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

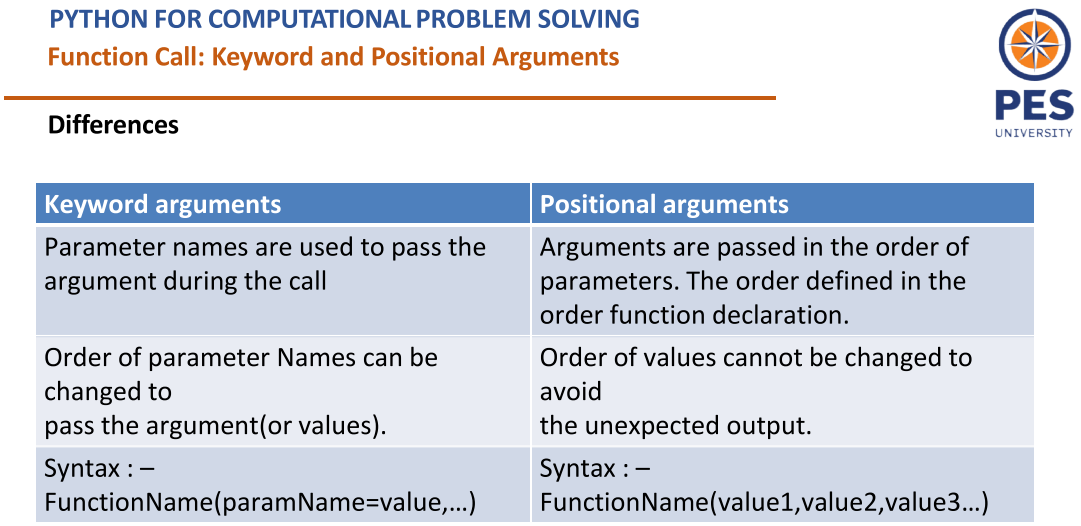


Figure G000113. UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args · source slide page 3.

EV0000415

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

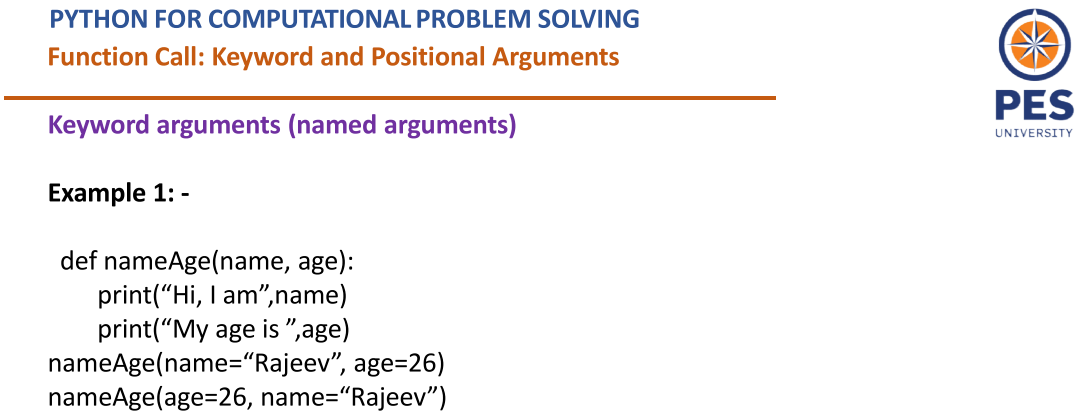


Figure G000114. UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args · source slide page 4.

EV0000418

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

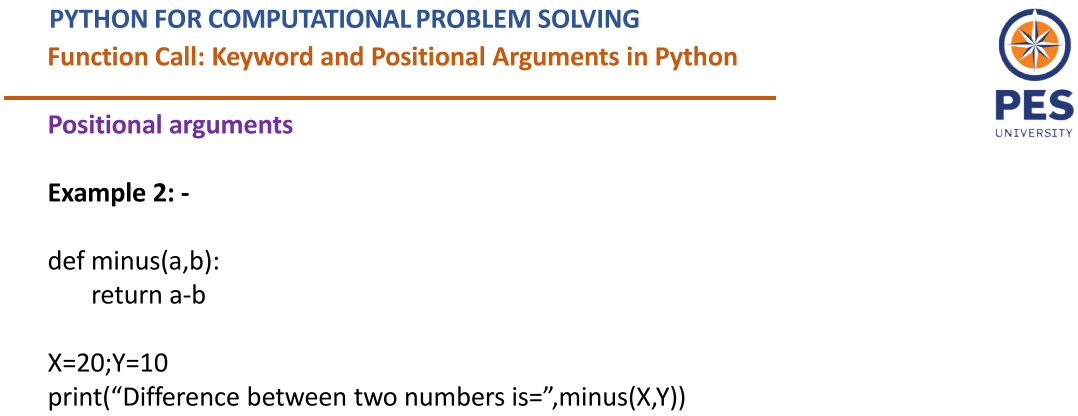


Figure G000115. UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args · source slide page 5.

EV0000421

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

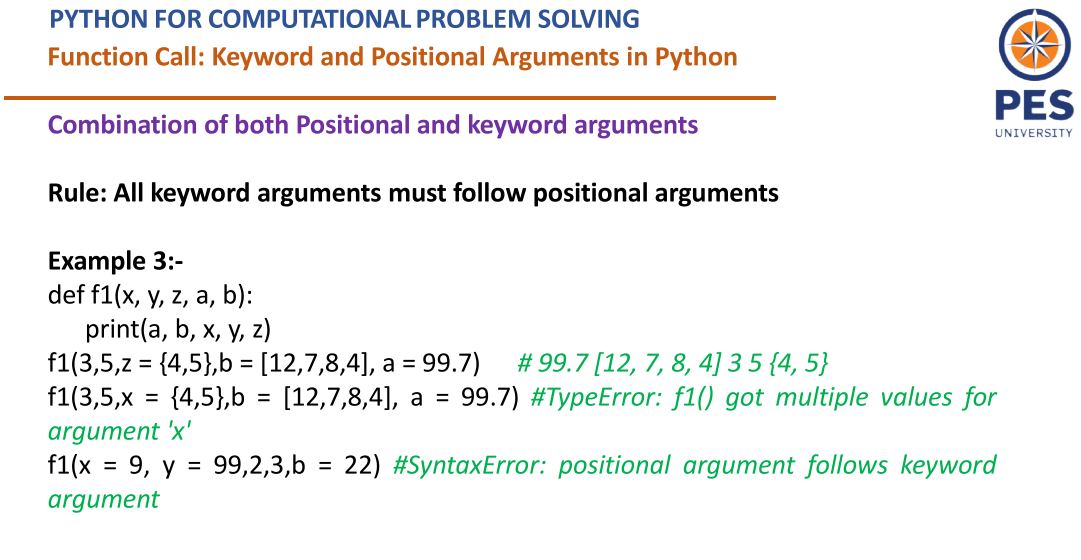


Figure G000116. UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args · source slide page 6.

EV0000424

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

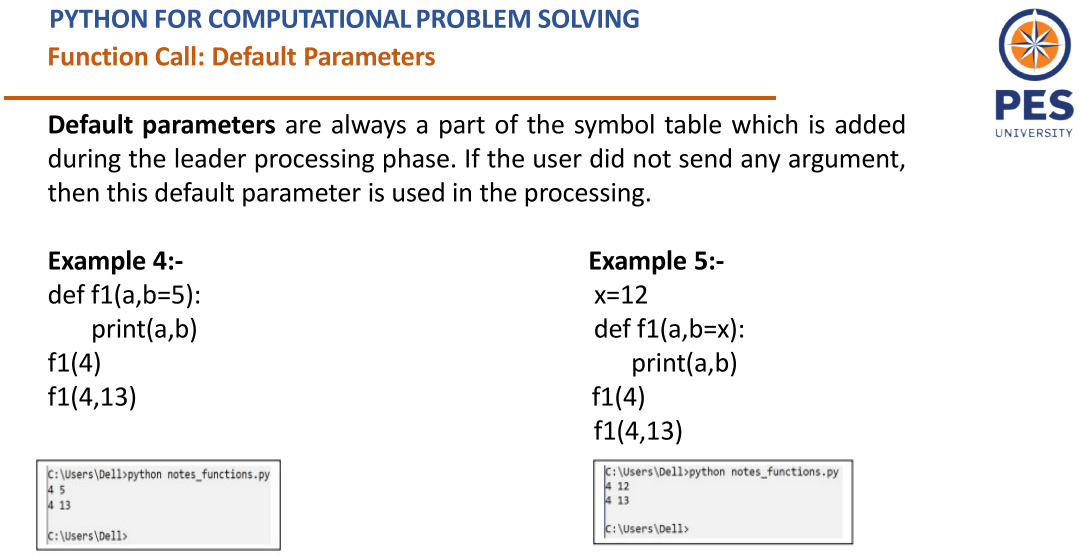


Figure G000117. UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args · source slide page 7.

EV0000427

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

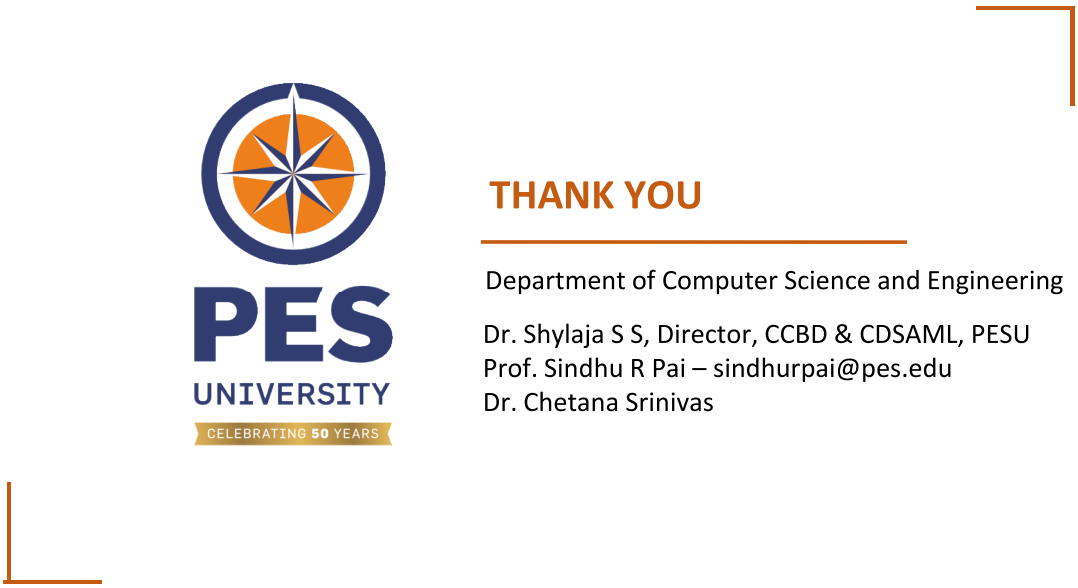


Figure G000118. UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args · source slide page 8.

EV0000430

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Functions: Positional and KKeyworddd AArgumentst Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000408 · EV0000409 · EV0000410
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function Call: Keyword and Positional Arguments When we call a function in Python, we can pass in two different types of arguments: 1. Keyword arguments (named arguments) – Arguments passed to a function or method which is preceded by a keyword (parameter_name) and an equalsequals signsign. ArgumentArgument namename mustmust bebe samesame asas thethe parameterparameter namename inin thethe function definition. 2. Positional arguments – Arguments that need to be included in the proper position or order. Note: If there are both keyword and positional arguments, keyword arguments must follow positional arguments. EV0000411 · EV0000412 · EV0000413

3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function Call: Keyword and Positional Arguments Differences Keyword arguments Positional arguments Parameter names are used to pass the Arguments are passed in the order of argumentg duringg the call parameters.p The order defined in the order function declaration. Order of parameter Names can be Order of values cannot be changed to changed to avoid pass the argument(or values). the unexpected output. Syntax : – Syntax : – FunctionName(paramName=value,...) FunctionName(value1,value2,value3...) EV0000414 · EV0000415 · EV0000416
4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function Call: Keyword and Positional Arguments Keyword arguments (named arguments) Example 1: - def nameAge(name, age): print("Hiprint(Hi, II am"name)am ,name) print("My age is ",age) nameAge(name="Rajeev", age=26) nameAge(age=26, name="Rajeev") EV0000417 · EV0000418 · EV0000419
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function Call: Keyword and Positional Arguments in Python Positional arguments Example 2: - def minus(a,b): returnreturn aa-bb X=20;Y=10 print("Difference between two numbers is=",minus(X,Y)) EV0000420 · EV0000421 · EV0000422
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function Call: Keyword and Positional Arguments in Python Combination of both Positional and keyword arguments Rule: All keyword arguments must follow positional arguments Example 3:- defdef f1(xf1(x, yy, zz, aa, b):b): print(a, b, x, y, z) f1(3,5,z = {4,5},b = [12,7,8,4], a = 99.7) # 99.7 [12, 7, 8, 4] 3 5 {4, 5} f1(3,5,x = {4,5},b = [12,7,8,4], a = 99.7) #TypeError: f1() got multiple values for argument 'x' f1(x = 9, y = 99,2,3,b = 22) #SyntaxError: positional argument follows keyword argument EV0000423 · EV0000424 · EV0000425
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function Call: Default Parameters Default parameters are always a part of the symbol table which is added during the leader processing phase. If the user did not send any argument, then this default parameter is used in the processing. Example 4:- Example 5:- defdef f1(af1(a,b=5):b=5): x=12x=12 print(a,b) def f1(a,b=x): f1(4) print(a,b) f1(4,13) f1(4) f1(4,13) EV0000426 · EV0000427 · EV0000428
8. Source slide page 8 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shylaja S S, Director, CCBD & CDSAML, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Dr. Chetana Srinivas EV0000429 · EV0000430 · EV0000431

SHORT-NOTE RECAP

Source deck preserved: UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args. Its extracted page text is listed as searchable points and all rendered source pages are attached as figures. Match to a specific video is candidate-based; this index does not claim a manual content review.

EV0000408

9 Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args

This entry preserves the complete slide deck titled UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args as searchable extracted page text and page-faithful rendered images. Its association with a particular video is not confirmed; it remains a separate topic under Python for computational Problem Solving, Unit_02, to avoid an unsupported match. EV0000432 · EV0000433

Graphs and diagrams

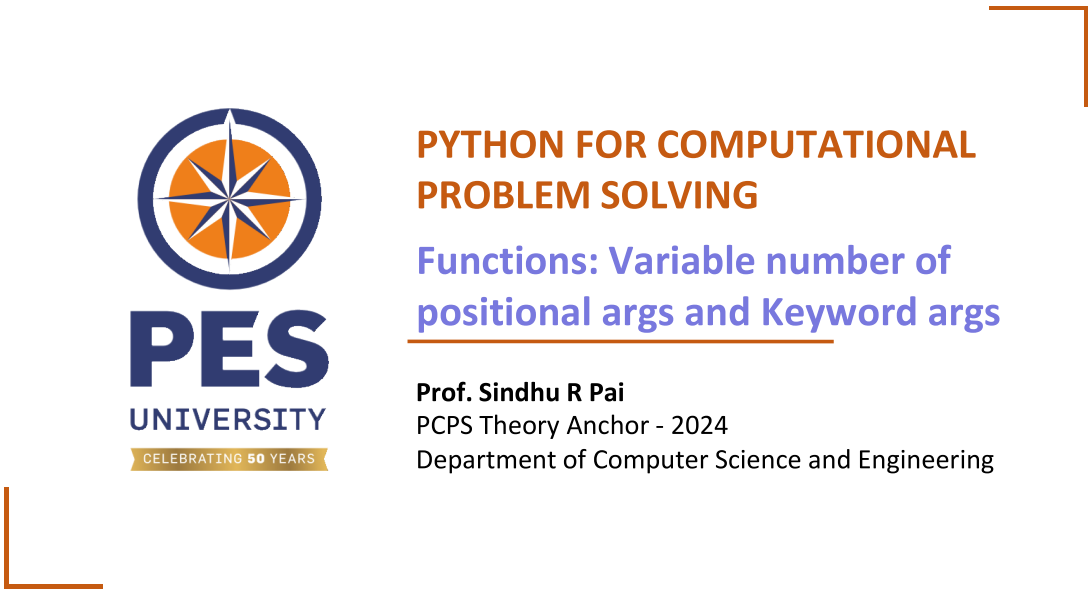


Figure G000119. UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args · source slide page 1.

EV0000433

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Function Call: Arguments and Parameters

PES UNIVERSITY

- In the function definition, variables are specified which receive the arguments when the function is called or invoked. **These are called parameters**.
- The parameters are **always variables**.
- When the function call is made, the control is transferred to the function definition. The arguments are evaluated and copied to the corresponding parameters.
- This is called **parameter passing by value or call by value**.
- The value of arguments are passed by value to the parameters.

2

Figure G000120. UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args · source slide page 2.

EV0000436

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Function Call: Arguments and Parameters

PES UNIVERSITY

- When a function performs a specified task, it may require some values to operate on.
- These values have to be provided when calling a function as input.
- These values have to be put within the pair of round parentheses in the function call.
- **They are called as arguments**.

3

Figure G000121. UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args · source slide page 3.

EV0000439

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Function: Global and Local variables

PES UNIVERSITY

There are two types of variables

- **Global Variables:** Variables created outside all functions. Scope of Global variable is within the file and outside that file too.
- **Local variables:** Variables created inside the function and accessible only to that function. Scope of the local variable is within the function.

Example 1:

```
x=10
print("global first outside",x)
def f1():
    x=11
    #local variable x.
   #Life and scope is only within this function
   print("inside",x)
f1()
print("global second outside",x)
```

```
C:\Users\Dell>python notes_functions.py
global first outside 10
inside 11
global second outside 10

C:\Users\Dell>
```

4

Figure G000122. UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args · source slide page 4.

EV0000442

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

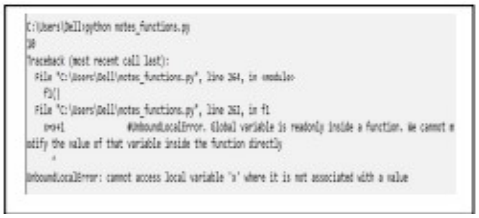
PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Function: Global and Local variables

PES UNIVERSITY

Example 2: Global variable is read-only inside a function. We cannot modify the value of that variable inside the function directly.

```
x=10
print(x)
def f1():
    x=x+1 #UnboundLocalError.
    print("inside",x)
f1()
print("outside",x)
```



5

Figure G000123. UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args · source slide page 5.

EV0000445

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

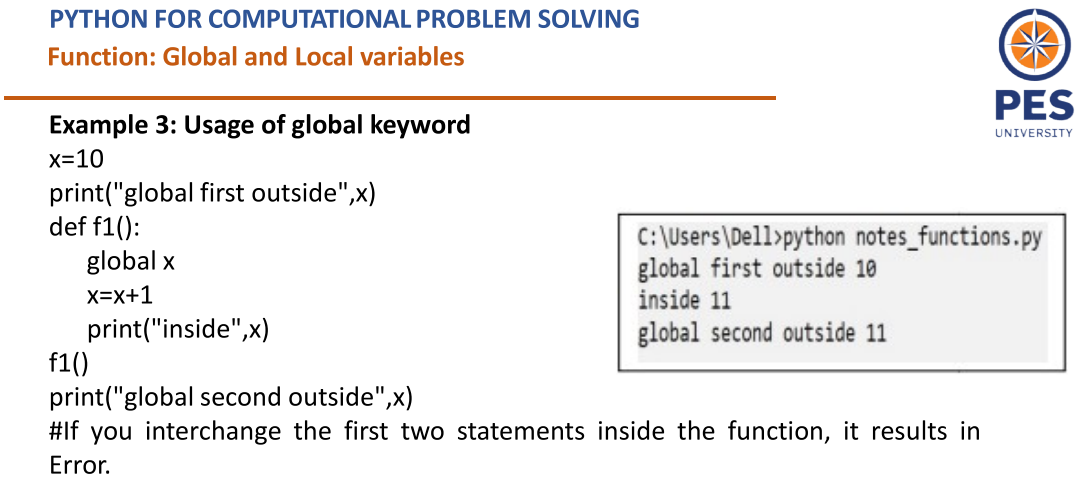


Figure G000124. UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args · source slide page 6.

EV0000448

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Function Call: Arguments and Parameters

PES UNIVERSITY

Example 4:- Calculate the sum of two numbers given and return the sum.

```
def add_numbers(x,y):
    sum = x + y
    return sum

output = add_Numbers(10,5)
print("The sum of two numbers is = ",output)
```

Output:

The sum of two numbers is = 15

7

Figure G000125. UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args · source slide page 7.

EV0000451

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Function Call: Arguments and Parameters

PES UNIVERSITY

Example 5:- Calculate the area of triangle given the dimensions.

```
def area_tri(b,h) :
    a=0.5*b*h
    return a
Area=area_tri(5,6)
print("area=",Area)
```

- The function definition has two parameters, which indicate that it requires two arguments to be passed when the function is called/invoked.
- The arguments are constants in this example, 5 and 6.
- These values are copied to the parameters b and h respectively.

8

Figure G000126. UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args · source slide page 8.

EV0000454

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Function Call: Arguments and Parameters

When a function is called, it is imperative that the number of arguments passed **MUST ALWAYS MATCH** the number of parameters in the function definition

Example 6:-

```
def  product(x,y,z) :
        a=x*y*z return a
r=product(5,6,2)
#r=product(5) #error : too few arguments
#r=product(5,6,2,7) #error : too many arguments
print(“product =",r)
```

Output:

product = 60

9

Figure G000127. UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args · source slide page 9.

EV0000457

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

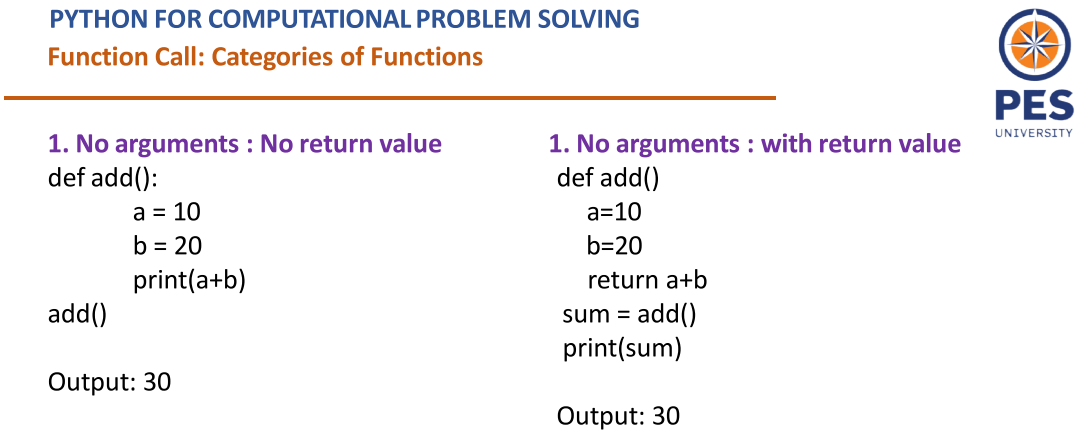


Figure G000128. UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args · source slide page 10.

EV0000460

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

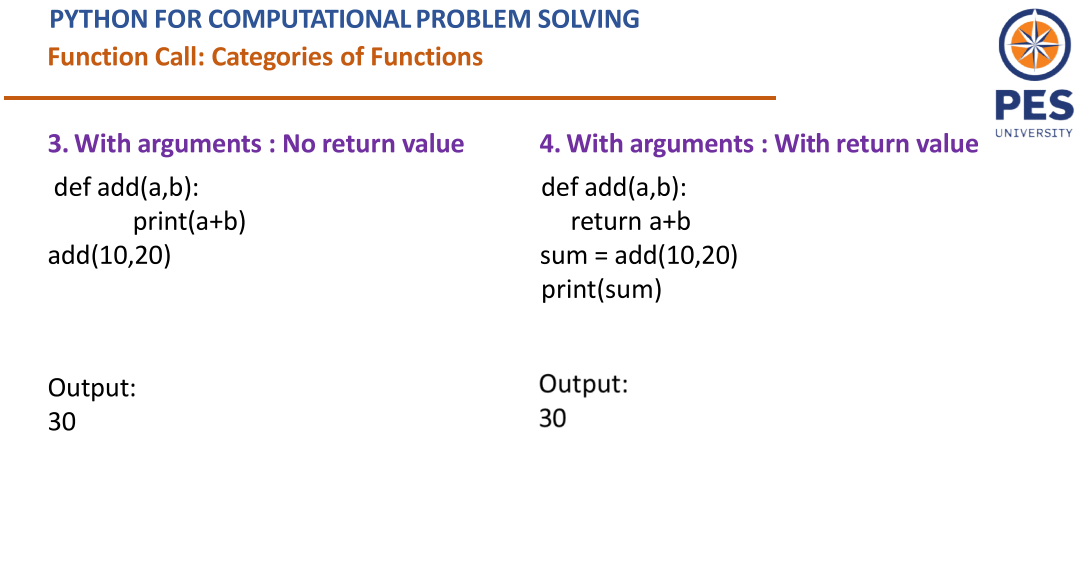


Figure G000129. UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args · source slide page 11.

EV0000463

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Variable number of positional arguments: *arg [Any variable can be used with *]

PES UNIVERSITY

- * handles the variable number of positional arguments in the function.
- arg is of type tuple inside the function.

Example 7:

```
def f1(*arg):
    print(arg, type(arg)) #arg is of type tuple
    for i in arg:
        print(i)
f1(2,1,6,4,9,7) #All three function calls are valid
f1()
f1(2,4)
```

```
C:\Users\Dell>python notes_functions.py
(2, 1, 6, 4, 9, 7) <class 'tuple'>
2
1
6
4
9
7
() <class 'tuple'>
(2, 4) <class 'tuple'>
2
4

C:\Users\Dell>
```

12

Figure G000130. UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args · source slide page 12.

EV0000466

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Variable number of keyword arguments: **kwarg [Any variable can be used with **]

PES UNIVERSITY

- ** handles the variable number of keyword arguments in the function.
- kwarg is of type dictionary inside the function.

Example 8:

```
def f1(**kwarg):
    print(kwarg, type(kwarg)) #kwarg is of type dict
    for i in kwarg:
        print(i,end="")
    print()
    for i in kwarg.keys():
        print(i,end="")
    print()
    for i in kwarg.values():
        print(i,end="")
f1(a=2,b=1,c=6)
```

```
C:\Users\Dell>python notes_functions.py
{'a': 2, 'b': 1, 'c': 6} <class 'dict'>
a b c
a b c
2 1 6
C:\Users\Dell>
```

13

Figure G000131. UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args · source slide page 13.

EV0000469

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

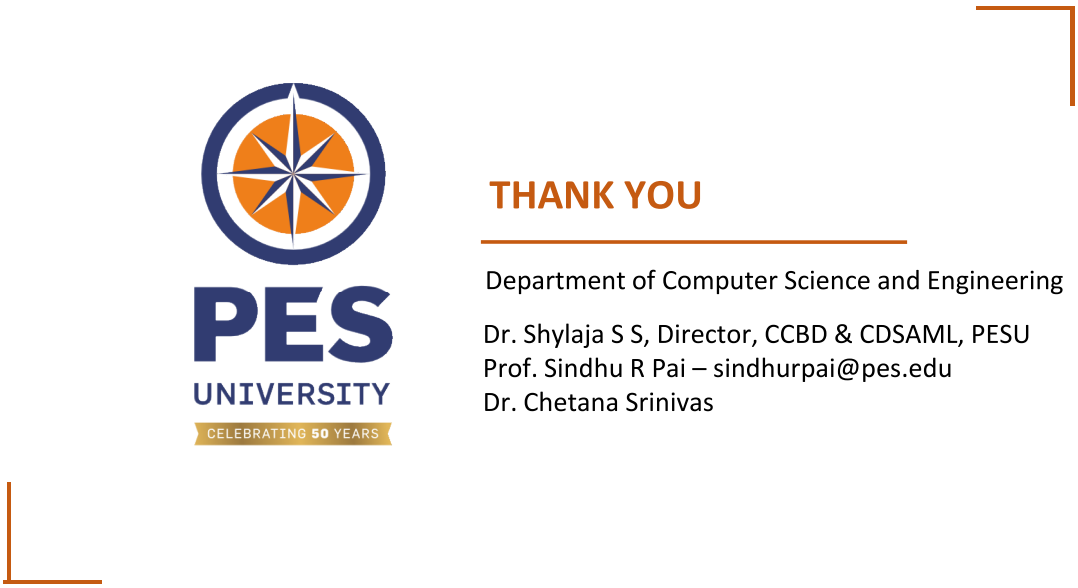


Figure G000132. UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args · source slide page 14.

EV0000472

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Functions: Variable number of positionaliti l args andd KKeywordd args Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000432 · EV0000433 · EV0000434
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function Call: Arguments and Parameters • In the function definition, variables are specified which receive the arguments when the function is called or invoked. These are called parameters. • The parameters are always variables. • When the function call is made, the control is transferred to the function definition. The arguments are evaluated and copied to the corresponding parameters. • This is called parameter passing by value or call by value. • The value of arguments are passed by value to the parameters. 2 EV0000435 · EV0000436 · EV0000437

3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function Call: Arguments and Parameters • When a function performs a specified task, it may require some values to operate on. • These values have to be provided when calling a function as input. • These values have to be put within the pair of round parentheses in the function call. • They are called as arguments. 3 EV0000438 · EV0000439 · EV0000440

4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function: Global and Local variables There are two types of variables • Global Variables: Variables created outside all functions. Scope of Global variable is within the file and outside that file too. • Local variables: Variables created inside the function and accessible only to that function. Scope of the local variable is within the function. ExampleExample 1:1: x=10 print("global first outside",x) def f1(): x=11 #local variable x. #Life and scope is only within this function print("inside",x) f1() print("global second outside",x) 4 EV0000441 · EV0000442 · EV0000443

5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function: Global and Local variables Example 2: Global variable is read-only inside a function. We cannot modify the value of that variable inside the function directly. x=10 print(x) def f1(): x=x+1x=x+1 #UnboundLocalError#UnboundLocalError. print("inside",x) f1() print("outside",x) 5 EV0000444 · EV0000445 · EV0000446

6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function: Global and Local variables Example 3: Usage of global keyword x=10 print("global first outside",x) def f1(): global x x=x+1x=x+1 print("inside",x) f1() print("global second outside",x) #If you interchange the first two statements inside the function, it results in Error. 6 EV0000447 · EV0000448 · EV0000449

7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Function Call: Arguments and Parameters Example 4:- Calculate the sum of two numbers given and return the sum. def add_numbers(x,y): sumsum == xx ++ yy return sum output = add_Numbers(10,5) print(“The sum of two numbers is = ”,output) Output: The sum of two numbers is = 15 7 EV0000450 · EV0000451 · EV0000452

8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function Call: Arguments and Parameters Example 5:- Calculate the area of triangle given the dimensions. def area_tri(b,h) : a=0.5*b*h return a Area=area_tri(5,6) print("area=",Area) • The function definition has two parameters, which indicate that it requires two arguments to be passed when the function is called/invoked. • The arguments are constants in this example, 5 and 6. • These values are copied to the parameters b and h respectively. 8 EV0000453 · EV0000454 · EV0000455

9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function Call: Arguments and Parameters When a function is called, it is imperative that the number of arguments passed MUST ALWAYS MATCH the number of parameters in the function definition Example 6:- def product(x,y,z) : a=x*y*z return a r=product(5,6,2) #r=product(5) #error : too few arguments #r=product(5,6,2,7) #error : too many arguments print(“product =",r) Output: product = 60 9 EV0000456 · EV0000457 · EV0000458

10. Source slide page 10 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function Call: Categories of Functions 1. No arguments : No return value 1. No arguments : with return value def add(): def add() a = 10 a=10 b = 20 b=20 print(a+b) return a+b add() sum = add() print(sum) Output: 30 Output: 30 10 EV0000459 · EV0000460 · EV0000461
11. Source slide page 11 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Function Call: Categories of Functions 3. With arguments : No return value 4. With arguments : With return value def add(a,b): def add(a,b): print(a+b) return a+b add(10,20) sum = add(10,20) print(sum)print(sum) Output: 30 11 EV0000462 · EV0000463 · EV0000464
12. Source slide page 12 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Variable number of positional arguments: *arg [Any variable can be used with *] • * handles the variable number of positional arguments in the function. • arg is of type tuple inside the function. Example 7: def f1(*arg): print(arg, type(arg)) #arg is of type tuple for i in arg: print(i) f1(2,1,6,4,9,7) #All three function calls are valid f1() f1(2,4) 12 EV0000465 · EV0000466 · EV0000467
13. Source slide page 13 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Variable number of keyword arguments: **kwarg [Any variable can be used with **] • ** handles the variable number of keyword arguments in the function. • kwarg is of type dictionary inside the function. Example 8: def f1(**kwarg): print(kwarg, type(kwarg)) #kwarg is of type dict for i in kwarg: print(i,end="") print() for i in kwarg.keys(): print(i,end="") print() for i in kwarg.values(): print(i,end="") 13 f1(a=2,b=1,c=6) EV0000468 · EV0000469 · EV0000470
14. Source slide page 14 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shylaja S S, Director, CCBD & CDSAML, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Dr. Chetana Srinivas EV0000471 · EV0000472 · EV0000473

SHORT-NOTE RECAP

Source deck preserved: UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args. Its extracted page text is listed as searchable points and all rendered source pages are attached as figures. Match to a specific video is candidate-based; this index does not claim a manual content review.

EV0000432

A Symbol glossary

Symbol	Name	Meaning	Appears in

B Sources, coverage and gaps

Source	Path	Use
Official unit catalogue	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/units.json	Official course/unit title and index.

Source	Path	Use
Lists and Tuples transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/029_Lists_and_Tuples__c3dbcf69ee3f547f.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Lists and Tuples transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/029_Lists_and_Tuples__c3dbcf69ee3f547f.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Lists and Tuples transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/029_Lists_and_Tuples__c3dbcf69ee3f547f.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9995; only the JSON segmentation supplies the displayed timestamps.
Lists and Tuples transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/029_Lists_and_Tuples__c3dbcf69ee3f547f.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Dictionary transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/034_Dictionary__9c4cb33a65041296.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Dictionary transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/034_Dictionary__9c4cb33a65041296.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Dictionary transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/034_Dictionary__9c4cb33a65041296.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9998; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Dictionary transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/034_Dictionary__9c4cb33a65041296.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Files transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/046_Files__6b7a6928ed4ec68a.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Files transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/046_Files__6b7a6928ed4ec68a.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Files transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/046_Files__6b7a6928ed4ec68a.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9998; only the JSON segmentation supplies the displayed timestamps.
Files transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/046_Files__6b7a6928ed4ec68a.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Function_Call_Definition_Variable_arguments transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/051_Function_Call_Definition_Variable_arguments__33f527b86ff6abf2.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Function_Call_Definition_Variable_arguments transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/051_Function_Call_Definition_Variable_arguments__33f527b86ff6abf2.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Function_Call_Definition_Variable_arguments transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/051_Function_Call_Definition_Variable_arguments__33f527b86ff6abf2.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9998; only the JSON segmentation supplies the displayed timestamps.
Function_Call_Definition_Variable_arguments transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/051_Function_Call_Definition_Variable_arguments__33f527b86ff6abf2.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 2 SindhuRPai Class 3 tuples (converted PDF; original hash f48314ea61e3a37dde957739f25e19751931c96d26a4c1d7f4015fc3e3fd15f1)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.

Source	Path	Use
Slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 2 SindhuRPai Class 5 Dictionary (converted PDF; original hash e12e33099905b6cc61d27afa43ae9798feddcd77af96ff7b373169cdeaff78c2)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 2 SindhuRPai Class 7 sets (converted PDF; original hash d55f37758c0a92a8cfd16debf5d71e7fe5eb8f198e1c939871c68b44e0d77594)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.

Source	Path	Use
Slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 2 SindhuRPai Class 12 13 14 File handling text files CSE files (converted PDF; original hash c60226e6bca6d6586282798100dc860deb793fc64d9dc613935de662682a2273)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: JSON	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: JSON	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for JSON (converted PDF; original hash 6d3e445fc984614d9591c7e9bf6f4fb66a9b7bce55ca051618d12a4126586a18)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.

Source	Path	Use
Slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 2 SindhuRPai Class 15 function definition call (converted PDF; original hash c658adc22df5ed0170035e56c1f1e3a699d4178f0fb43230ab11d8c0d18821b8)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.

Source	Path	Use
Page-render source for UE24CS151A PCPS Unit 2 SindhuRPai Class 16 function positional keyword args (converted PDF; original hash 792efdd1303b9d91389b09b4a2042f4c06dcc9c89cf41cf767c7af0de93e7e12)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 2 SindhuRPai Class 17 18 Variable number args kw args (converted PDF; original hash 14a7b0878d91cb0b8e9ab72ffa872ba4715bd95ed3834b9c977389c05def10f9)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.

Historical question-source excerpts are shown only when a page has a strong text-based unit match. They are source references, not new solutions or exam predictions.

Known gaps, conflicts and corrections

1. No historical archive question page met the unit-match candidate threshold; all subject-level and ambiguous pages remain in datasets/question-archive.json.
2. 3 transcript segments have low ASR confidence; they are flagged with timestamps in the transcript blocks and indexed in the corpus.
3. This is a visibly labelled source-preservation draft. Human page-level semantic and visual review is not complete; no unsupported subject summary or formula correction is claimed.

4. The 19.26 GiB MP4 files remain hosted on Hugging Face; the full video path/hash and transcript coverage are indexed. The full audio/video was not watched end-to-end.
5. Formula candidates are machine-extracted text with source locators; use the adjacent original slide page and the formula index before treating a candidate as an authoritative equation.

Coverage

Every inventoried source unit has an explicit coverage decision. Covered means reviewed and mapped to content; exclusions and gaps remain visible.

Evidence	Source unit	Status	Decision
EV00000001	SRC00001/catalogue	covered	Unit title and index copied from the official units.json catalogue; human content review is pending.
EV00000002	SRC00002/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000003	SRC00003/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000004	SRC00004/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000005	SRC00005/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000006	SRC00002/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0023 to s0040. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000007	SRC00003/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0023 to s0040. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000008	SRC00004/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0023 to s0040. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000009	SRC00005/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0023 to s0040. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000010	SRC00002/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0041 to s0061. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000011	SRC00003/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0041 to s0061. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000012	SRC00004/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0041 to s0061. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000013	SRC00005/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0041 to s0061. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000014	SRC00002/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0062 to s0081. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000015	SRC00003/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0062 to s0081. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000016	SRC00004/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0062 to s0081. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000017	SRC00005/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0062 to s0081. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000018	SRC00002/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0082 to s0099. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000019	SRC00003/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0082 to s0099. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000020	SRC00004/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0082 to s0099. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000021	SRC00005/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0082 to s0099. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000022	SRC00002/t001500-001799	covered	Timestamp interval t001500-001799; canonical segment IDs s0100 to s0100. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000023	SRC00003/t001500-001799	covered	Timestamp interval t001500-001799; canonical segment IDs s0100 to s0100. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000024	SRC00004/t001500-001799	covered	Timestamp interval t001500-001799; canonical segment IDs s0100 to s0100. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000025	SRC00005/t001500-001799	covered	Timestamp interval t001500-001799; canonical segment IDs s0100 to s0100. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000026	SRC00006/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000027	SRC00007/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV00000028	SRC00008/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000029	SRC00009/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000030	SRC00006/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0019 to s0037. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000031	SRC00007/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0019 to s0037. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000032	SRC00008/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0019 to s0037. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000033	SRC00009/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0019 to s0037. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000034	SRC00006/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0038 to s0056. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000035	SRC00007/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0038 to s0056. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000036	SRC00008/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0038 to s0056. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000037	SRC00009/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0038 to s0056. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000038	SRC00006/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0057 to s0079. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000039	SRC00007/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0057 to s0079. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000040	SRC00008/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0057 to s0079. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000041	SRC00009/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0057 to s0079. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000042	SRC00006/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0080 to s0084. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000043	SRC00007/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0080 to s0084. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000044	SRC00008/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0080 to s0084. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000045	SRC00009/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0080 to s0084. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000046	SRC00010/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000047	SRC00011/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000048	SRC00012/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV00000049	SRC00013/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000050	SRC00010/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000051	SRC00011/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000052	SRC00012/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000053	SRC00013/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000054	SRC00010/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0041 to s0063. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000055	SRC00011/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0041 to s0063. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000056	SRC00012/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0041 to s0063. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000057	SRC00013/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0041 to s0063. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000058	SRC00010/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0064 to s0084. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000059	SRC00011/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0064 to s0084. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000060	SRC00012/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0064 to s0084. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000061	SRC00013/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0064 to s0084. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000062	SRC00010/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0085 to s0104. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000063	SRC00011/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0085 to s0104. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000064	SRC00012/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0085 to s0104. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000065	SRC00013/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0085 to s0104. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000066	SRC00014/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000067	SRC00015/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000068	SRC00016/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000069	SRC00017/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000070	SRC00014/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000071	SRC00015/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000072	SRC00016/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000073	SRC00017/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000074	SRC00014/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0041 to s0060. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000075	SRC00015/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0041 to s0060. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000076	SRC00016/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0041 to s0060. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000077	SRC00017/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0041 to s0060. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000078	SRC00018/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000079	SRC00020/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000080	SRC00019/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000081	SRC00018/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000082	SRC00020/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.pdf; visual page included as vector figure. Human layout/content review is pending.

Evidence	Source unit	Status	Decision
EV0000083	SRC00019/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.
EV0000084	SRC00018/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000085	SRC00020/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000086	SRC00019/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000087	SRC00018/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000088	SRC00020/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000089	SRC00019/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000090	SRC00018/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000091	SRC00020/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000092	SRC00019/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0000093	SRC00018/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000094	SRC00020/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000095	SRC00019/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000096	SRC00018/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000097	SRC00020/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000098	SRC00019/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV0000099	SRC00018/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000100	SRC00020/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000101	SRC00019/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000102	SRC00018/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000103	SRC00020/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000104	SRC00019/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.
EV0000105	SRC00018/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000106	SRC00020/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000107	SRC00019/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000108	SRC00018/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000109	SRC00020/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000110	SRC00019/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.
EV0000111	SRC00018/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000112	SRC00020/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000113	SRC00019/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000114	SRC00018/p13	covered	Extracted slide page 13 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000115	SRC00020/p13	covered	Original slide page 13 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000116	SRC00019/p13	covered	Alternate extracted slide text page 13; normalized text matches the .slides.md page. Human comparison is pending.
EV0000117	SRC00018/p14	covered	Extracted slide page 14 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000118	SRC00020/p14	covered	Original slide page 14 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000119	SRC00019/p14	covered	Alternate extracted slide text page 14; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000120	SRC00018/p15	covered	Extracted slide page 15 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000121	SRC00020/p15	covered	Original slide page 15 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000122	SRC00019/p15	covered	Alternate extracted slide text page 15; normalized text matches the .slides.md page. Human comparison is pending.
EV0000123	SRC00018/p16	covered	Extracted slide page 16 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000124	SRC00020/p16	covered	Original slide page 16 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/029_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__3_tuples__224e54b49870463d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000125	SRC00019/p16	covered	Alternate extracted slide text page 16; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00001126	SRC00021/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00001127	SRC00023/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00001128	SRC00022/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV00001129	SRC00021/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00001130	SRC00023/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00001131	SRC00022/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000132	SRC00021/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000133	SRC00023/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000134	SRC00022/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000135	SRC00021/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000136	SRC00023/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000137	SRC00022/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000138	SRC00021/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000139	SRC00023/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000140	SRC00022/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0000141	SRC00021/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000142	SRC00023/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000143	SRC00022/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000144	SRC00021/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000145	SRC00023/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000146	SRC00022/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV0000147	SRC00021/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000148	SRC00023/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000149	SRC00022/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000150	SRC00021/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000151	SRC00023/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000152	SRC00022/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.
EV0000153	SRC00021/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000154	SRC00023/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000155	SRC00022/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000156	SRC00021/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000157	SRC00023/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000158	SRC00022/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.
EV0000159	SRC00021/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000160	SRC00023/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000161	SRC00022/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000162	SRC00021/p13	covered	Extracted slide page 13 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000163	SRC00023/p13	covered	Original slide page 13 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000164	SRC00022/p13	covered	Alternate extracted slide text page 13; normalized text matches the .slides.md page. Human comparison is pending.
EV0000165	SRC00021/p14	covered	Extracted slide page 14 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000166	SRC00023/p14	covered	Original slide page 14 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000167	SRC00022/p14	covered	Alternate extracted slide text page 14; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000168	SRC00021/p15	covered	Extracted slide page 15 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000169	SRC00023/p15	covered	Original slide page 15 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000170	SRC00022/p15	covered	Alternate extracted slide text page 15; normalized text matches the .slides.md page. Human comparison is pending.
EV0000171	SRC00021/p16	covered	Extracted slide page 16 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000172	SRC00023/p16	covered	Original slide page 16 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/034_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__5_Dictionary__6034ce750573f32b.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000173	SRC00022/p16	covered	Alternate extracted slide text page 16; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000174	SRC00024/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000175	SRC00026/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000176	SRC00025/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000177	SRC00024/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000178	SRC00026/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000179	SRC00025/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000180	SRC00024/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000181	SRC00026/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000182	SRC00025/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000183	SRC00024/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000184	SRC00026/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000185	SRC00025/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00001186	SRC00024/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00001187	SRC00026/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00001188	SRC00025/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV00001189	SRC00024/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00001190	SRC00026/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00001191	SRC00025/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000192	SRC00024/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000193	SRC00026/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000194	SRC00025/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV0000195	SRC00024/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000196	SRC00026/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000197	SRC00025/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000198	SRC00024/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000199	SRC00026/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000200	SRC00025/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.
EV0000201	SRC00024/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000202	SRC00026/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000203	SRC00025/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000204	SRC00024/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000205	SRC00026/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000206	SRC00025/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.
EV0000207	SRC00024/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000208	SRC00026/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000209	SRC00025/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000210	SRC00024/p13	covered	Extracted slide page 13 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000211	SRC00026/p13	covered	Original slide page 13 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000212	SRC00025/p13	covered	Alternate extracted slide text page 13; normalized text matches the .slides.md page. Human comparison is pending.
EV0000213	SRC00024/p14	covered	Extracted slide page 14 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000214	SRC00026/p14	covered	Original slide page 14 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000215	SRC00025/p14	covered	Alternate extracted slide text page 14; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000216	SRC00024/p15	covered	Extracted slide page 15 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000217	SRC00026/p15	covered	Original slide page 15 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000218	SRC00025/p15	covered	Alternate extracted slide text page 15; normalized text matches the .slides.md page. Human comparison is pending.
EV0000219	SRC00024/p16	covered	Extracted slide page 16 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000220	SRC00026/p16	covered	Original slide page 16 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000221	SRC00025/p16	covered	Alternate extracted slide text page 16; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000222	SRC00024/p17	covered	Extracted slide page 17 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000223	SRC00026/p17	covered	Original slide page 17 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000224	SRC00025/p17	covered	Alternate extracted slide text page 17; normalized text matches the .slides.md page. Human comparison is pending.
EV0000225	SRC00024/p18	covered	Extracted slide page 18 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000226	SRC00026/p18	covered	Original slide page 18 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000227	SRC00025/p18	covered	Alternate extracted slide text page 18; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000228	SRC00024/p19	covered	Extracted slide page 19 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000229	SRC00026/p19	covered	Original slide page 19 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000230	SRC00025/p19	covered	Alternate extracted slide text page 19; normalized text matches the .slides.md page. Human comparison is pending.
EV0000231	SRC00024/p20	covered	Extracted slide page 20 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000232	SRC00026/p20	covered	Original slide page 20 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000233	SRC00025/p20	covered	Alternate extracted slide text page 20; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000234	SRC00024/p21	covered	Extracted slide page 21 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000235	SRC00026/p21	covered	Original slide page 21 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000236	SRC00025/p21	covered	Alternate extracted slide text page 21; normalized text matches the .slides.md page. Human comparison is pending.
EV0000237	SRC00024/p22	covered	Extracted slide page 22 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000238	SRC00026/p22	covered	Original slide page 22 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000239	SRC00025/p22	covered	Alternate extracted slide text page 22; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000240	SRC00024/p23	covered	Extracted slide page 23 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000241	SRC00026/p23	covered	Original slide page 23 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000242	SRC00025/p23	covered	Alternate extracted slide text page 23; normalized text matches the .slides.md page. Human comparison is pending.
EV0000243	SRC00024/p24	covered	Extracted slide page 24 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000244	SRC00026/p24	covered	Original slide page 24 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/036_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__7_sets__443afd074f4cc8c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000245	SRC00025/p24	covered	Alternate extracted slide text page 24; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000246	SRC00027/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000247	SRC00029/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000248	SRC00028/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000249	SRC00027/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000250	SRC00029/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000251	SRC00028/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000252	SRC00027/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000253	SRC00029/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000254	SRC00028/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000255	SRC00027/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000256	SRC00029/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000257	SRC00028/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000258	SRC00027/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000259	SRC00029/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000260	SRC00028/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0000261	SRC00027/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000262	SRC00029/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000263	SRC00028/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000264	SRC00027/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000265	SRC00029/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000266	SRC00028/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV0000267	SRC00027/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000268	SRC00029/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000269	SRC00028/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00002 70	SRC00027/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00002 71	SRC00029/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00002 72	SRC00028/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.
EV00002 73	SRC00027/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00002 74	SRC00029/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00002 75	SRC00028/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000276	SRC00027/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000277	SRC00029/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000278	SRC00028/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.
EV0000279	SRC00027/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000280	SRC00029/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000281	SRC00028/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000282	SRC00027/p13	covered	Extracted slide page 13 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000283	SRC00029/p13	covered	Original slide page 13 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000284	SRC00028/p13	covered	Alternate extracted slide text page 13; normalized text matches the .slides.md page. Human comparison is pending.
EV0000285	SRC00027/p14	covered	Extracted slide page 14 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000286	SRC00029/p14	covered	Original slide page 14 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000287	SRC00028/p14	covered	Alternate extracted slide text page 14; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000288	SRC00027/p15	covered	Extracted slide page 15 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000289	SRC00029/p15	covered	Original slide page 15 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000290	SRC00028/p15	covered	Alternate extracted slide text page 15; normalized text matches the .slides.md page. Human comparison is pending.
EV0000291	SRC00027/p16	covered	Extracted slide page 16 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000292	SRC00029/p16	covered	Original slide page 16 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000293	SRC00028/p16	covered	Alternate extracted slide text page 16; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000294	SRC00027/p17	covered	Extracted slide page 17 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000295	SRC00029/p17	covered	Original slide page 17 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000296	SRC00028/p17	covered	Alternate extracted slide text page 17; normalized text matches the .slides.md page. Human comparison is pending.
EV0000297	SRC00027/p18	covered	Extracted slide page 18 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000298	SRC00029/p18	covered	Original slide page 18 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000299	SRC00028/p18	covered	Alternate extracted slide text page 18; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000300	SRC00027/p19	covered	Extracted slide page 19 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000301	SRC00029/p19	covered	Original slide page 19 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000302	SRC00028/p19	covered	Alternate extracted slide text page 19; normalized text matches the .slides.md page. Human comparison is pending.
EV0000303	SRC00027/p20	covered	Extracted slide page 20 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000304	SRC00029/p20	covered	Original slide page 20 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/046_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__12_13_14_File_handling_text_files_CSE_files__62c9b9d6e05b82d2.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000305	SRC00028/p20	covered	Alternate extracted slide text page 20; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000306	SRC00030/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000307	SRC00032/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000308	SRC00031/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000309	SRC00030/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000310	SRC00032/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000311	SRC00031/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.
EV0000312	SRC00030/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000313	SRC00032/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000314	SRC00031/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000315	SRC00030/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000316	SRC00032/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000317	SRC00031/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.
EV0000318	SRC00030/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000319	SRC00032/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.pdf; visual page included as vector figure. Human layout/content review is pending.

Evidence	Source unit	Status	Decision
EV0000320	SRC00031/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0000321	SRC00030/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000322	SRC00032/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000323	SRC00031/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.
EV0000324	SRC00030/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000325	SRC00032/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000326	SRC00031/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00003327	SRC00030/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00003328	SRC00032/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00003329	SRC00031/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.
EV00003330	SRC00030/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00003331	SRC00032/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00003332	SRC00031/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.
EV00003333	SRC00030/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000334	SRC00032/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000335	SRC00031/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.
EV0000336	SRC00030/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000337	SRC00032/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000338	SRC00031/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.
EV0000339	SRC00030/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000340	SRC00032/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.pdf; visual page included as vector figure. Human layout/content review is pending.

Evidence	Source unit	Status	Decision
EV0000341	SRC00031/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.
EV0000342	SRC00030/p13	covered	Extracted slide page 13 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000343	SRC00032/p13	covered	Original slide page 13 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/050_JSON__ebbb542771842bb1.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000344	SRC00031/p13	covered	Alternate extracted slide text page 13; normalized text matches the .slides.md page. Human comparison is pending.
EV0000345	SRC00033/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000346	SRC00035/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000347	SRC00034/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000348	SRC00033/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000349	SRC00035/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000350	SRC00034/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.
EV0000351	SRC00033/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000352	SRC00035/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000353	SRC00034/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000354	SRC00033/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000355	SRC00035/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000356	SRC00034/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.
EV0000357	SRC00033/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000358	SRC00035/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000359	SRC00034/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00003360	SRC00033/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00003361	SRC00035/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00003362	SRC00034/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.
EV00003363	SRC00033/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00003364	SRC00035/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00003365	SRC00034/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00003366	SRC00033/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00003367	SRC00035/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00003368	SRC00034/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.
EV00003369	SRC00033/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00003370	SRC00035/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00003371	SRC00034/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000372	SRC00033/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000373	SRC00035/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000374	SRC00034/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.
EV0000375	SRC00033/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000376	SRC00035/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000377	SRC00034/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00003378	SRC00033/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00003379	SRC00035/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00003380	SRC00034/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.
EV00003381	SRC00033/p13	covered	Extracted slide page 13 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00003382	SRC00035/p13	covered	Original slide page 13 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00003383	SRC00034/p13	covered	Alternate extracted slide text page 13; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000384	SRC00033/p14	covered	Extracted slide page 14 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000385	SRC00035/p14	covered	Original slide page 14 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000386	SRC00034/p14	covered	Alternate extracted slide text page 14; normalized text matches the .slides.md page. Human comparison is pending.
EV0000387	SRC00033/p15	covered	Extracted slide page 15 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000388	SRC00035/p15	covered	Original slide page 15 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000389	SRC00034/p15	covered	Alternate extracted slide text page 15; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000390	SRC00033/p16	covered	Extracted slide page 16 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000391	SRC00035/p16	covered	Original slide page 16 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000392	SRC00034/p16	covered	Alternate extracted slide text page 16; normalized text matches the .slides.md page. Human comparison is pending.
EV0000393	SRC00033/p17	covered	Extracted slide page 17 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000394	SRC00035/p17	covered	Original slide page 17 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000395	SRC00034/p17	covered	Alternate extracted slide text page 17; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00003 96	SRC00033/p18	covered	Extracted slide page 18 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00003 97	SRC00035/p18	covered	Original slide page 18 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00003 98	SRC00034/p18	covered	Alternate extracted slide text page 18; normalized text matches the .slides.md page. Human comparison is pending.
EV00003 99	SRC00033/p19	covered	Extracted slide page 19 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00004 00	SRC00035/p19	covered	Original slide page 19 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00004 01	SRC00034/p19	covered	Alternate extracted slide text page 19; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000402	SRC00033/p20	covered	Extracted slide page 20 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000403	SRC00035/p20	covered	Original slide page 20 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000404	SRC00034/p20	covered	Alternate extracted slide text page 20; normalized text matches the .slides.md page. Human comparison is pending.
EV0000405	SRC00033/p21	covered	Extracted slide page 21 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000406	SRC00035/p21	covered	Original slide page 21 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/051_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__15_function_definition_call__cdc703c5984aeb67.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000407	SRC00034/p21	covered	Alternate extracted slide text page 21; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000408	SRC00036/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000409	SRC00038/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000410	SRC00037/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000411	SRC00036/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000412	SRC00038/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000413	SRC00037/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000414	SRC00036/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000415	SRC00038/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000416	SRC00037/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000417	SRC00036/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000418	SRC00038/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000419	SRC00037/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000420	SRC00036/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000421	SRC00038/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000422	SRC00037/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0000423	SRC00036/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000424	SRC00038/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000425	SRC00037/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000426	SRC00036/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000427	SRC00038/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000428	SRC00037/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV0000429	SRC00036/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000430	SRC00038/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/052_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__16_function_positional_keyword_args__4822af8d67cde0e0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000431	SRC00037/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000432	SRC00039/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000433	SRC00041/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000434	SRC00040/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000435	SRC00039/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000436	SRC00041/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000437	SRC00040/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000438	SRC00039/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000439	SRC00041/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000440	SRC00040/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000441	SRC00039/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000442	SRC00041/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000443	SRC00040/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000444	SRC00039/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000445	SRC00041/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000446	SRC00040/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0000447	SRC00039/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000448	SRC00041/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000449	SRC00040/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00004 50	SRC00039/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00004 51	SRC00041/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00004 52	SRC00040/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV00004 53	SRC00039/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00004 54	SRC00041/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00004 55	SRC00040/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000456	SRC00039/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000457	SRC00041/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000458	SRC00040/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.
EV0000459	SRC00039/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000460	SRC00041/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000461	SRC00040/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000462	SRC00039/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000463	SRC00041/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000464	SRC00040/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.
EV0000465	SRC00039/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000466	SRC00041/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000467	SRC00040/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000468	SRC00039/p13	covered	Extracted slide page 13 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000469	SRC00041/p13	covered	Original slide page 13 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000470	SRC00040/p13	covered	Alternate extracted slide text page 13; normalized text matches the .slides.md page. Human comparison is pending.
EV0000471	SRC00039/p14	covered	Extracted slide page 14 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000472	SRC00041/p14	covered	Original slide page 14 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/Slides/053_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__17_18_Variable_number_args_kw_args__e992709b22b58075.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000473	SRC00040/p14	covered	Alternate extracted slide text page 14; normalized text matches the .slides.md page. Human comparison is pending.

C Revision and retrieval practice